第 6 章 图

【考纲内容】

（一）图的基本概念

（二）图的存储及基本操作

邻接矩阵；邻接表；邻接多重表；十字链表

（三）图的遍历

深度优先搜索；广度优先搜索

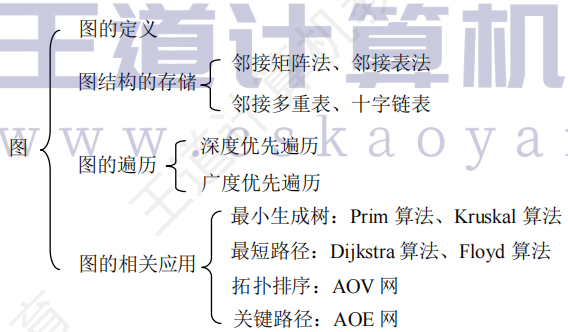
（四）图的基本应用

最小（代价）生成树；最短路径；拓扑排序；关键路径

扫一扫

视频讲解

【知识框架】



【复习提示】

图算法的难度较大，主要掌握深度优先搜索与广度优先搜索。掌握图的基本概念及基本性质、图的存储结构（邻接矩阵、邻接表、邻接多重表和十字链表）及特性、存储结构之间的转化、基于存储结构上的各种遍历操作和各种应用（拓扑排序、最小生成树、最短路径和关键路径）等。图的相关算法较多，通常只需掌握其基本思想和实现步骤，而实现代码不是重点。

6.1 图的基本概念

6.1.1 图的定义

图 G 由顶点集 V 和边集 E 组成，记为 $G=(V, E)$，其中 $V(G)$表示图 G 中顶点的有限非空集；$E(G)$表示图 G 中顶点之间的关系（边）集合。若 $V=\{v_1, v_2, \cdots, v_n\}$，则用$|V|$表示图 G 中顶点的个数，$E=\{(u, v) \mid u \in V, v \in V\}$，用$|E|$表示图 G 中边的条数。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

注　意

线性表可以是空表，树可以是空树，但图不可以是空图。也就是说，图中不能一个顶点也没有，图的顶点集 V 一定非空，但边集 E 可以为空，此时图中只有顶点而没有边。

下面是图的一些基本概念及术语。

1．有向图

若 E 是有向边（也称弧）的有限集合，则图 G 为有向图。弧是顶点的有序对，记为<v, w>，其中 v, w 是顶点，v 称为弧尾，w 称为弧头，<v, w>称为从 v 到 w 的弧，也称 v 邻接到 w。

图 6.1(a)所示的有向图 G_1 可表示为

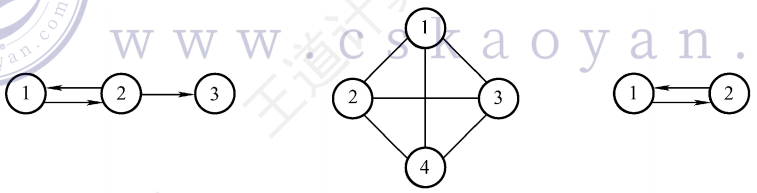
$$G_1 = (V_1, E_1)$$
$$V_1 = \{1, 2, 3\}$$
$$E_1 = \{<1, 2>, <2, 1>, <2, 3>\}$$

2．无向图

若 E 是无向边（简称边）的有限集合，则图 G 为无向图。边是顶点的无序对，记为(v, w)或(w, v)。可以说 w 和 v 互为邻接点。边(v, w)依附于 w 和 v，或称边(v, w)和 v, w 相关联。

图 6.1(b)所示的无向图 G_2 可表示为

$$G_2 = (V_2, E_2)$$
$$V_2 = \{1, 2, 3, 4\}$$
$$E_2 = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$$



(a) 有向图G_1　(b) 无向图G_2　(c) 有向完全图G_3

图 6.1　图的示例

3．简单图、多重图

一个图 G 若满足：①不存在重复边；②不存在顶点到自身的边，则称图 G 为简单图。图 6.1 中 G_1 和 G_2 均为简单图。若图 G 中某两个顶点之间的边数大于 1 条，又允许顶点通过一条边和自身关联，则称图 G 为多重图。多重图和简单图的定义是相对的。本书中仅讨论简单图。

4．顶点的度、入度和出度

命题追踪 ▶▶ 无向图中顶点和边的关系（2009、2017）

在无向图中，顶点 v 的度是指依附于顶点 v 的边的条数，记为 TD(v)。在图 6.1(b)中，每个顶点的度均为 3。无向图的全部顶点的度之和等于边数的 2 倍，因为每条边和两个顶点相关联。

在有向图中，顶点 v 的度分为入度和出度，入度是以顶点 v 为终点的有向边的数目，记为 ID(v)；而出度是以顶点 v 为起点的有向边的数目，记为 OD(v)。在图 6.1(a)中，顶点 2 的出度为 2、入度为 1。顶点 v 的度等于其入度与出度之和，即 TD(v) = ID(v) + OD(v)。有向图的全部顶点的入度之和与出度之和相等，并且等于边数，这是因为每条有向边都有一个起点和终点。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

5．路径、路径长度和回路

顶点 v_p 到顶点 v_q 之间的一条路径是指顶点序列 $v_p, v_{i_1}, v_{i_2}, \cdots, v_{i_m}, v_q$，当然关联的边也可理解为路径的构成要素。路径上的边的数目称为路径长度。第一个顶点和最后一个顶点相同的路径称为回路或环。若一个图有 n 个顶点，且有大于 $n-1$ 条边，则此图一定有环。

6．简单路径、简单回路

命题追踪 ▶ 路径、回路、简单路径、简单回路的定义（2011）

在路径序列中，顶点不重复出现的路径称为简单路径。除第一个顶点和最后一个顶点外，其余顶点不重复出现的回路称为简单回路。

7．距离

从顶点 u 出发到顶点 v 的最短路径若存在，则此路径的长度称为从 u 到 v 的距离。若从 u 到 v 根本不存在路径，则记该距离为无穷（∞）。

8．子图

设有两个图 $G=(V, E)$ 和 $G'=(V', E')$，若 V' 是 V 的子集，且 E' 是 E 的子集，则称 G' 是 G 的子图。若有满足 $V(G')=V(G)$ 的子图 G'，则称其为 G 的生成子图。图 6.1 中 G_3 为 G_1 的子图。

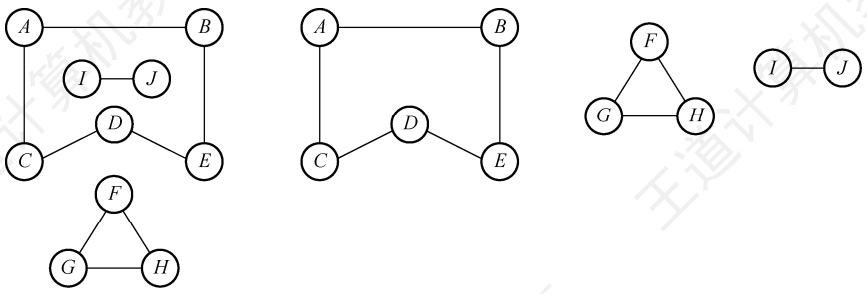
注 意

并非 V 和 E 的任何子集都能构成 G 的子图，因为这样的子集可能不是图，即 E 的子集中的某些边关联的顶点可能不在这个 V 的子集中。

9．连通、连通图和连通分量

命题追踪 ▶ 图的连通性与边和顶点的关系（2010、2022）

在无向图中，若从顶点 v 到顶点 w 有路径存在，则称 v 和 w 是连通的。若图 G 中任意两个顶点都是连通的，则称图 G 为连通图，否则称为非连通图。无向图中的极大连通子图称为连通分量，在图 6.2(a)中，图 G_4 有 3 个连通分量如图 6.2(b)所示。假设一个图有 n 个顶点，若边数小于 $n-1$，则此图必是非连通图；思考，若该图是非连通图，则最多可以有多少条边？[①]



(a) 无向图G_4　　(b) G_4的三个连通分量

图 6.2　无向图及其连通分量

10．强连通图、强连通分量

在有向图中，若有一对顶点 v 和 w，从 v 到 w 和从 w 到 v 之间都有路径，则称这两个顶点

① 非连通情况下边最多的情况：由 $n-1$ 个顶点构成一个完全图，此时再加入一个顶点则变成非连通图。

配套课程公众号[考机研]
永久联系微信:4550060

是强连通的。若图中任意一对顶点都是强连通的，则称此图为强连通图。有向图中的极大强连通子图称为有向图的强连通分量，图 G_1 的强连通分量如图 6.3 所示。思考，假设一个有向图有 n 个顶点，若该图是强连通图，则最少需要有多少条边？①

注　意

在无向图中讨论连通性，在有向图中讨论强连通性。

11．生成树、生成森林

连通图的生成树是包含图中全部顶点的一个极小连通子图。若图中顶点数为 n，则它的生成树含有 $n-1$ 条边。包含图中全部顶点的极小连通子图，只有生成树满足这个极小条件，对生成树而言，若砍去它的一条边，则会变成非连通图，若加上一条边则会形成一个回路。在非连通图中，连通分量的生成树构成了非连通图的生成森林。图 G_2 的一个生成树如图 6.4 所示。

图 6.3　图 G_1 的强连通分量

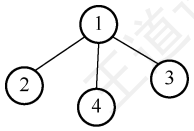


图 6.4　图 G_2 的一个生成树

注　意

区分极大连通子图和极小连通子图。极大连通子图要求子图必须连通，而且包含尽可能多的顶点和边；极小连通子图是既要保持子图连通又要使得边数最少的子图。

12．边的权、网和带权路径长度

在一个图中，每条边都可以标上具有某种含义的数值，该数值称为该边的权值。这种边上带有权值的图称为带权图，也称网。路径上所有边的权值之和，称为该路径的带权路径长度。

13．完全图（也称简单完全图）

对于无向图，$|E|$的取值范围为 0 到 $n(n-1)/2$，有 $n(n-1)/2$ 条边的无向图称为完全图，在完全图中任意两个顶点之间都存在边。对于有向图，$|E|$的取值范围为 0 到 $n(n-1)$，有 $n(n-1)$条弧的有向图称为有向完全图，在有向完全图中任意两个顶点之间都存在方向相反的两条弧。图 6.1 中 G_2 为无向完全图，而 G_3 为有向完全图。

14．稠密图、稀疏图

边数很少的图称为稀疏图，反之称为稠密图。稀疏和稠密本身是模糊的概念，稀疏图和稠密图常常是相对而言的。一般当图 G 满足$|E|<|V|\log_2|V|$时，可以将 G 视为稀疏图。

15．有向树

一个顶点的入度为 0、其余顶点的入度均为 1 的有向图，称为有向树。

6.1.2　本节试题精选

一、单项选择题

01. 图中有关路径的定义是（　）。

① 有向图强连通情况下边最少的情况：至少需要 n 条边，构成一个环路。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

A. 由顶点和相邻顶点序偶构成的边所形成的序列
B. 由不同顶点所形成的序列
C. 由不同边所形成的序列
D. 上述定义都不是

02. 一个有 n 个顶点和 n 条边的无向图一定是（ ）。
A. 连通的　　B. 不连通的　　C. 无环的　　D. 有环的

03. 若从无向图的任意顶点出发进行一次深度优先搜索即可访问所有顶点，则该图一定是（ ）。
A. 强连通图　　B. 连通图　　C. 有回路　　D. 一棵树

04. 以下关于图的叙述中，正确的是（ ）。
A. 图与树的区别在于图的边数大于或等于顶点数
B. 假设有图 $G=\{V,\{E\}\}$，顶点集 $V'\subseteq V$，$E'\subseteq E$，则 V'和$\{E'\}$构成 G 的子图
C. 无向图的连通分量是指无向图中的极大连通子图
D. 图的遍历就是从图中某一顶点出发访遍图中其余顶点

05. 以下关于图的叙述中，正确的是（ ）。
A. 强连通有向图的任何顶点到其他所有顶点都有弧
B. 图的任意顶点的入度等于出度
C. 有向完全图一定是强连通有向图
D. 有向图的边集的子集和顶点集的子集都构成原有向图的子图

06. 一个有 28 条边的非连通无向图至少有（ ）个顶点。
A. 7　　B. 8　　C. 9　　D. 10

07. 对于一个有 n 个顶点的图：若是连通无向图，其边的个数至少为（ ）；若是强连通有向图，则其边的个数至少为（ ）。
A. $n-1,n$　　B. $n-1,n(n-1)$　　C. n,n　　D. $n,n(n-1)$

08. 无向图 G 有 23 条边，度为 4 的顶点有 5 个，度为 3 的顶点有 4 个，其余都是度为 2 的顶点，则图 G 有（ ）个顶点。
A. 11　　B. 12　　C. 15　　D. 16

09. 在有 n 个顶点的有向图中，顶点的度最大可达（ ）。
A. n　　B. $n-1$　　C. $2n$　　D. $2n-2$

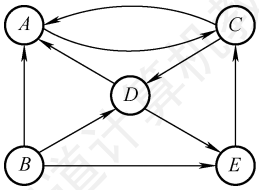
10. 具有 6 个顶点的无向图，当有（ ）条边时能确保是一个连通图。
A. 8　　B. 9　　C. 10　　D. 11

11. 设有无向图 $G=(V,E)$和 $G'=(V',E')$，若 G'是 G 的生成树，则下列不正确的是（ ）。
I. G'为 G 的连通分量
II. G'为 G 的无环子图
III. G'为 G 的极小连通子图且 $V'=V$
A. I、II　　B. 只有 III　　C. II、III　　D. 只有 I

12. 具有 51 个顶点和 21 条边的无向图的连通分量最多为（ ）。
A. 33　　B. 34　　C. 45　　D. 32

13. 在如下图所示的有向图中，共有（ ）个强连通分量。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



A. 1　　B. 2　　C. 3　　D. 4

14. 若具有 n 个顶点的图是一个环，则它有（ ）棵生成树。

A. n^2　　B. n　　C. $n-1$　　D. 1

15. 若一个具有 n 个顶点、e 条边的无向图是一个森林，则该森林中必有（ ）棵树。

A. n　　B. e　　C. $n-e$　　D. 1

16. 【2009 统考真题】下列关于无向连通图特性的叙述中，正确的是（ ）。

I. 所有顶点的度之和为偶数

II. 边数大于顶点个数减 1

III. 至少有一个顶点的度为 1

A. 只有 I　　B. 只有 II　　C. I 和 II　　D. I 和 III

17. 【2010 统考真题】若无向图 $G=(V, E)$中含有 7 个顶点，要保证图 G 在任何情况下都是连通的，则需要的边数最少是（ ）。

A. 6　　B. 15　　C. 16　　D. 21

18. 【2017 统考真题】已知无向图 G 含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其他顶点的度均小于 3。图 G 所含的顶点个数至少是（ ）。

A. 10　　B. 11　　C. 13　　D. 15

19. 【2022 统考真题】对于无向图 $G=(V, E)$，下列选项中，正确的是（ ）。

A. 当$|V|>|E|$时，G 一定是连通的

B. 当$|V|<|E|$时，G 一定是连通的

C. 当$|V|=|E|-1$ 时，G 一定是不连通的

D. 当$|V|>|E|+1$ 时，G 一定是不连通的

二、综合应用题

01. 图 G 是一个非连通无向图，共有 28 条边，该图至少有多少个顶点？

6.1.3　答案与解析

一、单项选择题

01. A

本题是北京交通大学考研真题，不同教材对路径的定义可能略有不同，顶点之间关联的边也可理解为路径的构成要素。对于选项 B，路径的定义中并没有要求是不同顶点，比如简单回路的第一个顶点和最后一个顶点是可以相同的，此外选项 B 也没有说明这些顶点之间有边相联。

02. D

若一个无向图有 n 个顶点和 $n-1$ 条边，可以使它连通但没有环（生成树），但若再加一条边，在不考虑重边的情形下，则必然会构成环。

03. B

强连通图是有向图，与题意矛盾，选项 A 错误；对无向连通图做一次深度优先搜索，可以访问到该连通图的所有顶点，选项 B 正确；有回路的无向图不一定是连通图，因为回路不一定

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

包含图的所有结点，选项C错误；连通图可能是树，也可能存在环，选项D错误。

04．C

图与树的区别是逻辑上的区别，而不是边数的区别，图的边数也可能小于树的边数，选项A错误；若E'中的边对应的顶点不是V'的元素，V'和$\{E'\}$无法构成图，选项B错误；无向图的极大连通子图称为连通分量，选项C正确；图的遍历要求每个结点只能被访问一次，且若图非连通，则从某一顶点出发无法访问到其他全部顶点，选项D的说法不准确。

05．C

强连通有向图的任何顶点到其他所有顶点都有路径，但未必有弧；无向图任意顶点的入度等于出度，但有向图未必满足；若边集中的某条边对应的某个顶点不在对应的顶点集中，则有向图的边集的子集和顶点集的子集无法构成子图。

06．C

考查至少有多少个顶点的情形，我们考虑该非连通图最极端的情况，即它由一个完全图加一个独立的顶点构成，此时若再加一条边，则必然使图变成连通图。在$28 = n(n-1)/2 = 8\times7/2$条边的完全无向图中，总共有8个顶点，再加上1个不连通的顶点，共9个顶点。

07．A

对于连通无向图，边最少即构成一棵树的情形；对于强连通有向图，边最少即构成一个有向环的情形。

08．D

因为在具有n个顶点、e条边的无向图中，有$\sum_{i=1}^{n}\mathrm{TD}(v_i)=2e$，所以求得度为2的顶点数为7，从而共有16个顶点。

09．D

在有向图中，顶点的度等于入度与出度之和。n个顶点的有向图中，任意一个顶点最多还可以与其他$n-1$个顶点有一对指向相反的边相连。注意，数据结构中仅讨论简单图。

10．D

5个顶点构成一个完全无向图，需要$n(n-1)/2 = 10$条边；再加上1条边后，能保证第6个顶点必然与此完全无向图构成一个连通图，所以共需11条边。

11．D

一个连通图的生成树是一个极小连通子图，显然它是无环的，因此选项II、III正确。极大连通子图称为连通分量，G'连通但非连通分量。这里再补充一下“极大连通子图”：若图本来就是连通的，且每个子部分包含其本身的所有顶点和边，则它就是极大连通子图。

12．C

初始考虑只有51个顶点的无向图G，此时G中每个顶点都是连通分量，问题转化为向G中添加21条边，如何添加这21条边使得连通分量数目最多。若向两个不同的连通分量之间添加边，则连通分量数目会减1，所以应尽可能地将这21条边加入同一个连通分量且让其接近完全图，含有7个顶点的完全图有21条边，所以用7个顶点构成一个含有21条边的连通分量，剩下$51-7=44$个顶点对应44个连通分量，共有45个连通分量。

13．B

强连通分量是极大强连通子图，任意两个顶点之间有方向相反的两条路径。由定义不难得出，若一个顶点只有出边或入边，则该顶点必定单独构成一个连通分量。图中，顶点B只有出边，其他所有顶点都不可能有到顶点B的路径，所以顶点B单独构成一个强连通分量。在顶点A、C、D、E中，任意两个顶点之间都有方向相反的两条路径，所以可构成一个强连通分量。

14．B

王道计算机教育 www.wangdaojsjy.com
配套课程公众号[考机研]
永久联系微信:4550060

n 个顶点的生成树是具有 $n-1$ 条边的极小连通子图，因为 n 个顶点构成的环共有 n 条边，去掉任意一条边就是一棵生成树，所以共有 n 种情况，所以可以有 n 棵不同的生成树。

15．C

n 个结点的树有 $n-1$ 条边，假设森林中有 x 棵树，将每棵树的根连到一个添加的结点，则成为一棵树，结点数是 $n+1$，边数是 $e+x$，从而可知 $x=n-e$。

【另解】设森林中有 x 棵树，则再用 $x-1$ 条边就可将所有的树连接成一棵树，此时边数 + 1 = 顶点数，即 $e+(x-1)+1=n$，所以 $x=n-e$。

16．A

每条边都连接了两个顶点，在计算顶点的度之和时每条边都被计算了两次，所以所有顶点的度之和偶数。无向连通图对应的生成树也是无向连通图，但此时边数等于顶点数减 1，选项 II 错误。考虑 2 个或以上的顶点恰好构成一个环的情况，此时每个顶点的度都是 2，选项 III 错误。

17．C

题干要求无论如何分配边，都能使 7 个顶点连通，这不同于只要 6 条边两两相连就能构成一个连通图的情形。考虑最极端的情形，即图 G 的某 6 个顶点构成一个完全无向图，此时若再添加一条边，则都将连通第 7 个顶点，使该图变成一个连通图。所以最少边数 = 6×5/2 + 1 = 16。若边数 n 小于或等于 15，可以使这 n 条边仅连接图 G 中的某 6 个顶点，从而导致第 7 个顶点无法与这 6 个顶点构成连通图（不满足“在任何情况下”）。

为简单起见，以 5 个顶点为例，左边 4 个顶点和 4×3/2 = 6 条边构成一个完全图，此时若再添加一条边（可以是虚线中的任意一条），则能保证这 5 个顶点在任何情况下都是连通的，如下图所示。若边数小于 7，则不能保证 5 个顶点在任何情况下都是连通的。

王道计算机教育 www.cskaoyan.com

18．B

无向图边数的 2 倍等于各顶点度数的总和。要求至少的顶点数，应使每个顶点的度取最大，而其他顶点的度均小于 3，因此可设它们的度都为 2，并设它们的数量是 x，列出方程 4×3 + 3×4 + $2x$ = 16×2，解得 $x=4$。因此至少包含 4 + 4 + 3 = 11 个顶点。

19．D

对于此类分析图的边数、顶点数与连通性问题，思路是寻找临界情况，在临界情况下任意增加或减少一条边，都会改变图的连通性。第一种临界情况如图 1 所示，此时若减少任意一条边，图就由连通变为不连通，即无向图连通的最小边数是 $|V|-1$，因此，当 $|E|<|V|-1$ 时，图一定不连通，选项 C 错误，选项 D 正确。第二种临界情况如图 2 所示，此时若增加任意一条边，则图就由不连通变为连通，即无向图不连通的最大边数是 $(|V|-1)(|V|-2)/2$（此时 $|V|-1$ 个顶点构成一个完全图），因此，仅当 $|E|\geqslant(|V|-1)(|V|-2)/2+1$ 时才能保证无向图一定连通，选项 A、B 错误。

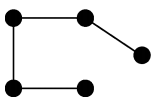


图 1

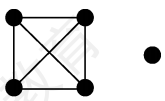


图 2

二、综合应用题

01．【解答】

图 G 是一个非连通无向图，当边数固定时，顶点数最少的情况是该图由两个连通子图构

配套课程公众号[考机研]
永久联系微信:4550060

成，且其中之一只含一个顶点，另一个为完全图。其中只含一个顶点的子图没有边，另一个完全图的边数为 $n(n-1)/2=28$，得 $n=8$。所以该图至少有 $1+8=9$ 个顶点。

6.2 图的存储及基本操作

图的存储必须要完整、准确地反映顶点集和边集的信息。根据不同图的结构和算法，采用不同的存储方式将对程序的效率产生相当大的影响，因此所选的存储结构应适合于待求解的问题。

6.2.1 邻接矩阵法

所谓邻接矩阵存储，是指用一个一维数组存储图中顶点的信息，用一个二维数组存储图中边的信息（各顶点之间的邻接关系），存储顶点之间邻接关系的二维数组称为邻接矩阵。

顶点数为 n 的图 $G=(V,E)$ 的邻接矩阵 $\boldsymbol{A}$ 是 $n\times n$ 的，将 G 的顶点编号为 $v_1,v_2,\cdots,v_n$，则

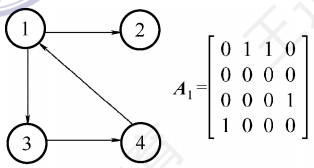
$$A[i][j]=\begin{cases}1, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\ 0, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$

命题追踪 ▶▶ 图的邻接矩阵存储及相互转换（2011、2015、2018）

对带权图而言，若顶点 v_i 和 v_j 之间有边相连，则邻接矩阵中对应项存放着该边对应的权值，若顶点 V_i 和 V_j 不相连，则通常用 0 或∞来代表这两个顶点之间不存在边：

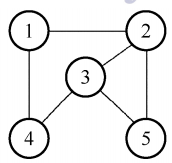
$$A[i][j]=\begin{cases}w_{ij}, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\ 0\text{或}\infty, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$

有向图、无向图和网对应的邻接矩阵示例如图 6.5 所示。



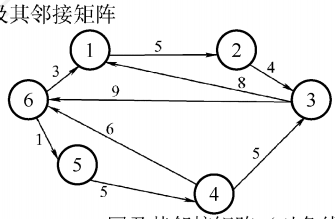
$$\boldsymbol{A}_1=\begin{bmatrix}0&1&1&0\\0&0&0&0\\0&0&0&1\\1&0&0&0\end{bmatrix}$$

(a) 有向图 G_1 及其邻接矩阵



$$\boldsymbol{A}_2=\begin{bmatrix}0&1&0&1&0\\1&0&1&0&1\\0&1&0&1&1\\1&0&1&0&0\\0&1&1&0&0\end{bmatrix}$$

(b) 无向图 G_2 及其邻接矩阵



$$\boldsymbol{A}_3=\begin{bmatrix}\infty&5&\infty&\infty&\infty&\infty\\\infty&\infty&4&\infty&\infty&\infty\\8&\infty&\infty&\infty&\infty&9\\\infty&\infty&5&\infty&\infty&6\\\infty&\infty&\infty&5&\infty&\infty\\3&\infty&\infty&\infty&1&\infty\end{bmatrix}$$

(c) 网及其邻接矩阵（对角线元素也经常用0表示）

图 6.5　有向图、无向图及网的邻接矩阵

命题追踪 ▶▶（算法题）邻接矩阵的遍历及顶点的度的计算（2021、2023）

图的邻接矩阵存储结构定义如下：

```
#define MaxVertexNum 100                    //顶点数目的最大值
typedef char VertexType;                    //顶点对应的数据类型
typedef int EdgeType;                       //边对应的数据类型
typedef struct{
    VertexType vex[MaxVertexNum];           //顶点表
    EdgeType edge[MaxVertexNum][MaxVertexNum];//邻接矩阵，边表
```

配套课程公众号[考机研]
永久联系微信:4550060

```
    int vexnum,arcnum;                    //图的当前顶点数和边数
}MGraph;
```

注　意

① 在简单应用中，可直接用二维数组作为图的邻接矩阵（顶点信息等均可省略）。
② 当邻接矩阵的元素仅表示相应边是否存在时，EdgeType 可用值为 0 和 1 的枚举类型。
③ 无向图的邻接矩阵是对称矩阵，对规模特大的邻接矩阵可采用压缩存储。
④ 邻接矩阵表示法的空间复杂度为 $O(n^2)$，其中 n 为图的顶点数$|V|$。

命题追踪 ▶▶ 邻接矩阵的遍历的时间复杂度（2021）

图的邻接矩阵存储表示法具有以下特点：

① 无向图的邻接矩阵一定是一个对称矩阵（并且唯一）。因此，在实际存储邻接矩阵时只需存储上（或下）三角矩阵的元素。

命题追踪 ▶▶ 基于邻接矩阵的顶点的度的计算（2013、2021、2023）

② 对于无向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非∞元素）的个数正好是顶点 i 的度 $TD(v_i)$。

③ 对于有向图，邻接矩阵的第 i 行非零元素（或非∞元素）的个数正好是顶点 i 的出度 $OD(v_i)$；第 i 列非零元素（或非∞元素）的个数正好是顶点 i 的入度 $ID(v_i)$。

④ 用邻接矩阵存储图，很容易确定图中任意两个顶点之间是否有边相连。但是，要确定图中有多少条边，则必须按行、按列对每个元素进行检测，所花费的时间代价很大。

⑤ 稠密图（边数较多的图）适合采用邻接矩阵的存储表示。

命题追踪 ▶▶ 计算 A^2 并说明 $A^n[i][j]$的含义（2015）

⑥ 设图 G 的邻接矩阵为 $\boldsymbol{A}$，$\boldsymbol{A}^n$ 的元素 $A^n[i][j]$等于由顶点 i 到顶点 j 的长度为 n 的路径的数目。该结论了解即可，证明方法可参考离散数学教材。

6.2.2　邻接表法

当一个图为稀疏图时，使用邻接矩阵法显然会浪费大量的存储空间，而图的邻接表法结合了顺序存储和链式存储方法，大大减少了这种不必要的浪费。

所谓邻接表，是指对图 G 中的每个顶点 v_i 建立一个单链表，第 i 个单链表中的结点表示依附于顶点 v_i 的边（对于有向图则是以顶点 v_i 为尾的弧），这个单链表就称为顶点 v_i 的边表（对于有向图则称为出边表）。边表的头指针和顶点的数据信息采用顺序存储，称为顶点表，所以在邻接表中存在两种结点：顶点表结点和边表结点，如图 6.6 所示。

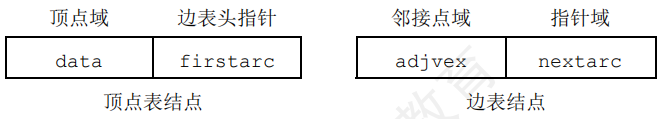


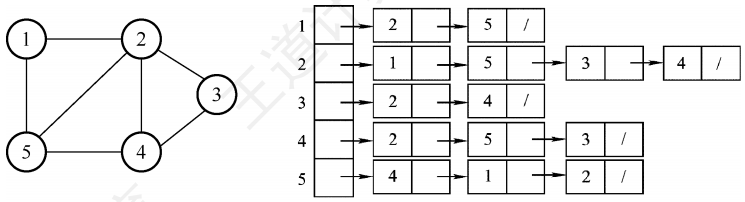
图 6.6　顶点表和边表结点结构

顶点表结点由两个域组成：顶点域（data）存储顶点 v_i 的相关信息，边表头指针域（firstarc）指向第一条边的边表结点。边表结点至少由两个域组成：邻接点域（adjvex）存储与头结点顶点 v_i 邻接的顶点编号，指针域（nextarc）指向下一条边的边表结点。

配套课程公众号[考机研]
永久联系微信:4550060

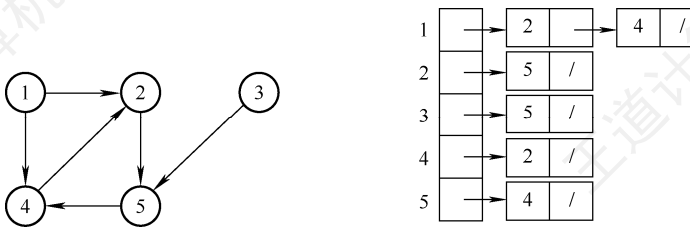
命题追踪 ▶▶ 图的邻接表存储的应用（2014）

无向图和有向图的邻接表的实例分别如图 6.7 和图 6.8 所示。



(a)无向图G　　(b)无向图G的邻接表的表示

图 6.7　无向图邻接表表示法实例



(a)有向图 G　　(b)有向图 G 的邻接表的表示

图 6.8　有向图邻接表表示法实例

图的邻接表存储结构定义如下：

```
#define MaxVertexNum 100        //图中顶点数目的最大值
typedef struct ArcNode{         //边表结点
    int adjvex;                 //该弧所指向的顶点的位置
    struct ArcNode *nextarc;    //指向下一条弧的指针
    //InfoType info;            //网的边权值
}ArcNode;
typedef struct VNode{           //顶点表结点
    VertexType data;            //顶点信息
    ArcNode *firstarc;          //指向第一条依附该顶点的弧的指针
}VNode,AdjList[MaxVertexNum];
typedef struct{
    AdjList vertices;           //邻接表
    int vexnum,arcnum;          //图的顶点数和弧数
}ALGraph;                       //ALGraph 是以邻接表存储的图类型
```

图的邻接表存储方法具有以下特点：

① 若 G 为无向图，则所需的存储空间为 $O(|V|+2|E|)$；若 G 为有向图，则所需的存储空间为 $O(|V|+|E|)$。前者的倍数 2 是因为在无向图中，每条边在邻接表中出现了两次。

命题追踪 ▶▶ 邻接矩阵法和邻接表法的适用性差异（2011）

② 对于稀疏图（边数较少的图），采用邻接表表示将极大地节省存储空间。

③ 在邻接表中，给定一个顶点，能很容易地找出它的所有邻边，因为只需要读取它的邻接表。在邻接矩阵中，相同的操作则需要扫描一行，花费的时间为 $O(n)$。但是，若要确定给定的两个顶点间是否存在边，则在邻接矩阵中可以立刻查到，而在邻接表中则需要在相应结点对应的边表中查找另一结点，效率较低。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

④ 在无向图的邻接表中，求某个顶点的度只需计算其邻接表中的边表结点个数。在有向图的邻接表中，求某个顶点的出度只需计算其邻接表中的边表结点个数；但求某个顶点 x 的入度则需遍历全部的邻接表，统计邻接点（adjvex）域为 x 的边表结点个数。

⑤ 图的邻接表表示并不唯一，因为在每个顶点对应的边表中，各边结点的链接次序可以是任意的，它取决于建立邻接表的算法及边的输入次序。

6.2.3　十字链表

十字链表是有向图的一种链式存储结构。在十字链表中，有向图的每条弧用一个结点（弧结点）来表示，每个顶点也用一个结点（顶点结点）来表示。两种结点的结构如下所示。

弧结点

tailvex	headvex	hlink	tlink	(info)

顶点结点

data	firstin	firstout

弧结点中有 5 个域：tailvex 域和 headvex 域分别存放弧尾和弧头这两个顶点的编号；头链域 hlink 指向弧头相同的下一条弧；尾链域 tlink 指向弧尾相同的下一条弧；info 域存放该弧的相关信息。这样，弧头相同的弧在同一个链表上，弧尾相同的弧也在同一个链表上。

顶点结点中有 3 个域：data 域存放该顶点的数据信息，如顶点名称；firstin 域指向以该顶点为弧头的第一条弧；firstout 域指向以该顶点为弧尾的第一条弧。

图 6.9 为有向图的十字链表表示法。

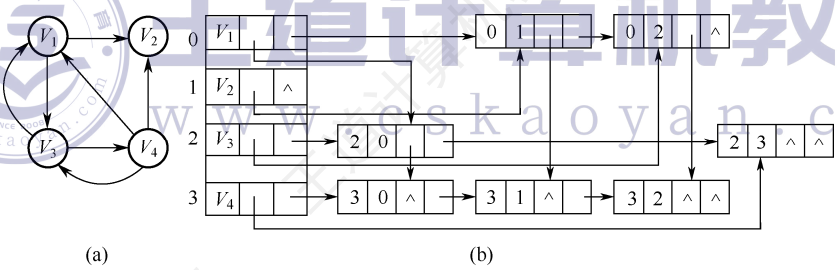


图 6.9　有向图的十字链表表示（弧结点省略 info 域）

注意，顶点结点之间是顺序存储的，弧结点省略了 info 域。

在十字链表中，既容易找到 V_i 为尾的弧，也容易找到 V_i 为头的弧，因而容易求得顶点的出度和入度。图的十字链表表示是不唯一的，但一个十字链表表示唯一确定一个图。

6.2.4　邻接多重表

邻接多重表是无向图的一种链式存储结构。在邻接表中，容易求得顶点和边的各种信息，但求两个顶点之间是否存在边而执行删除边等操作时，需要分别在两个顶点的边表中遍历，效率较低。与十字链表类似，在邻接多重表中，每条边用一个结点表示，其结构如下所示。

ivex	ilink	jvex	jlink	(info)

其中，ivex 域和 jvex 域存放该边依附的两个顶点的编号；ilink 域指向依附于顶点 ivex 的下一条边；jlink 域指向依附于顶点 jvex 的下一条边，info 域存放该边的相关信息。

每个顶点也用一个结点表示，它由如下所示的两个域组成。

data	firstedge

配套课程公众号[考机研]
永久联系微信:4550060

其中，data域存放该顶点的相关信息，firstedge域指向依附于该顶点的第一条边。

在邻接多重表中，所有依附于同一顶点的边串联在同一链表中，因为每条边依附于两个顶点，所以每个边结点同时链接在两个链表中。对无向图而言，其邻接多重表和邻接表的差别仅在于，同一条边在邻接表中用两个结点表示，而在邻接多重表中只有一个结点。

命题追踪 ▶ 图的邻接多重表表示的分析（2024）

图6.10为无向图的邻接多重表表示法。邻接多重表的各种基本操作的实现和邻接表类似。

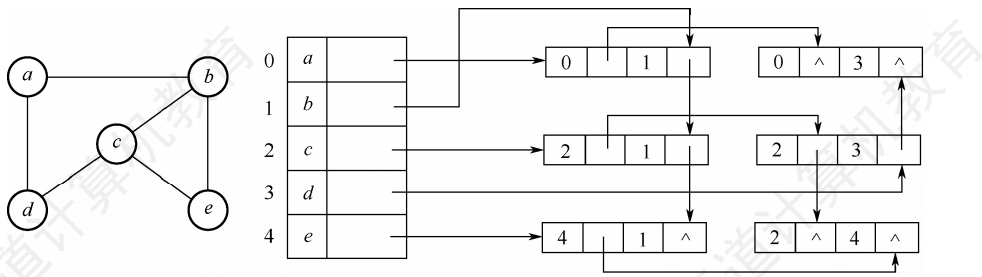


图6.10　无向图的邻接多重表表示（边结点省略info域）

图的四种存储方式的总结如表6.1所示。

表6.1　图的四种存储方式的总结

	邻接矩阵	邻接表	十字链表	邻接多重表
空间复杂度	$O(\|V\|^2)$	无向图：$O(\|V\|+2\|E\|)$ 有向图：$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
找相邻边	遍历对应行或列的时间复杂度为$O(\|V\|)$	找有向图的入度必须遍历整个邻接表	很方便	很方便
删除边或顶点	删除边很方便，删除顶点需要大量移动数据	无向图中删除边或顶点都不方便	很方便	很方便
适用于	稠密图	稀疏图和其他	只能存有向图	只能存无向图
表示方式	唯一	不唯一	不唯一	不唯一

6.2.5　图的基本操作

图的基本操作是独立于图的存储结构的。而对于不同的存储方式，操作算法的具体实现会有着不同的性能。在设计具体算法的实现时，应考虑采用何种存储方式的算法效率会更高。

图的基本操作主要包括（仅抽象地考虑，所以忽略各变量的类型）：

- `Adjacent(G,x,y)`：判断图G是否存在边<x,y>或(x,y)。
- `Neighbors(G,x)`：列出图G中与结点x邻接的边。
- `InsertVertex(G,x)`：在图G中插入顶点x。
- `DeleteVertex(G,x)`：从图G中删除顶点x。
- `AddEdge(G,x,y)`：若无向边(x,y)或有向边<x,y>不存在，则向图G中添加该边。
- `RemoveEdge(G,x,y)`：若无向边(x,y)或有向边<x,y>存在，则从图G中删除该边。
- `FirstNeighbor(G,x)`：求图G中顶点x的第一个邻接点，若有则返回顶点号。若x没有邻接点或图中不存在x，则返回-1。
- `NextNeighbor(G,x,y)`：假设图G中顶点y是顶点x的一个邻接点，返回除y外顶点x的下一个邻接点的顶点号，若y是x的最后一个邻接点，则返回-1。

配套课程公众号[考机研]
永久联系微信:4550060

- `Get_edge_value(G,x,y)`：获取图 G 中边(x,y)或<x,y>对应的权值。
- `Set_edge_value(G,x,y,v)`：设置图 G 中边(x,y)或<x,y>对应的权值为 v。

此外，还有图的遍历算法：按照某种方式访问图中的每个顶点且仅访问一次。图的遍历算法包括深度优先遍历和广度优先遍历，具体见下一节的内容。

6.2.6　本节试题精选

一、单项选择题

01. 下列关于图的存储结构的说法中，错误的是（ ）。

A. 使用邻接矩阵存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中的顶点数有关，与边数无关

B. 邻接表只用于有向图的存储，邻接矩阵适用于有向图和无向图

C. 若一个有向图的邻接矩阵的对角线以下的元素为 0，则该图的拓扑序列必定存在

D. 存储无向图的邻接矩阵是对称的，所以只需存储邻接矩阵的下（或上）三角部分

02. 若图的邻接矩阵中主对角线上的元素皆为 0，其余元素全为 1，则该图一定（ ）。

A. 是无向图　　B. 是有向图　　C. 是完全图　　D. 不是带权图

03. 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中，零元素的个数为（ ）。

A. e　　B. $2e$　　C. n^2-e　　D. n^2-2e

04. 带权有向图 G 用邻接矩阵存储，则 v_i 的入度等于邻接矩阵中（ ）。

A. 第 i 行非∞的元素个数　　B. 第 i 列非∞的元素个数

C. 第 i 行非∞且非 0 的元素个数　　D. 第 i 列非∞且非 0 的元素个数

05. 一个有 n 个顶点的图用邻接矩阵 $\boldsymbol{A}$ 表示，若图为有向图，顶点 v_i 的入度是（ ）；若图为无向图，顶点 v_i 的度是（ ）。

A. $\sum_{i=1}^{n} A[i][j]$　　B. $\sum_{j=1}^{n} A[j][i]$

C. $\sum_{i=1}^{n} A[j][i]$　　D. $\sum_{j=1}^{n} A[j][i]$ 或 $\sum_{j=1}^{n} A[i][j]$

06. 从邻接矩阵 $\boldsymbol{A}=\begin{bmatrix}0&1&0\\1&0&1\\0&1&0\end{bmatrix}$ 可以看出，该图共有（①）个顶点；若是有向图，则该图共有（②）条弧；若是无向图，则共有（③）条边。

①A. 9　B. 3　C. 6　D. 1　E. 以上答案均不正确

②A. 5　B. 4　C. 3　D. 2　E. 以上答案均不正确

③A. 5　B. 4　C. 3　D. 2　E. 以上答案均不正确

07. 以下关于图的存储结构的叙述中，正确的是（ ）。

A. 一个图的邻接矩阵表示唯一，邻接表表示唯一

B. 一个图的邻接矩阵表示唯一，邻接表表示不唯一

C. 一个图的邻接矩阵表示不唯一，邻接表表示唯一

D. 一个图的邻接矩阵表示不唯一，邻接表表示不唯一

08. 矩阵 $\boldsymbol{A}$ 是有向图 G 的邻接矩阵，若矩阵 $\boldsymbol{A}^2$ 的某元素 $a_{i,j}^2=3$，则说明（ ）。

A. 从顶点 i 到 j 存在 3 条长度为 2 的路径

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

B. 从顶点 i 到 j 存在3条长度不超过2的路径

C. 从顶点 i 到 j 存在2条长度为3的路径

D. 从顶点 i 到 j 存在2条长度不超过3的路径

09. 用邻接表法存储图所用的空间大小（ ）。

A. 与图的顶点数和边数有关　　B. 只与图的边数有关

C. 只与图的顶点数有关　　D. 与边数的平方有关

10. 若邻接表中有奇数个边表结点，则（ ）。

A. 图中有奇数个结点　　B. 图中有偶数个结点

C. 图为无向图　　D. 图为有向图

11. 在有向图的邻接表存储结构中，顶点 v 在边表中出现的次数是（ ）。

A. 顶点 v 的度　　B. 顶点 v 的出度

C. 顶点 v 的入度　　D. 依附于顶点 v 的边数

12. n 个顶点的无向图的邻接表最多有（ ）个边表结点。

A. n^2　　B. $n(n-1)$　　C. $n(n+1)$　　D. $n(n-1)/2$

13. 设某无向图中有 n 个顶点和 e 条边，则建立该图的邻接表的时间复杂度是（ ）。

A. $O(n+e)$　　B. $O(n^2)$　　C. $O(ne)$　　D. $O(n^3)$

14. 假设有 n 个顶点、e 条边的有向图用邻接表表示，则删除与某个顶点 v 相关的所有边的时间复杂度为（ ）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

15. 设 n 个顶点、e 条边的有向图用邻接表表示，则某个顶点 v 的入度的时间复杂度为（ ）。

$O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

16. 对邻接表的叙述中，（ ）是正确的。

A. 无向图的邻接表中，第 i 个顶点的度为第 i 个链表中结点数的两倍

B. 邻接表比邻接矩阵的操作更简便

C. 邻接矩阵比邻接表的操作更简便

D. 求有向图结点的度，必须遍历整个邻接表

17. 邻接多重表是（ ）的存储结构。

A. 无向图　　B. 有向图

C. 无向图和有向图　　D. 都不是

18. 十字链表是（ ）的存储结构。

A. 无向图　　B. 有向图

C. 无向图和有向图　　D. 都不是

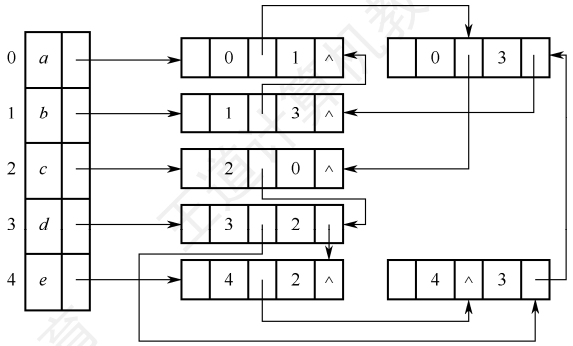
19. 【2013统考真题】设图的邻接矩阵 $\boldsymbol{A}$ 如下所示，各顶点的度依次是（ ）。

$$\boldsymbol{A}=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

A. 1, 2, 1, 2　　B. 2, 2, 1, 1　　C. 3, 4, 2, 3　　D. 4, 4, 2, 2

20. 【2024统考真题】若无向图 $G=(V, E)$ 的邻接多重表如下图所示，则 G 中顶点 b 与 d 的度分别是（ ）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



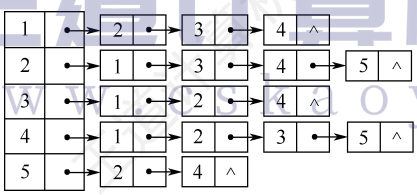
A. 0，2　　B. 2，4　　C. 2，5　　D. 3，4

二、综合应用题

01. 已知带权有向图 G 的邻接矩阵如下图所示，请画出该带权有向图 G。

0	15	2	12	∞	∞	∞
∞	0	∞	∞	6	∞	∞
∞	∞	0	∞	8	4	∞
∞	∞	∞	0	∞	∞	3
∞	∞	∞	∞	0	∞	9
∞	∞	∞	5	∞	0	10
∞	4	∞	∞	∞	∞	0

02. 设图 $G=(V, E)$以邻接表存储，如下图所示。画出其邻接矩阵存储及图 G。



03. 对 n 个顶点的无向图和有向图，分别采用邻接矩阵和邻接表表示时，试问：

1）如何判别图中有多少条边？

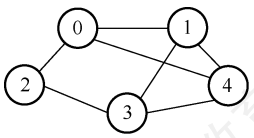
2）如何判别任意两个顶点 i 和 j 是否有边相连？

3）任意一个顶点的度是多少？

04. 如何对无环有向图中的顶点重新编号，使得该图的邻接矩阵中所有的 1 都集中到对角线以上？

05. 写出从图的邻接表表示转换成邻接矩阵表示的算法。

06. 【2015 统考真题】已知含有 5 个顶点的图 G 如下图所示。



请回答下列问题：

1）写出图 G 的邻接矩阵 $\boldsymbol{A}$（行、列下标从 0 开始）。

2）求 $\boldsymbol{A}^2$，矩阵 $\boldsymbol{A}^2$ 中位于 0 行 3 列元素值的含义是什么？

3）若已知具有 n（$n\geqslant 2$）个顶点的图的邻接矩阵为 $\boldsymbol{B}$，则 $\boldsymbol{B}^m$（$2\leqslant m\leqslant n$）中非零元素的含义是什么？

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

07.【2021统考真题】已知无向连通图 G 由顶点集 V 和边集 E 组成，$|E|>0$，当 G 中度为奇数的顶点个数为不大于 2 的偶数时，G 存在包含所有边且长度为$|E|$的路径（称为 EL 路径）。设图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct{                          //图的定义
    int  numVertices,numEdges;           //图中实际的顶点数和边数
    char VerticesList[MAXV];             //顶点表。MAXV 为已定义常量
    int Edge[MAXV][MAXV];                //邻接矩阵
  }MGraph;
```

请设计算法 int IsExistEL(MGraph G)，判断 G 是否存在 EL 路径，若存在，则返回 1，否则返回 0。要求：

1）给出算法的基本设计思想。

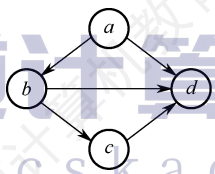
2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

08.【2023统考真题】已知有向图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct{                          //图的类型定义
    int numVertices,numEdges;            //图的顶点数和有向边数
    char VerticesList[MAXV];             //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];                //邻接矩阵
}MGraph;
```

将图中出度大于入度的顶点称为 K 顶点。例如，在下图中，顶点 a 和 b 为 K 顶点。



请设计算法 int printVertices(MGraph G)，对给定的任意非空有向图 G，输出 G 中所有的 K 顶点，并返回 K 顶点的个数。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

6.2.7 答案与解析

一、单项选择题

01. B

n 个顶点的图，若采用邻接矩阵表示，不考虑压缩存储，则存储空间大小为 $O(n^2)$，选项 A 正确。邻接表可用于存储无向图，只是把每条边都视为两条方向相反的有向边，因此需要存储两次，选项 B 错误。因为邻接矩阵中对角线以下的元素全为 0，所以若存在$<i,j>$，则必有 $i<j$，由传递性可知图中路径的顶点编号是依次递增的，假设存在环 $k\rightarrow\cdots\rightarrow j\rightarrow k$，由题设可知 $k<j<k$，矛盾，所以不存在环，拓扑序列必定存在，选项 C 正确。选项 D 显然正确。

注 意

若邻接矩阵对角线以下（或以上）的元素全为 0，则图中必然不存在环，即拓扑序列一定存在，但这并不能说明拓扑序列是唯一的。

02. C

除主对角线上的元素外，其余元素全为 1，说明任意两个顶点之间都有边相连，因此该图一

配套课程公众号[考机研]
永久联系微信:4550060

定是完全图。

03．D

在无向图的邻接矩阵中，矩阵大小为 n^2，非零元素的个数为 $2e$，所以零元素的个数为 n^2-2e。读者应掌握此题的变体，即当无向图变为有向图时，能够求出零的个数和非零的个数。

04．D

带权有向图的邻接矩阵中，0 和∞表示的都不是有向边，而入度是由邻接矩阵的列中元素计算出来的；出度是由邻接矩阵的行中元素计算出来的。

05．B、D

有向图的入度是其第 i 列的非 0 元素之和，无向图的度是第 i 行或第 i 列的非 0 元素之和。

06．B、B、D

邻接矩阵的顶点数等于矩阵的行（列）数，有向图的边数等于矩阵中非零元素的个数，无向图的边数等于矩阵中非零元素个数的一半。

注　意

本题中所给的矩阵为对称矩阵，若不是对称矩阵，则必然不可能是无向图。

07．B

邻接矩阵表示唯一是因为图中边的信息在矩阵中有确定的位置，邻接表不唯一是因为邻接表的建立取决于读入边的顺序和边表中的插入算法。

08．A

设图 G 的邻接矩阵为 $\boldsymbol{A}$，$\boldsymbol{A}^n$ 的元素 $a^n_{i,j}$ 等于从顶点 i 到 j 的长度为 n 的路径的数目，因此 $a^2_{i,j}=3$ 表示从顶点 i 到 j 存在 3 条长度为 2 的路径。该结论记住即可。

09．A

邻接表存储时，顶点数 n 决定了顶点表的大小，边数 e 决定了边表结点的个数，且无向图的每条边存储两次，总存储空间为 $O(n+2e)$。而邻接矩阵只与图的顶点数有关，为 $O(n^2)$。

10．D

无向图采用邻接表表示时，每条边存储两次，所以其边表结点的个数为偶数。题中边表结点为奇数个，所以必然是有向图，且有奇数条边。

11．C

题中的边表是不包括顶点表的。因为任何顶点 u 对应的边表中存放的都是以 u 为起点的边所对应的另一个顶点 v。从而 v 在边表中出现的次数也就是它的入度。

12．B

最多有 $n(n-1)/2$ 条边，每条边在邻接表中存储两次，因此边表结点最多为 $n(n-1)$个。

13．A

建立图的邻接表需要遍历所有的顶点和边，每个顶点有一个顶点表结点，每条边需要创建一个边表结点并插入到相应的链表中。因此，共需 $n+2e$ 次操作，时间复杂度为 $O(n+e)$。

14．C

与顶点 v 相关的边包括出边和入边，对于出边，只需遍历 v 的顶点表结点和其指向的边表；对于入边，则需遍历整个边表。先删除出边：删除 v 的顶点表结点的单链表，出边数最多为 $n-1$，时间复杂度为 $O(n)$；再删除入边：扫描整个边表（扫描剩余全部顶点表结点及其指向的边表），删除所有的顶点 v 的入边，时间复杂度为 $O(n+e)$。因此总时间复杂度为 $O(n+e)$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

15. C

为了求顶点 v 的入度，只需要遍历邻接表中的所有边表，检查每条边是否指向顶点 v，这相当于遍历整个邻接表，因此算法的时间复杂度为 $O(n+e)$ 。

16. D

无向图的邻接表中，第 i 个顶点的度为第 i 个链表中的结点数，所以选项 A 错。邻接表和邻接矩阵对于不同的操作各有优势，选项 B 和 C 都不准确。有向图结点的度包括出度和入度，对于出度，需要遍历顶点表结点所对应的边表；对于入度，则需要遍历剩下的全部边表。

17. A

邻接多重表是无向图的存储结构。

18. B

十字链表是有向图的存储结构。

19. C

邻接矩阵 $\boldsymbol{A}$ 为非对称矩阵，说明图是有向图，度为入度与出度之和。各顶点的度是矩阵中此结点对应的行（对应出度）和列（对应入度）的非零元素之和。

20. B

在邻接多重表中，统计一个结点的入度、出度或度数（入度+出度）时，只需考虑所有的边结点。下图是邻接多重表的边结点的结构：

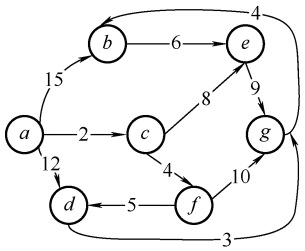
ivex	ilink	jvex	jlink	info

其中，`ivex` 表示弧头的编号，`ilink` 域指向下一个弧头相同的边结点，`jvex` 表示弧尾的编号，`jlink` 域指向下一个弧尾相同的边结点。根据定义可知：求 x 号结点的入度时，只需统计 `jvex` 域等于 x 的边结点个数；求 x 号结点的出度时，只需统计 `ivex` 域等于 x 的边结点个数；x 号结点的度数为入度和出度之和。顶点 b 的编号为 1，统计可知：入度为 1，出度为 1，度数为 2。顶点 d 的编号为 3，统计可知：入度为 1，出度为 3，度数为 4。

二、综合应用题

01.【解答】

带权有向图 G 如下图所示。



02.【解答】

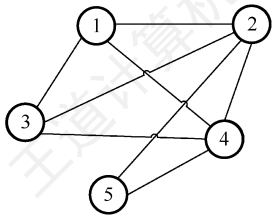
其邻接矩阵存储如下所示。

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

在邻接表中，每条边存储了 2 次，在没有特殊说明时，通常默认其为无向图（当然，无向

配套课程公众号[考机研]
永久联系微信:4550060

图也可视为具有对边的有向图）。该邻接表对应的图 G 如下图所示。



03.【解答】

1）对于邻接矩阵表示的无向图，边数等于矩阵中 1 的个数除以 2；对于邻接表表示的无向图，边数等于边结点的个数除以 2。对于邻接矩阵表示的有向图，边数等于矩阵中 1 的个数；对于邻接表表示的有向图，边数等于边结点的个数。

2）在邻接矩阵表示的无向图或有向图中，对于任意两个顶点 i 和 j，邻接矩阵中 `arcs[i][j]` 或 `arcs[j][i]` 为 1 表示有边相连，否则表示无边相连。在邻接表表示的无向图或有向图中，对于任意两个顶点 i 和 j，若从顶点表结点 i 出发找到编号为 j 的边表结点或从顶点表结点 j 出发找到编号为 i 的边表结点，表示有边相连；否则为无边相连。

3）对于邻接矩阵表示的无向图，顶点 i 的度等于第 i 行中 1 的个数；对于邻接矩阵表示的有向图，顶点 i 的出度等于第 i 行中 1 的个数；入度等于第 i 列中 1 的个数；度数等于它们的和。对于邻接表表示的无向图，顶点 i 的度等于顶点表结点 i 的单链表中边表结点的个数；对于邻接表表示的有向图，顶点 i 的出度等于顶点表结点 i 的单链表中边表结点的个数，顶点 i 的入度等于邻接表中所有编号为 i 的边表结点数；度数等于入度与出度之和。

04.【解答】

按各顶点的出度进行排序。n 个顶点的有向图，其顶点的最大出度是 $n-1$，最小出度为 0。这样排序后，出度最大的顶点编号为 1，出度最小的顶点编号为 n。之后，进行调整，即只要存在弧$<i, j>$，就不管顶点 j 的出度是否大于顶点 i 的出度，都把 i 编号在顶点 j 的编号之前，因为只有 $i \leqslant j$，弧$<i, j>$对应的 1 才能出现在邻接矩阵的上三角。

通过后面小节的学习，会发现采用拓扑排序并依次编号是一种更为简便的方法。

05.【解答】

算法思想：设图的顶点分别存储在数组 `v[n]` 中。首先初始化邻接矩阵。遍历邻接表，在依次遍历顶点 `v[i]` 的边链表时，修改邻接矩阵的第 `i` 行的元素值。若链表边结点的值为 `j`，则置 `arcs[i][j]=1`。遍历完邻接表时，整个转换过程结束。此算法对于无向图、有向图均适用。

算法的实现如下：

```
void  Convert(ALGraph &G,int arcs[M][N]){
//此算法将邻接表方式表示的图 G 转换为邻接矩阵 arcs
    for(i=0;i<n;i++){              //依次遍历各顶点表结点为头的边链表
        p=(G->v[i]).firstarc;      //取出顶点 i 的第一条出边
        while(p!=NULL){            //遍历边链表
            arcs[i][p->adjvex]=1;
            p=p->nextarc;          //取下一条出边
        }                          //while
    }                              //for
}
```

06.【解答】

考查图的邻接矩阵的性质。

配套课程公众号[考机研]
永久联系微信:4550060

1）图 G 的邻接矩阵 $\boldsymbol{A}$ 如下：

$$\boldsymbol{A}=\begin{bmatrix}0&1&1&0&1\\1&0&0&1&1\\1&0&0&1&0\\0&1&1&0&1\\1&1&0&1&0\end{bmatrix}$$

2）$\boldsymbol{A}^2$ 如下：

$$\boldsymbol{A}^2=\begin{bmatrix}3&1&0&3&1\\1&3&2&1&2\\0&2&2&0&2\\3&1&0&3&1\\1&2&2&1&3\end{bmatrix}$$

0 行 3 列的元素值 3 表示从顶点 0 到顶点 3 之间长度为 2 的路径共有 3 条。

3）$\boldsymbol{B}^m$（$2\leqslant m\leqslant n$）中位于 i 行 j 列（$0\leqslant i,\ j\leqslant n-1$）的非零元素的含义是，图中从顶点 i 到顶点 j 的长度为 m 的路径条数。

07.【解答】

1）算法的基本设计思想：

本算法题属于送分题，题干已经告诉我们算法的思想。对于采用邻接矩阵存储的无向图，在邻接矩阵的每一行（列）中，非零元素的个数为本行（列）对应顶点的度。可以依次计算连通图 G 中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

2）算法实现

```
int IsExistEL(MGraph G){
    //采用邻接矩阵存储，判断图是否存在 EL 路径
    int degree,i,j,count=0;
    for(i=0;i<G.numVertices;i++){
        degree=0;
        for(j=0;j<G.numVertices;j++)
            degree+=G.Edge[i][j];          //依次计算各个顶点的度
        if(degree%2!=0)
            count++;      //对度为奇数的顶点计数
    }
    if(count==0||count==2)
        return 1;              //存在 EL 路径，返回 1
    else
        return 0;              //不存在 EL 路径，返回 0
}
```

3）时间复杂度和空间复杂度

算法需要遍历整个邻接矩阵，所以时间复杂度是 $O(n^2)$，空间复杂度是 $O(1)$。

08.【解答】

1）算法的基本设计思想：

采用邻接矩阵表示有向图时，一行中 1 的个数为该行对应顶点的出度，一列中 1 的个数为该列对应顶点的入度。使用一个初值为零的计数器记录 K 顶点的个数。对图 G 的每个顶点，根据邻接矩阵计算其出度 outdegree 和入度 indegree。若 outdegree-indegree>0，则输出该顶点且计数器加 1。最后返回计数器的值。

2）用 C 语言描述的算法：

```
int printVertices(MGraph G){
//采用邻接矩阵存储，输出 K 顶点，返回个数
    int indegree,outdegree,k,m,count=0;
    for(k=0;k<G.numVertices;k++){
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
            indegree=outdegree=0;
            for(m=0;m<G.numVertices;m++)      //计算顶点的出度
                outdegree+=G.Edge[k][m];
            for(m=0;m<G.numVertices;m++)      //计算顶点的入度
                indegree+=G.Edge[m][k];
            if(outdegree>indegree){
                printf("%c",G.VerticesList[k]);
                count++;
            }
        }
        return count;                         //返回 K 顶点的个数
    }
```

6.3 图的遍历

图的遍历是指从图中的某一顶点出发，按照某种搜索方法沿着图中的边对图中的所有顶点访问一次，且仅访问一次。注意到树是一种特殊的图，所以树的遍历实际上也可视为一种特殊的图的遍历。图的遍历算法是求解图的连通性问题、拓扑排序和求关键路径等算法的基础。

图的遍历比树的遍历要复杂得多，因为图的任意一个顶点都可能和其余的顶点相邻接，所以在访问某个顶点后，可能沿着某条路径搜索又回到该顶点。为避免同一顶点被访问多次，在遍历图的过程中，必须记下每个已访问过的顶点，为此可以设一个辅助数组 visited[]来标记顶点是否被访问过。图的遍历算法主要有两种：广度优先搜索和深度优先搜索。

6.3.1 广度优先搜索

广度优先搜索（Breadth-First-Search, BFS）类似于树的层序遍历。基本思想是：首先访问起始顶点 v，接着由 v 出发，依次访问 v 的各个未访问过的邻接顶点 $w_1, w_2, \cdots, w_i$，然后依次访问 $w_1, w_2, \cdots, w_i$ 的所有未被访问过的邻接顶点；再从这些访问过的顶点出发，访问它们所有未被访问过的邻接顶点，直至图中所有顶点都被访问过为止。若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作为始点，重复上述过程，直至图中所有顶点都被访问到为止。Dijkstra 单源最短路径算法和 Prim 最小生成树算法也应用了类似的思想。

换句话说，广度优先搜索遍历图的过程是以 v 为起始点，由近至远依次访问和 v 有路径相通且路径长度为 1, 2,… 的顶点。广度优先搜索是一种分层的查找过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有往回退的情况，因此它不是一个递归的算法。为了实现逐层的访问，算法必须借助一个辅助队列，以记忆正在访问的顶点的下一层顶点。

广度优先搜索算法的伪代码如下：

```
bool visited[MAX_VERTEX_NUM];          //访问标记数组
void BFSTraverse(Graph G){             //对图 G 进行广度优先遍历
    for(i=0;i<G.vexnum;++i)
        visited[i]=FALSE;              //访问标记数组初始化
    InitQueue(Q);                      //初始化辅助队列 Q
    for(i=0;i<G.vexnum;++i)            //从 0 号顶点开始遍历
        if(!visited[i])                //对每个连通分量调用一次 BFS()
            BFS(G,i);                  //若 vi 未访问过，从 vi 开始调用 BFS()
}
```

用邻接表实现广度优先搜索的算法如下：

```
void BFS(ALGraph G,int i){
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        visit(i);                          //访问初始顶点 i
        visited[i]=TRUE;                   //对 i 做已访问标记
        EnQueue(Q,i);                      //顶点 i 入队
        while(!IsEmpty(Q)){
            DeQueue(Q,v);                  //队首顶点 v 出队
            for(p=G.vertices[v].firstarc;p;p=p->nextarc){ //检测 v 的所有邻接点
                w=p->adjvex;
                if(visited[w]==FALSE){
                    visit(w);              // w 为 v 的尚未访问的邻接点，访问 w
                    visited[w]=TRUE;       //对 w 做已访问标记
                    EnQueue(Q,w);          //顶点 w 入队
                }
            }
        }
    }
```

用邻接矩阵实现广度优先搜索的算法如下：

```
    void BFS(MGraph G,int i){
        visit(i);                          //访问初始顶点 i
        visited[i]=TRUE;                   //对 i 做已访问标记
        EnQueue(Q,i);                      //顶点 i 入队
        while(!IsEmpty(Q)){
            DeQueue(Q,v);                  //队首顶点 v 出队
            for(w=0;w<G.vexnum;w++)        //检测 v 的所有邻接点
                if(visited[w]==FALSE&&G.edge[v][w]==1){
                visit(w);                  // w 为 v 的尚未访问的邻接点，访问 w
                visited[w]=TRUE;           //对 w 做已访问标记
                EnQueue(Q,w);              //顶点 w 入队
                }
        }
    }
```

辅助数组 visited[] 标志顶点是否被访问过，其初始状态为 FALSE。在图的遍历过程中，一旦某个顶点 v_i 被访问，就立即置 visited[i] 为 TRUE，防止它被多次访问。

命题追踪 ▶▶ 广度优先遍历的过程（2013）

下面通过实例演示广度优先搜索的过程，给定图 G 如图 6.11 所示。假设从顶点 a 开始访问，a 先入队。此时队列非空，取出队头元素 a，因为 b, c 与 a 邻接且未被访问过，于是依次访问 b, c，并将 b, c 依次入队。队列非空，取出队头元素 b，依次访问与 b 邻接且未被访问的顶点 d, e，并将 d, e 入队（注意：a 与 b 也邻接，但 a 已置访问标记，所以不再重复访问）。此时队列非空，取出队头元素 c，访问与 c 邻接且未被访问的顶点 f, g，并将 f, g 入队。此时，取出队头元素 d，但与 d 邻接且未被访问的顶点为空，所以不进行任何操作。继续取出队头元素 e，将 h 入队列……最终取出队头元素 h 后，队列为空，从而循环自动跳出。遍历结果为 $abcdefgh$。

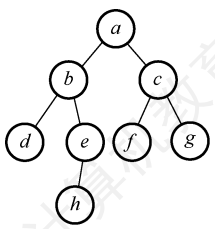


图 6.11　一个无向图 G

配套课程公众号[考机研]
永久联系微信:4550060

从上例不难看出，图的广度优先搜索的过程与二叉树的层序遍历是完全一致的，这也说明了图的广度优先搜索遍历算法是二叉树的层次遍历算法的扩展。

1．BFS 算法的性能分析

无论是邻接表还是邻接矩阵的存储方式，BFS 算法都需要借助一个辅助队列 Q，n 个顶点均需入队一次，在最坏的情况下，空间复杂度为 $O(|V|)$。

命题追踪 ▶▶ 基于邻接表存储的 BFS 的效率（2012）

遍历图的过程实质上是对每个顶点查找其邻接点的过程，耗费的时间取决于所采用的存储结构。采用邻接表存储时，每个顶点均需搜索（或入队）一次，时间复杂度为 $O(|V|)$，在搜索每个顶点的邻接点时，每条边至少访问一次，时间复杂度为 $O(|E|)$，总的时间复杂度为 $O(|V|+|E|)$。采用邻接矩阵存储时，查找每个顶点的邻接点所需的时间为 $O(|V|)$，总时间复杂度为 $O(|V|^2)$。

2．BFS 算法求解单源最短路径问题

若图 $G=(V, E)$为非带权图，定义从顶点 u 到顶点 v 的最短路径 $d(u, v)$为从 u 到 v 的任何路径中最少的边数；若从 u 到 v 没有通路，则 $d(u, v)=\infty$。

使用 BFS，我们可以求解一个满足上述定义的非带权图的单源最短路径问题，这是由广度优先搜索总是按照距离由近到远来遍历图中每个顶点的性质决定的。

BFS 算法求解单源最短路径问题的算法如下：

```
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从 u 到 i 结点的最短路径
    for(i=0;i<G.vexnum;++i)
        d[i]=∞;                    //初始化路径长度
    visited[u]=TRUE; d[u]=0;
    EnQueue(Q,u);
    while(!isEmpty(Q)){            //BFS 算法主过程
        DeQueue(Q,u);              //队头元素 u 出队
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
            if(!visited[w]){       //w 为 u 的尚未访问的邻接顶点
                visited[w]=TRUE;   //设已访问标记
                d[w]=d[u]+1;       //路径长度加 1
                EnQueue(Q,w);      //顶点 w 入队
            }
    }
}
```

3．广度优先生成树

在广度遍历的过程中，我们可以得到一棵遍历树，称为广度优先生成树，如图 6.12 所示。需要注意的是，同一个图的邻接矩阵存储表示是唯一的，所以其广度优先生成树也是唯一的，但因为邻接表存储表示不唯一，所以其广度优先生成树也是不唯一的。

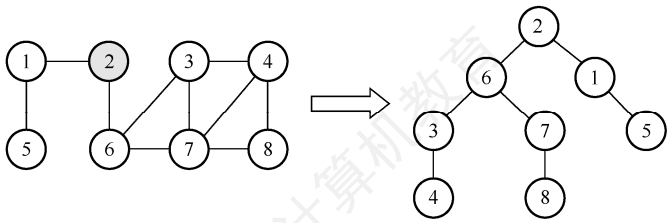


图 6.12　图的广度优先生成树

配套课程公众号[考机研]
永久联系微信:4550060

6.3.2 深度优先搜索

与广度优先搜索不同，深度优先搜索（Depth-First-Search，DFS）类似于树的先序遍历。如其名称中所暗含的意思一样，这种搜索算法所遵循的策略是尽可能“深”地搜索一个图。

它的基本思想如下：首先访问图中某一起始顶点 v，然后由 v 出发，访问与 v 邻接且未被访问的任意一个顶点 w_1，再访问与 w_1 邻接且未被访问的任意一个顶点 w_2……重复上述过程。当不能再继续向下访问时，依次退回到最近被访问的顶点，若它还有邻接顶点未被访问过，则从该点开始继续上述搜索过程，直至图中所有顶点均被访问过为止。

一般情况下，其递归形式的算法十分简洁，算法过程如下：

```
bool visited[MAX_VERTEX_NUM];          //访问标记数组
void DFSTraverse(Graph G){             //对图 G 进行深度优先遍历
    for(i=0;i<G.vexnum;i++)
        visited[i]=FALSE;              //初始化已访问标记数组
    for(i=0;i<G.vexnum;i++)            //本代码中是从 v0 开始遍历
        if(!visited[i])                //对尚未访问的顶点调用 DFS()
            DFS(G,i);
}
```

用邻接表实现深度优先搜索的算法如下：

```
void DFS(ALGraph G,int i){
    visit(i);                          //访问初始顶点 i
    visited[i]=TRUE;                   //对 i 做已访问标记
    for(p=G.vertices[i].firstarc;p;p=p->nextarc){ //检测 i 的所有邻接点
        j=p->adjvex;
        if(visited[j]==FALSE)
            DFS(G,j);                  // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}
```

用邻接矩阵实现深度优先搜索的算法如下：

```
void DFS(MGraph G,int i){
    visit(i);                          //访问初始顶点 i
    visited[i]=TRUE;                   //对 i 做已访问标记
    for(j=0;j<G.vexnum;j++){           //检测 i 的所有邻接点
        if(visited[j]==FALSE&&G.edge[i][j]==1)
            DFS(G,j);                  // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}
```

命题追踪 ▶▶ 深度优先遍历的过程（2015、2016）

以图 6.11 的无向图为例，深度优先搜索的过程：首先访问 a，并置 a 访问标记；然后访问与 a 邻接且未被访问的顶点 b，置 b 访问标记；然后访问与 b 邻接且未被访问的顶点 d，置 d 访问标记。此时 d 已没有未被访问过的邻接点，所以返回上一个访问的顶点 b，访问与其邻接且未被访问的顶点 e，置 e 访问标记，以此类推，直至图中所有顶点都被访问一次。遍历结果为 *abdehcfg*。

注 意

图的邻接矩阵表示是唯一的，但对邻接表来说，若边的输入次序不同，则生成的邻接表也不同。因此，对同样一个图，基于邻接矩阵的遍历得到的 DFS 序列和 BFS 序列是唯一的，基于邻接表的遍历得到的 DFS 序列和 BFS 序列是不唯一的。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

1．DFS 算法的性能分析

DFS 算法是一个递归算法，需要借助一个递归工作栈，所以其空间复杂度为 $O(|V|)$。

遍历图的过程实质上是通过边查找邻接点的过程，因此两种遍历方式的时间复杂度都相同，不同之处仅在于对顶点访问顺序的不同。采用邻接矩阵存储时，总时间复杂度为 $O(|V|^2)$。采用邻接表存储时，总的时间复杂度为 $O(|V|+|E|)$。

2．深度优先的生成树和生成森林

与广度优先搜索一样，深度优先搜索也会产生一棵深度优先生成树。当然，这是有条件的，即对连通图调用 DFS 才能产生深度优先生成树，否则产生的将是深度优先生成森林，如图 6.13 所示。与 BFS 类似，基于邻接表存储的深度优先生成树是不唯一的。

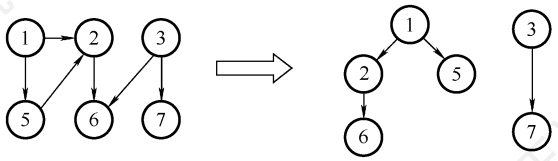


图 6.13　图的深度优先生成森林

6.3.3　图的遍历与图的连通性

图的遍历算法可以用来判断图的连通性。对于无向图来说，若无向图是连通的，则从任意一个结点出发，仅需一次遍历就能够访问图中的所有顶点；若无向图是非连通的，则从某一个顶点出发，一次遍历只能访问到该顶点所在连通分量的所有顶点，而对于图中其他连通分量的顶点，则无法通过这次遍历访问。对于有向图来说，若从初始顶点到图中的每个顶点都有路径，则能够访问到图中的所有顶点，否则不能访问到所有顶点。

因此，在 `BFSTraverse()` 或 `DFSTraverse()` 中添加了第二个 `for` 循环，再选取初始点，继续进行遍历，以防止一次无法遍历图的所有顶点。对于无向图，上述两个函数调用 `BFS(G,i)` 或 `DFS(G,i)` 的次数等于该图的连通分量数；而对于有向图则不是这样，因为一个连通的有向图分为强连通的和非强连通的，它的连通子图也分为强连通分量和非强连通分量，非强连通分量一次调用 `BFS(G,i)` 或 `DFS(G,i)` 不一定能访问到该子图的所有顶点，如图 6.14 所示。

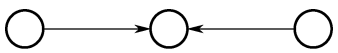


图 6.14　有向图的非强连通分量

6.3.4　本节试题精选

一、单项选择题

01. 下列关于广度优先算法的说法中，正确的是（　）。

I. 当各边的权值相等时，广度优先算法可以解决单源最短路径问题

II. 当各边的权值不等时，广度优先算法可用来解决单源最短路径问题

III. 广度优先遍历算法类似于树中的后序遍历算法

IV. 实现图的广度优先算法时，使用的数据结构是队列

A. I、IV　　B. II、III、IV　　C. II、IV　　D. I、III、IV

02. 下列关于图的说法中，错误的是（　）。

配套课程公众号[考机研]
永久联系微信:4550060

I. 对一个无向图进行深度优先遍历时，得到的深度优先遍历序列是唯一的

II. 若有向图不存在回路，即使不用访问标志位，同一结点也不会被访问两次

III. 采用深度优先遍历或拓扑排序算法可以判断一个有向图中是否有环（回路）

IV. 对任何非强连通图必须2次或以上调用广度优先遍历算法才可访问所有的顶点

A. I、II、III　　B. II、III　　C. I、II　　D. I、II、IV

03. 对于一个非连通无向图 G，采用深度优先遍历访问所有顶点，在 DFSTraverse 函数（见考点讲解 DFS 部分）中调用 DFS 的次数正好等于（　）。

A. 顶点数　　B. 边数　　C. 连通分量数　　D. 不确定

04. 对一个有 n 个顶点、e 条边的图采用邻接表表示时，进行 DFS 遍历的时间复杂度为（　），空间复杂度为（　）；进行 BFS 遍历的时间复杂度为（　），空间复杂度为（　）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(1)$

05. 图的广度优先遍历算法中使用队列作为其辅助数据结构，那么在算法执行过程中，每个顶点的入队次数最多为（　）。

A. 1　　B. 2　　C. 3　　D. 4

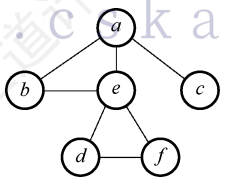
06. 对有 n 个顶点、e 条边的图采用邻接矩阵表示时，进行 DFS 遍历的时间复杂度为（　），进行 BFS 遍历的时间复杂度为（　）。

A. $O(n^2)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(e^2)$

07. 无向图 $G=(V, E)$，其中 $V=\{a, b, c, d, e, f\}$，$E=\{(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)\}$，对该图从 a 开始进行深度优先遍历，得到的顶点序列正确的是（　）。

A. a, b, e, c, d, f　　B. a, c, f, e, b, d　　C. a, e, b, c, f, d　　D. a, e, d, f, c, b

08. 如下图所示，在下面的5个序列中，符合深度优先遍历的序列个数是（　）。



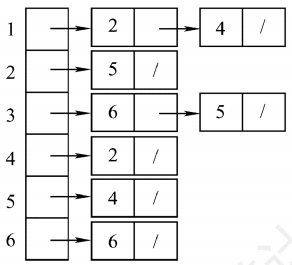
1. $aebfdc$　2. $acfdeb$　3. $aedfcb$　4. $aefdbc$　5. $aecfdb$

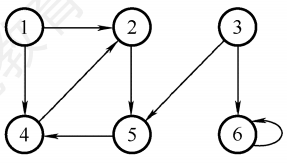
A. 5　　B. 4　　C. 3　　D. 2

09. 用邻接表存储的图的深度优先遍历算法类似于树的（　），而其广度优先遍历算法类似于树的（　）。

A. 中序遍历　　B. 先序遍历　　C. 后序遍历　　D. 按层次遍历

10. 一个有向图 G 的邻接表存储如下图所示，从顶点1出发，对图 G 调用深度优先遍历所得顶点序列是（　）；按广度优先遍历所得顶点序列是（　）。





A. 125436　　B. 124536　　C. 124563　　D. 362514

配套课程公众号[考机研]
永久联系微信:4550060

11. 无向图 $G=(V, E)$，其中 $V=\{a, b, c, d, e, f\}$，$E=\{(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)\}$。对该图进行深度优先遍历，不能得到的序列是（　）。

A. *acfdeb*　　B. *aebdfc*　　C. *aedfcb*　　D. *abecdf*

12. 判断有向图中是否存在回路，除拓扑排序外，还可以利用（　）。（注：涉及下节内容）

A. 求关键路径的方法　　B. 求最短路径的 Dijkstra 算法

C. 深度优先遍历算法　　D. 广度优先遍历算法

13. 设无向图 $G=(V, E)$和 $G'=(V', E')$，若 G'是 G 的生成树，则下列说法错误的是（　）。

A. G'为 G 的子图　　B. G'为 G 的连通分量

C. G'为 G 的极小连通子图且 $V=V'$　　D. G'是 G 的一个无环子图

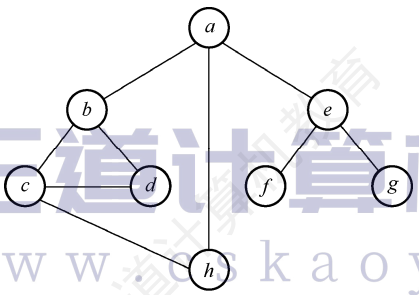
14. 图的广度优先生成树的树高比深度优先生成树的树高（　）。

A. 小或相等　　B. 小　　C. 大或相等　　D. 大

15. 【2012 统考真题】对有 n 个顶点、e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法的时间复杂度是（　）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

16. 【2013 统考真题】下列选项中，不是如下无向图的广度优先遍历序列的是（　）。



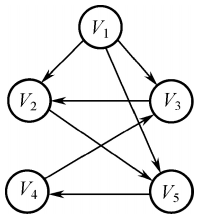
A. *h, c, a, b, d, e, g, f*　　B. *e, a, f, g, b, h, c, d*

C. *d, b, c, a, h, e, f, g*　　D. *a, b, c, d, h, e, f, g*

17. 【2015 统考真题】设有向图 $G=(V, E)$，顶点集 $V=\{V_0, V_1, V_2, V_3\}$，边集 $E=\{<v_0, v_1>, <v_0, v_2>, <v_0, v_3>, <v_1, v_3>\}$。若从顶点 V_0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（　）。

A. 2　　B. 3　　C. 4　　D. 5

18. 【2016 统考真题】下列选项中，不是下图深度优先搜索序列的是（　）。



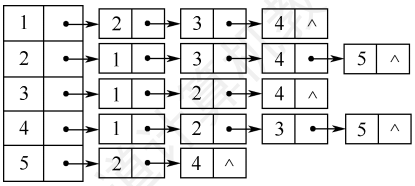
A. V_1, V_5, V_4, V_3, V_2　　B. V_1, V_3, V_2, V_5, V_4

C. V_1, V_2, V_5, V_4, V_3　　D. V_1, V_2, V_3, V_4, V_5

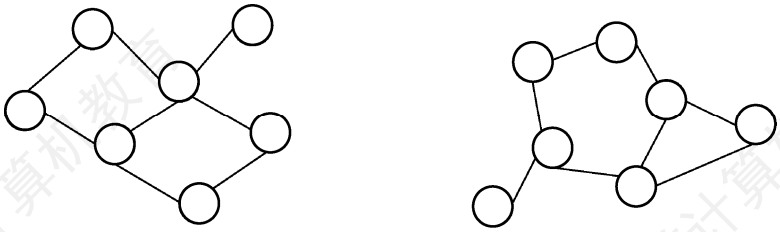
二、综合应用题

01. 图 $G=(V, E)$以邻接表存储，如下图所示，试画出图 G 的深度优先生成树和广度优先生成树（假设从结点 1 开始遍历）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



02. 给定一个连通无向图，采用邻接表存储，将图的所有顶点分别染成红色或蓝色，若存在一种染色方法使图中每条边的两个顶点的颜色都不相同，则称这个图能被二分。



1）判断上面两个无向图是否能被二分，若能被二分，则请标出每个顶点的颜色。

2）请设计一种算法用来判断图是否能被二分，仅用语言描述算法的思想即可。

3）给出你设计的算法的时间复杂度和空间复杂度。

03. 试设计一个算法，判断一个无向图 G 是否为一棵树。若是一棵树，则算法返回 `true`，否则返回 `false`。

04. 分别采用基于深度优先遍历和广度优先遍历算法判别以邻接表方式存储的有向图中是否存在由顶点 v_i 到顶点 v_j 的路径（$i \neq j$）。注意，算法中涉及的图的基本操作必须在此存储结构上实现。

05. 假设图用邻接表表示，设计一个算法，输出从顶点 V_i 到顶点 V_j 的所有简单路径。

6.3.5 答案与解析

一、单项选择题

01. A

广度优先搜索以起始结点为中心，一层一层地向外层扩展遍历图的顶点，因此无法考虑到边权值，只适合求边权值相等的图的单源最短路径。广度优先搜索相当于树的层序遍历，选项 III 错误。广度优先搜索需要用到队列，深度优先搜索需要用到栈，选项 IV 正确。

02. D

图的深度优先遍历序列通常是不唯一的，选项 I 错误。图 1 是一个不存在回路的有向图，从顶点 1 开始执行广度优先遍历，若不设置访问标志位，则会重复访问顶点 3，选项 II 错误。深度优先遍历（见本节后面习题的解析）或拓扑排序算法可以判断有向图中是否有环，选项 III 正确。图 2 是一个非强连通图，但从顶点 1 开始调用一次广度优先遍历算法就可访问所有顶点，选项 IV 错误。

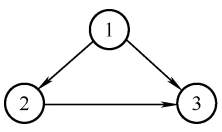


图 1

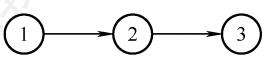


图 2

03. C

配套课程公众号[考机研]
永久联系微信:4550060

DFS（或 BFS）可用来计算无向图的连通分量数，因为一次遍历必然会将一个连通图中的所有顶点都访问到，所以计算图的连通分量数正好是 DFSTraverse()中 DFS 被调用的次数。

04．C、A、C、A

深度优先遍历时，每个顶点表结点和每个边表结点均查找一次，每个顶点递归调用一次，需要借助一个递归工作栈；而广度优先遍历时，也是每个顶点表结点和每个边表结点均查找一次，需要借助一个辅助队列。因此，时间复杂度都是 $O(n+e)$，空间复杂度都是 $O(n)$。

05．A

在图的广度优先遍历算法中，每个顶点被访问后立即做访问标记并入队。若队列不空，则队首顶点出队，若该顶点的邻接顶点未被访问，则访问之，做访问标记并入队；若被访问过，则跳过，如此反复，直至队空。因此，在广度优先遍历过程中，每个顶点最多入队一次。

06．A、A

采用邻接矩阵表示时，查找一个顶点所有出边的时间复杂度为 $O(n)$，共有 n 个顶点，所以时间复杂度均为 $O(n^2)$。

07．D

画出草图后，此类题可以根据边的邻接关系快速排除错误选项。以选项 A 为例，在遍历到 e 之后，应该访问与 e 邻接但未被访问的结点，(e, c)显然不在边集中。

08．D

仅 1 和 4 正确。以 2 为例，遍历到 c 之后，与 c 邻接且未被访问的结点为空集，所以应为 a 的邻接点 b 或 e 入栈。以 3 为例，因为遍历要按栈退回，所以是先 b 后 c，而不能先 c 后 b。

09．B、D

图的深度优先搜索类似于树的先根遍历，即先访问结点，再递归向外层结点遍历，都采用回溯算法。图的广度优先搜索类似于树的层序遍历，即一层一层向外层扩展遍历，都需要采用队列来辅助算法的实现。

10．A、B

DFS 序列产生的路径为<1, 2>, <2, 5>, <5, 4>, <3, 6>；BFS 序列产生的路径为<1, 2>, <1, 4>, <2, 5>, <3, 6>。

11．D

画出 V 和 E 对应的图 G，然后根据搜索算法求解。

注　意

为什么本题序列是不唯一的，而上题序列却是唯一的呢？

因为上题给出了具体的存储结构，此时就必须按照算法的过程来执行，每个顶点的邻接点的顺序已固定，但本题中每个顶点的邻接点的顺序是非固定的。

12．C

利用深度优先遍历可以判断图 G 中是否存在回路。

对于无向图来说，若深度优先遍历过程中遇到了回边，则必定存在环；对于有向图来说，这条回边可能是指向深度优先森林中另一棵生成树上的顶点的弧；但是，从有向图的某个顶点 v 出发进行深度优先遍历时，若在 DFS(v)结束之前出现一条从顶点 u 到顶点 v 的回边，且 u 在生成树上是 v 的子孙，则有向图必定存在包含顶点 v 和顶点 u 的环。

13．B

配套课程公众号[考机研]
永久联系微信:4550060

连通分量是无向图的极大连通子图，其中极大的含义是将依附于连通分量中顶点的所有边都加上，所以连通分量中可能存在回路，这样就不是生成树了。

注 意

极大连通子图是无向图（不一定连通）的连通分量，极小连通子图是连通无向图的生成树。极小和极大是在满足连通的前提下，针对边的数目而言的。极大连通子图包含连通分量的全部边；极小连通子图（生成树）包含连通图的全部顶点，且使其连通的边数最少。

14. A

对于无向图的广度优先搜索生成树，起点到其他顶点的路径是图中对应的最短路径，即所有生成树中树高最小。此外，深度优先总是尽可能“深”地搜索图，因此其路径也尽可能长，所以深度优先生成树的树高总是大于或等于广度优先生成树的树高。

15. C

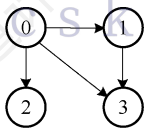
广度优先遍历需要借助队列实现。采用邻接表存储方式对图进行广度优先遍历时，每个顶点均需入队一次（顶点表遍历），所以时间复杂度为 $O(n)$，在搜索所有顶点的邻接点的过程中，每条边至少访问一次（出边表遍历），所以时间复杂度为 $O(e)$，算法总的时间复杂度为 $O(n+e)$。

16. D

只要掌握 DFS 和 BFS 的遍历过程，便能轻易解决。逐个代入，手工模拟，选项 D 是深度优先遍历，而不是广度优先遍历。

17. D

画出该有向图，如下图所示。采用图的深度优先遍历，共有 5 种可能：$<v_0, v_1, v_3, v_2>$，$<v_0, v_2, v_3, v_1>$，$<v_0, v_2, v_1, v_3>$，$<v_0, v_3, v_2, v_1>$，$<v_0, v_3, v_1, v_2>$。



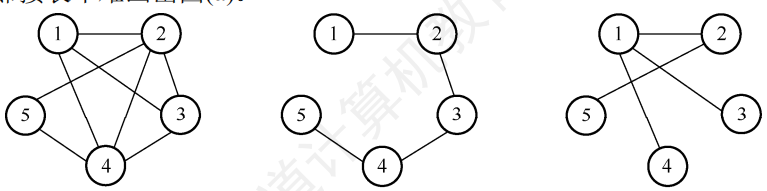
18. D

按深度优先遍历的策略进行遍历。对于选项 A：先访问 V_1，然后访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），此时访问 V_5，然后从 V_5 出发，访问与 V_5 邻接且未被访问的任意一个顶点（满足的只有 V_4），然后从 V_4 出发，访问与 V_4 邻接且未被访问的任意一个顶点（满足的只有 V_3），然后从 V_3 出发，访问与 V_3 邻接且未被访问的任意一个顶点（满足的只有 V_2），结束遍历。选项 B 和 C 的分析方法与 A 相同。对于选项 D，首先访问 V_1，然后从 V_1 出发，访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），然后从 V_2 出发，访问与 V_2 邻接且未被访问的任意一个顶点（满足的只有 V_5），按规则本应该访问 V_5，但选项 D 却访问了 V_3，错误。

二、综合应用题

01.【解答】

根据 G 的邻接表不难画出图(a)。



(a) 图G　(b) 深度优先生成树　(c) 广度优先生成树

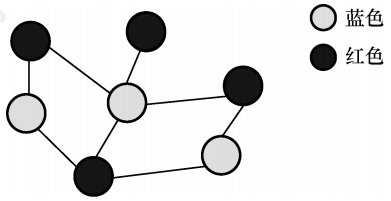
配套课程公众号[考机研]
永久联系微信:4550060

1）采用深度优先遍历。深度优先搜索总是尽可能“深”地搜索图，根据存储结构可知深度优先搜索的路径次序为(1, 2), (2, 3), (3, 4), (4, 5)，深度优先生成树如图(b)所示。需要注意的是，当存储结构固定时，生成树的树形也就固定了，比如不能先搜索(1, 3)。

2）采用广度优先遍历。广度优先搜索总是尽可能“广”地搜索图，一层一层地向外扩展，根据存储结构可知广度优先搜索的路径次序为(1, 2), (1, 3), (1, 4), (2, 5)，广度优先生成树如图(c)所示。

02.【解答】

1）右图不能被二分，左图能被二分，染色情况如下图所示。



2）从任意一个结点开始，将其染成红色，并从该结点开始对整个图进行遍历，在遍历过程中，若当前遍历的结点 a 有一条边指向 b，则可能出现三种情况：①b 未被染色，将它染成与结点 a 不同的颜色，并且继续遍历与 b 相连的结点；②a 与 b 的颜色相同，说明该图不能被二分，直接返回；③ a 与 b 的颜色不同，跳过 b 点。

3）上述遍历无论是使用深度优先还是使用广度优先，时间复杂度都为 $O(n+m)$，其中的 n 和 m 分别是顶点数和边数。需要一个数组来存储各结点的颜色及是否已访问，空间复杂度为 $O(n)$。

03.【解答】

一个无向图 G 是一棵树的条件是，G 必须是无回路的连通图或有 $n-1$ 条边的连通图。这里采用后者作为判断条件。对连通的判定，可以用能否一次遍历全部顶点来实现。可以采用深度优先搜索算法在遍历图的过程中统计可能访问到的顶点个数和边的条数，若一次遍历就能访问到 n 个顶点和 $n-1$ 条边，则可断定此图是一棵树。算法实现如下：

```
bool isTree(Graph& G){
    for(i=1;i<=G.vexnum;i++)
        visited[i]=FALSE;              //访问标记 visited[]初始化
    int Vnum=0,Enum=0;                 //记录顶点数和边数
    DFS(G,1,Vnum,Enum,visited);
    if(Vnum==G.vexnum&&Enum==2*(G.vexnum-1))
        return  true;                  //符合树的条件
    else
        return  false;                 //不符合树的条件
}
void DFS(Graph& G,int v,int& Vnum,int& Enum,int visited[]){
//深度优先遍历图G，统计访问过的顶点数和边数，通过 Vnum 和 Enum 返回
    visited[v]=TRUE;Vnum++;            //作访问标记，顶点计数
    int w=FirstNeighbor(G,v);          //取 v 的第一个邻接顶点
    while(w!=-1){                      //当邻接顶点存在
        Enum++;                        //边存在，边计数
        if(!visited[w])                //当该邻接顶点未访问过
            DFS(G,w,Vnum,Enum,visited);
        w=NextNeighbor(G,v,w);
    }
}
```

04.【解答】

配套课程公众号[考机研]
永久联系微信:4550060

两个不同的遍历算法都采用从顶点 v_i 出发，依次遍历图中每个顶点，直到搜索到顶点 v_j，若能够搜索到 v_j，则说明存在由顶点 v_i 到顶点 v_j 的路径。

深度优先遍历算法的实现如下：

```
int visited[MAXSIZE]={0};     //访问标记数组
void DFS(ALGraph G,int i,int j,bool &can_reach){
//深度优先判断有向图 G 中顶点 vi 到顶点 vj 是否有路径，用 can_reach 来标识
    if(i==j){
        can_reach=true;
        return;               //i 就是 j
    }
    visited[i]=1;             //置访问标记
    for(int p=FirstNeighbor(G,i);p>=0;p=NextNeighbor(G,i,p))
        if(!visited[p]&&!can_reach)     //递归检测邻接点
            DFS(G,p,j,can_reach);
}
```

广度优先遍历算法的实现如下：

```
int visited[MAXSIZE]={0};         //访问标记数组
int BFS(ALGraph G,int i,int j){
//广度优先判断有向图 G 中顶点 vi 到顶点 vj 是否有路径，若是，则返回 1，否则返回 0
    InitQueue(Q); EnQueue(Q,i);  //顶点 i 入队
    while(!isEmpty(Q)){          //非空循环
        DeQueue(Q,i);            //队头顶点出队
        visited[i]=1;            //置访问标记
        if(i==j)  return 1;
        for(int p=FirstNeighbor(G,i);p;p=NextNeighbor(G,i,p)){
                                 //检查所有邻接点
            if(p==j)             //若 p==j，则查找成功
                return 1;
            if(!visited[p]){     //否则，顶点 p 入队
                EnQueue(Q,p);
                visited[p]=1;
            }
        }
    }
 return  0;
}
```

本题也可以这样解答：调用以 i 为参数的 DFS(G,i) 或 BFS(G,i)，执行结束后判断 visited[j] 是否为 TRUE，若是，则说明 v_j 已被遍历，图中必存在由 v_i 到 v_j 的路径。但此种解法每次都耗费最坏时间复杂度对应的时间，需要遍历与 v_i 连通的所有顶点。

05.【解答】

本题采用基于递归的深度优先遍历算法，从结点 u 出发，递归深度优先遍历图中结点，若访问到结点 v，则输出该搜索路径上的结点。为此，设置一个 path 数组来存放路径上的结点（初始为空），d 表示路径长度（初始为-1）。查找从顶点 u 到 v 的简单路径过程说明如下（假设查找函数名为 FindPath()）：

1）FindPath(G,u,v,path,d):d++;path[d]=u；若找到 u 的未访问过的相邻结点 u1，则继续下去，否则置 visited[u]=0 并返回。

2）FindPath(G,u1,v,path,d):d++;path[d]=u1；若找到 u1 的未访问过的相邻结点 u2，则继续下去，否则置 visited[u1]=0。

3）以此类推，继续上述递归过程，直到 ui=v，输出 path。

算法实现如下：

```
    void FindPath(AGraph *G,int u,int v,int path[],int d){
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        int w;
        ArcNode *p;
        d++;                              //路径长度增 1
        path[d]=u;                        //将当前顶点添加到路径中
        visited[u]=1;                     //置已访问标记
        if(u==v)                          //找到一条路径则输出
            print(path[]);                //输出路径上的结点
        p=G->adjlist[u]. firstarc;        //p 指向 u 的第一个相邻点
        while(p!=NULL){
            w=p->adjvex;                  //若顶点 w 未访问，递归访问它
            if(visited[w]==0)
                FindPath(G,w,v,path,d);
            p=p->nextarc;                 //p 指向 u 的下一个相邻点
        }
        visited[u]=0;                     //恢复环境，使该顶点可重新使用
    }
```

6.4 图的应用

本节是历年考查的重点。图的应用主要包括：最小生成（代价）树、最短路径、拓扑排序和关键路径。一般而言，这部分内容直接以算法设计题形式考查的可能性偏小，而更多的是结合图的实例来考查算法的具体操作过程，读者必须学会手工模拟给定图的各个算法的执行过程。此外，还需掌握对给定模型建立相应的图去解决问题的方法。

6.4.1 最小生成树

一个连通图的生成树包含图的所有顶点，并且只含尽可能少的边。对于生成树来说，若砍去一条边，则会使生成树变成非连通图；若增加一条边，则会在图中形成一条回路。

对于一个带权连通无向图 G，生成树不同，每棵树的权（树中所有边的权值之和）也可能不同。权值之和最小的那棵生成树称为 G 的最小生成树（Minimum-Spanning-Tree, MST）。

命题追踪 ▶▶ 最小生成树的性质（2012、2017）

不难看出，最小生成树具有如下性质：

1）若图 G 中存在权值相同的边，则 G 的最小生成树可能不唯一，即最小生成树的树形不唯一。当图 G 中的各边权值互不相等时，G 的最小生成树是唯一的；若无向连通图 G 的边数比顶点数少 1，即 G 本身是一棵树时，则 G 的最小生成树就是它本身。

2）虽然最小生成树不唯一，但其对应的边的权值之和总是唯一的，而且是最小的。

3）最小生成树的边数为顶点数减 1。

命题追踪 ▶▶ 最小生成树中某顶点到其他顶点是否具有最短路径的分析（2023）

注　意

最小生成树中所有边的权值之和最小，但不能保证任意两个顶点之间的路径是最短路径。如下图所示，最小生成树中 A 到 C 的路径长度为 5，但图中 A 到 C 的最短路径长度为 4。

配套课程公众号[考机研]
永久联系微信:4550060

构造最小生成树有多种算法，但大多数算法都利用了最小生成树的下列性质：假设 $G=(V, E)$ 是一个带权连通无向图，U 是顶点集 V 的一个非空子集。若(u, v)是一条具有最小权值的边，其中 $u\in U$，$v\in V-U$，则必存在一棵包含边(u, v)的最小生成树。

基于该性质的最小生成树算法主要有 Prim 算法和 Kruskal 算法，它们都基于贪心算法的策略。对这两种算法应主要掌握算法的本质含义和基本思想，并能动手模拟算法的实现步骤。

下面介绍一个通用的最小生成树算法：

```
GENERIC_MST(G){
    T=NULL;
    while T 未形成一棵生成树;
        do 找到一条最小代价边(u,v)并且加入 T 后不会产生回路;
           T=T∪(u,v);
}
```

通用算法每次加入一条边以逐渐形成一棵生成树，下面介绍两种实现上述通用算法的途径。

1．Prim 算法

Prim（普里姆）算法的执行非常类似于寻找图的最短路径的 Dijkstra 算法（见下一节）。

命题追踪 ▶ Prim 算法构造最小生成树的实例（2015、2017、2018）

Prim 算法构造最小生成树的过程如图 6.15 所示。初始时从图中任取一顶点（如顶点 1）加入树 T，此时树中只含有一个顶点，之后选择一个与当前 T 中顶点集合距离最近的顶点，并将该顶点和相应的边加入 T，每次操作后 T 中的顶点数和边数都增 1。以此类推，直至图中所有的顶点都并入 T，得到的 T 就是最小生成树。此时 T 中必然有 $n-1$ 条边。

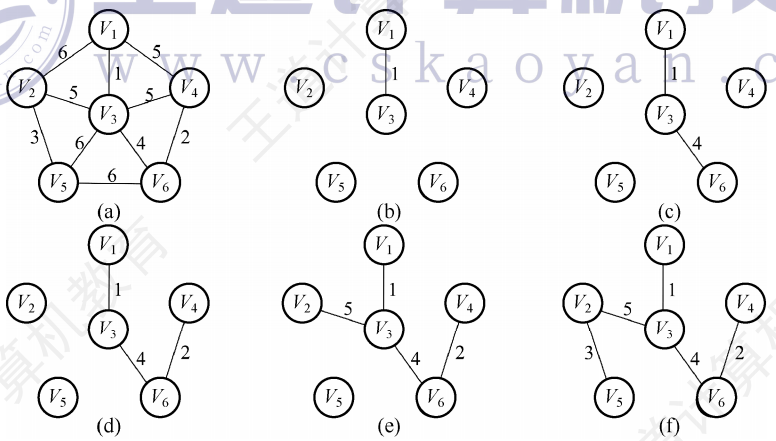


图 6.15　Prim 算法构造最小生成树的过程

Prim 算法的步骤如下：

假设 $G=\{V, E\}$是连通图，其最小生成树 $T=(U, E_T)$，E_T是最小生成树中边的集合。

初始化：向空树 $T=(U, E_T)$中添加图 $G=(V, E)$的任意一个顶点 u_0，使 $U=\{u_0\}$，$E_T=\varnothing$。

循环（重复下列操作直至 $U=V$）：从图 G 中选择满足$\{(u, v)|u\in U, v\in V-U\}$且具有最小权值的边$(u, v)$，加入树 T，置 $U=U\cup\{v\}$，$E_T=E_T\cup\{(u, v)\}$。

Prim 算法的简单实现如下：

```
void Prim(G,T){
    T=∅;                    //初始化空树
    U={w};                  //添加任意一个顶点 w
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        while((V-U)!=∅){          //若树中不含全部顶点
            设(u,v)是使 u∈U 与 v∈(V-U),且权值最小的边;
            T=T∪{(u,v)};          //边归入树
            U=U∪{v};              //顶点归入树
        }
    }
```

Prim 算法的时间复杂度为 $O(|V|^2)$，不依赖于$|E|$，因此它适用于求解边稠密的图的最小生成树。虽然采用其他方法能改进 Prim 算法的时间复杂度，但增加了实现的复杂性。

2．Kruskal 算法

与 Prim 算法从顶点开始扩展最小生成树不同，Kruskal 算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法。

命题追踪 ▶▶ Kruskal 算法构造最小生成树的实例（2015、2018、2020）

Kruskal 算法构造最小生成树的过程如图 6.16 所示。初始时为只有 n 个顶点而无边的非连通图 $T=\{V,\{\}\}$，每个顶点自成一个连通分量。然后按照边的权值由小到大的顺序，不断选取当前未被选取过且权值最小的边，若该边依附的顶点落在 T 中不同的连通分量上（使用并查集判断这两个顶点是否属于同一棵集合树），则将此边加入 T，否则舍弃此边而选择下一条权值最小的边。以此类推，直至 T 中所有顶点都在一个连通分量上。

Kruskal 算法的步骤如下：

假设 $G=(V,E)$是连通图，其最小生成树 $T=(U,E_T)$。

初始化：$U=V,E_T=\varnothing$。即每个顶点构成一棵独立的树，T此时是一个仅含$|V|$个顶点的森林。

循环（重复直至 T 是一棵树）：按 G 的边的权值递增顺序依次从 $E-E_T$ 中选择一条边，若这条边加入 T 后不构成回路，则将其加入 E_T，否则舍弃，直到 E_T中含有 $n-1$条边。

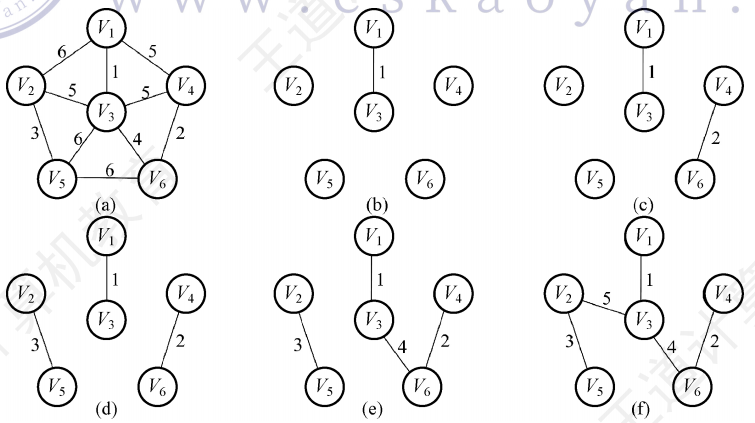


图 6.16　Kruskal 算法构造最小生成树的过程

Kruskal 算法的简单实现如下：

```
void Kruskal(V,T){
    T=V;                          //初始化树 T，仅含顶点
    numS=n;                       //连通分量数
    while(numS>1){                //若连通分量数大于 1
        从 E 中取出权值最小的边(v,u);
        if(v 和 u 属于 T 中不同的连通分量){
        T=T∪{(v,u)};              //将此边加入生成树中
        numS--;                   //连通分量数减 1
```

配套课程公众号[考机研]
永久联系微信:4550060

```
                }
            }
        }
```

根据图的相关性质，若一条边连接了两棵不同树中的顶点，则对这两棵树来说，它必定是连通的，将这条边加入森林中，完成两棵树的合并，直到整个森林合并成一棵树。

在 Kruskal 算法中，最坏情况需要对$|E|$条边各扫描一次。通常采用堆（见第 7 章）来存放边的集合，每次选择最小权值的边需要 $O(\log_2|E|)$的时间；每次使用并查集来快速判断两个顶点是否属于一个集合所需的时间为 $O(\alpha(|V|))$，$\alpha(|V|)$的增长极其缓慢，可视为常数。算法的总时间复杂度为 $O(|E|\log_2|E|)$，不依赖于$|V|$，因此 Kruskal 算法适合于边稀疏而顶点较多的图。

6.4.2 最短路径

命题追踪 ▶ 最短路径的分析与举例以及相关的算法（2009、2023）

6.3 节所述的广度优先搜索查找最短路径只是对无权图而言的。当图是带权图时，把从一个顶点 v_0 到图中其余任意一个顶点 v_i 的一条路径所经过边上的权值之和，定义为该路径的带权路径长度，把带权路径长度最短的那条路径（可能不止一条）称为最短路径。

求解最短路径的算法通常都依赖于一种性质，即两点之间的最短路径也包含了路径上其他顶点间的最短路径。带权有向图 G 的最短路径问题一般可分为两类：一是单源最短路径，即求图中某一顶点到其他各顶点的最短路径，可通过经典的 Dijkstra（迪杰斯特拉）算法求解；二是求每对顶点间的最短路径，可通过 Floyd（弗洛伊德）算法来求解。

1．Dijkstra 算法求单源最短路径问题

Dijkstra 算法设置一个集合 S 记录已求得的最短路径的顶点，初始时把源点 v_0 放入 S，集合 S 每并入一个新顶点 v_i，都要修改源点 v_0 到集合 $V-S$ 中顶点当前的最短路径长度值（这里可能不太好理解？没关系，继续往下看，相信会逐步理解）。

在构造的过程中还设置了三个辅助数组：

- `final[]`：标记各顶点是否已找到最短路径，即是否归入集合 S。
- `dist[]`：记录从源点 v_0 到其他各顶点当前的最短路径长度，它的初始值为：若从 v_0 到 v_i 有弧，则 `dist[i]`为弧上的权值；否则置 `dist[i]`为∞。
- `path[]`：`path[i]`表示从源点到顶点 i 之间的最短路径的前驱结点。在算法结束时，可根据其值追溯得到源点 v_0 到顶点 v_i 的最短路径。

假设从顶点 0 出发，即 $v_0=0$，集合 S 最初只包含顶点 0，邻接矩阵 `arcs` 表示带权有向图，`arcs[i][j]`表示有向边$<i,j>$的权值，若不存在有向边$<i,j>$，则 `arcs[i][j]`为∞。

Dijkstra 算法的步骤如下（不考虑对 `path[]`的操作）：

1）初始化：集合 S 初始为{0}，`dist[]`的初始值 `dist[i]=arcs[0][i]`，$i=1,2,\cdots,n-1$。

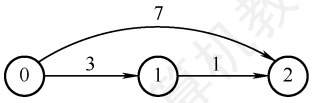
2）从顶点集合 $V-S$ 中选出 v_j，满足 `dist[j]=Min{dist[i]|` $v_i\in V-S$`}`，v_j 就是当前求得的一条从 v_0 出发的最短路径的终点，令 $S=S\cup\{j\}$。

3）修改从 v_0 出发到集合 $V-S$ 上任意一个顶点 v_k 可达的最短路径长度：若 `dist[j]+arcs[j][k]<dist[k]`，则更新 `dist[k]=dist[j]+arcs[j][k]`。

4）重复 2）～3）操作共 $n-1$ 次，直到所有的顶点都包含在集合 S 中。

步骤 3）也就是开头留下的疑问，每当一个顶点加入集合 S 后，可能需要修改源点 v_0 到集合 $V-S$ 中可达顶点当前的最短路径长度，下面举一简单例子证明。如下图所示，源点为 v_0，初始时 $S=\{v_0\}$，`dist[1]=3`，`dist[2]=7`，当将 v_1 并入集合 S 后，`dist[2]`需要更新为 4。

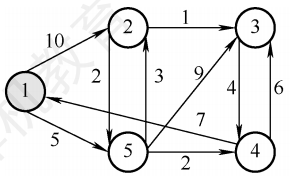
王道计算机教育 www.kaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



思考：Dijkstra 算法与 Prim 算法有何异同之处？[①]

命题追踪 ▶▶ Dijkstra 算法求解最短路径的实例（2012、2014、2016、2021）

例如，对图 6.17 中的图应用 Dijkstra 算法求从顶点 1 出发至其余顶点的最短路径的过程，如表 6.2 所示。算法执行过程的说明如下。



每轮得到的最短路径如下：
第1轮：1→5，路径距离为5
第2轮：1→5→4，路径距离为7
第3轮：1→5→2，路径距离为8
第4轮：1→5→2→3，路径距离为9

图 6.17　应用 Dijkstra 算法图

表 6.2　从 v_1 到各终点的 dist 值和最短路径的求解过程

顶点	第 1 轮	第 2 轮	第 3 轮	第 4 轮
2	10 $v_1 \rightarrow v_2$	8 $v_1 \rightarrow v_5 \rightarrow v_2$	**8** **$v_1 \rightarrow v_5 \rightarrow v_2$**	
3	∞	14 $v_1 \rightarrow v_5 \rightarrow v_3$	13 $v_1 \rightarrow v_5 \rightarrow v_4 \rightarrow v_3$	**9** **$v_1 \rightarrow v_5 \rightarrow v_2 \rightarrow v_3$**
4	∞	**7** **$v_1 \rightarrow v_5 \rightarrow v_4$**		
5	**5** **$v_1 \rightarrow v_5$**			
集合 S	**{1, 5}**	**{1, 5, 4}**	**{1, 5, 4, 2}**	**{1, 5, 4, 2, 3}**

初始化：集合 S 初始为 $\{v_1\}$，v_1 可达 v_2 和 v_5，v_1 不可达 v_3 和 v_4，因此 dist[]数组各元素的初始值依次设置为 dist[2]=10，dist[3]=∞，dist[4]=∞，dist[5]=5。

第 1 轮：选出最小值 dist[5]，将顶点 v_5 并入集合 S，即此时已找到 v_1 到 v_5 的最短路径。当 v_5 加入 S 后，从 v_1 到集合 $V-S$ 中可达顶点的最短路径长度可能会产生变化。因此需要更新 dist[]数组。v_5 可达 v_2，因 $v_1 \rightarrow v_5 \rightarrow v_2$ 的距离 8 比 dist[2]=10 小，更新 dist[2]=8；v_5 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_3$ 的距离 14，更新 dist[3]=14；v_5 可达 v_4，$v_1 \rightarrow v_5 \rightarrow v_4$ 的距离 7，更新 dist[4]=7。

第 2 轮：选出最小值 dist[4]，将顶点 v_4 并入集合 S。继续更新 dist[]数组。v_4 不可达 v_2，dist[2]不变；v_4 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_4 \rightarrow v_3$ 的距离 13 比 dist[3]小，故更新 dist[3]=13。

第 3 轮：选出最小值 dist[2]，将顶点 v_2 并入集合 S。继续更新 dist[]数组。v_2 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_2 \rightarrow v_3$ 的距离 9 比 dist[3]小，更新 dist[3]=9。

第 4 轮：选出唯一最小值 dist[3]，将顶点 v_3 并入集合 S，此时全部顶点都已包含在 S 中。

显然，Dijkstra 算法也是基于贪心策略的。

① Dijkstra 算法的流程、操作与 Prim 算法的都很相似，都基于贪心策略。区别在于，i. 目的不同：Dijkstra 算法的目的是构建单源点的最短路径树；Prim 算法的目的是构建最小生成树。ii. 算法思路略有不同：Prim 算法从一个点开始，每次选择权值最小的边，将其连接到已构建的生成树上，直至所有顶点都已加入；而 Dijkstra 算法每次找出到源点距离最近且未归入集合的点，并把它归入集合，同时以这个点为基础更新从源点到其他所有顶点的距离。iii. 适用的图不同：Prim 算法只能用于带权无向图；Dijkstra 算法可用于带权有向图或带权无向图。

配套课程公众号[考机研]
永久联系微信:4550060

使用邻接矩阵表示时，时间复杂度为$O(|V|^2)$。使用带权的邻接表表示时，虽然修改dist[]的时间可以减少，但因为在dist[]中选择最小分量的时间不变，所以时间复杂度仍为$O(|V|^2)$。

人们可能只希望找到从源点到某个特定顶点的最短路径，但这个问题和求解源点到其他所有顶点的最短路径一样复杂，时间复杂度也为$O(|V|^2)$。

注意，边上带有负权值时，Dijkstra 算法并不适用。若允许边上带有负权值，则在与集合 S（已求得最短路径的顶点集，归入 S 内的顶点的最短路径不再变更）内某顶点（记为 a）以负边相连的顶点（记为 b）确定其最短路径时，其最短路径长度加上这条负边的权值结果可能小于 a 原先确定的最短路径长度，而此时 a 在 Dijkstra 算法下是无法更新的。例如，对于图 6.18 所示的带权有向图，利用 Dijkstra 算法不一定能得到正确的结果。

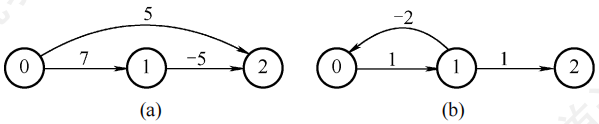


图 6.18　边上带有负权值的有向带权图

2．Floyd 算法求各顶点之间最短路径问题

求所有顶点之间的最短路径问题描述如下：已知一个各边权值均大于 0 的带权有向图，对任意两个顶点 $v_i \neq v_j$，要求求出 v_i 与 v_j 之间的最短路径和最短路径长度。

Floyd 算法的基本思想是：递推产生一个 n 阶方阵序列 $A^{(-1)}, A^{(0)}, \cdots, A^{(k)}, \cdots, A^{(n-1)}$，其中 $A^{(k)}[i][j]$表示从顶点 v_i 到顶点 v_j 的路径长度，k 表示绕行第 k 个顶点的运算步骤。初始时，对于任意两个顶点 v_i 和 v_j，若它们之间存在边，则以此边上的权值作为它们之间的最短路径长度；若它们之间不存在有向边，则以∞作为它们之间的最短路径长度。以后逐步尝试在原路径中加入顶点 k（$k=0, 1, \cdots, n-1$）作为中间顶点。若增加中间顶点后，得到的路径比原来的路径长度减少了，则以此新路径代替原路径。算法描述如下：

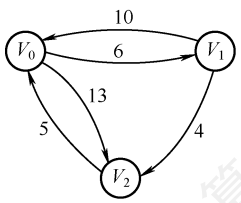
定义一个 n 阶方阵序列 $A^{(-1)}, A^{(0)}, \cdots, A^{(n-1)}$，其中，

$$A^{(-1)}[i][j] = \text{arcs}[i][j]$$

$$A^{(k)}[i][j] = \text{Min}\{A^{(k-1)}[i][j],\quad A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\},\ k = 0, 1, \cdots, n-1$$

式中，$A^{(0)}[i][j]$是从顶点 v_i 到 v_j、中间顶点是 v_0 的最短路径的长度，$A^{(k)}[i][j]$是从顶点 v_i 到 v_j、中间顶点的序号不大于 k 的最短路径的长度。Floyd 算法是一个迭代的过程，每迭代一次，在从 v_i 到 v_j 的最短路径上就多考虑了一个顶点；经过 n 次迭代后，所得到的 $A^{(n-1)}[i][j]$就是 v_i 到 v_j 的最短路径长度，即方阵 $A^{(n-1)}$中就保存了任意一对顶点之间的最短路径长度。

图 6.19 所示为带权有向图 G 及其邻接矩阵。应用 Floyd 算法求所有顶点之间的最短路径长度的过程如表 6.3 所示。算法执行过程的说明如下。



(a) 有向图 G

$$\begin{bmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & \infty & 0 \end{bmatrix}$$

(b) G 的邻接矩阵

图 6.19　带权有向图 G 及其邻接矩阵

配套课程公众号[考机研]
永久联系微信:4550060

初始化：方阵 $A^{(-1)}[i][j]=\text{arcs}[i][j]$。

第 1 轮：将 v_0 作为中间顶点，对于所有顶点对 $\{i,j\}$，若有 $A^{-1}[i][j]>A^{-1}[i][0]+A^{-1}[0][j]$，则将 $A^{-1}[i][j]$ 更新为 $A^{-1}[i][0]+A^{-1}[0][j]$。有 $A^{-1}[2][1]>A^{-1}[2][0]+A^{-1}[0][1]=11$，更新 $A^{-1}[2][1]=11$，更新后的方阵标记为 A^0。

第 2 轮：将 v_1 作为中间顶点，继续检测全部顶点对 $\{i,j\}$。有 $A^0[0][2]>A^0[0][1]+A^0[1][2]=10$，更新 $A^0[0][2]=10$，更新后的方阵标记为 A^1。

第 3 轮：将 v_2 作为中间顶点，继续检测全部顶点对 $\{i,j\}$。有 $A^1[1][0]>A^1[1][2]+A^1[2][0]=9$，更新 $A^1[1][0]=9$，更新后的方阵标记为 A^2。此时 A^2 中保存的就是任意顶点对的最短路径长度。

表 6.3　Floyd 算法的执行过程

A	$A^{(-1)}$			$A^{(0)}$			$A^{(1)}$			$A^{(2)}$		
	V_0	V_1	V_2	V_0	V_1	V_2	V_0	V_1	V_2	V_0	V_1	V_2
V_0	0	6	13	0	6	13	0	6	**10**	0	6	10
V_1	10	0	4	10	0	4	10	0	4	**9**	0	4
V_2	5	∞	0	5	**11**	0	5	11	0	5	11	0

Floyd 算法的时间复杂度为 $O(|V|^3)$。不过其代码很紧凑，且并不包含其他复杂的数据结构，因此隐含的常数系数是很小的，即使对于中等规模的输入来说，它仍然是相当有效的。

Floyd 算法允许图中有带负权值的边，但不允许包含总权值为负的回路。Floyd 算法同样适用于带权无向图，因为带权无向图可视为权值相同往返二重边的有向图。

也可以用单源最短路径算法来解决每对顶点之间的最短路径问题。轮流将每个顶点作为源点，并且在所有边权值均非负时，运行一次 Dijkstra 算法，其时间复杂度为 $O(|V|^2)\cdot|V|=O(|V|^3)$。

BFS 算法、Dijkstra 算法和 Floyd 算法求最短路径的总结如表 6.4 所示。

表 6.4　BFS 算法、Dijkstra 算法和 Floyd 算法求最短路径的总结

	BFS 算法	Dijkstra 算法	Floyd 算法
用途	求单源最短路径	求单源最短路径	求各顶点之间的最短路径
无权图	适用	适用	适用
带权图	不适用	适用	适用
带负权值的图	不适用	不适用	适用
带负权回路的图	不适用	不适用	不适用
时间复杂度	$O(\|V\|^2)$或 $O(\|V\|+\|E\|)$	$O(\|V\|^2)$	$O(\|V\|^3)$

6.4.3　有向无环图描述表达式

有向无环图：若一个有向图中不存在环，则称为有向无环图，简称 DAG 图。

命题追踪 ▶▶ 构建表达式的有向无环图（2019）

有向无环图是描述含有公共子式的表达式的有效工具。例如表达式

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

可以用上一章描述的二叉树来表示，如图 6.20 所示。仔细观察该表达式，可发现有一些相同的子表达式$(c+d)$和$(c+d)*e$，而在二叉树中，它们也重复出现。若利用有向无环图，则可实现对相同子式的共享，从而节省存储空间，图 6.21 所示为该表达式的有向无环图表示。

配套课程公众号[考机研]
永久联系微信:4550060

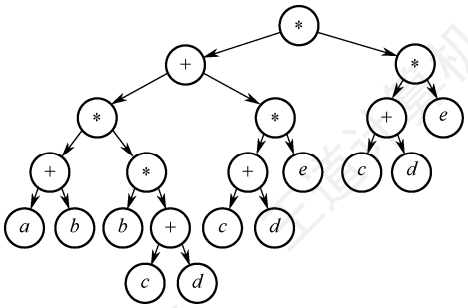


图 6.20 二叉树描述表达式

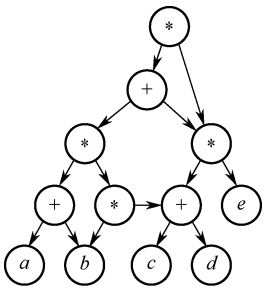


图 6.21 有向无环图描述表达式

用有向无环图描述表达式的解题方法举例见本书配套课程。

注 意

在表达式的有向无环图表示中，不可能出现重复的操作数顶点。

6.4.4 拓扑排序

AOV 网：若用有向无环图表示一个工程，其顶点表示活动，用有向边<V_i, V_j>表示活动 V_i 必须先于活动 V_j 进行的这样一种关系，则将这种有向图称为顶点表示活动的网络，简称 AOV 网。在 AOV 网中，活动 V_i 是活动 V_j 的直接前驱，V_j 是 V_i 的直接后继，这种前驱和后继关系具有传递性，且任何活动 V_i 不能以它自己作为自己的前驱或后继。

拓扑排序：在图论中，由一个有向无环图的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：

1）每个顶点出现且只出现一次。

2）若顶点 A 在序列中排在顶点 B 的前面，则在图中不存在从 B 到 A 的路径。

或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点 A 到顶点 B 的路径，则在排序中 B 出现在 A 的后面。每个 AOV 网都有一个或多个拓扑排序序列。

命题追踪 ▶ 拓扑排序和回路的关系（2011）

对一个 AOV 网进行拓扑排序的算法有很多，下面介绍比较常用的一种方法的步骤：

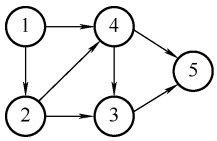
① 从 AOV 网中选择一个没有前驱（入度为 0）的顶点并输出。

② 从网中删除该顶点和所有以它为起点的有向边。

③ 重复①和②直到当前的 AOV 网为空或当前网中不存在无前驱的顶点为止。后一种情况说明有向图中必然存在环。

命题追踪 ▶ 拓扑排序的实例（2010、2014、2018、2021）

图 6.22 所示为拓扑排序过程的示例。每轮选择一个入度为 0 的顶点并输出，然后删除该顶点和所有以它为起点的有向边，最后得到拓扑排序的结果为{1, 2, 4, 3, 5}。

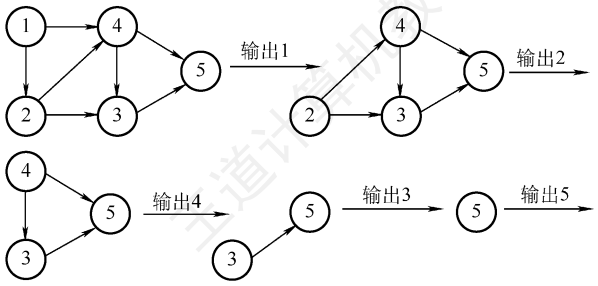


结点号	1	2	3	4	5
初始入度	0	1	2	2	2
第 1 轮		0	2	1	2
第 2 轮			1	0	2
第 3 轮			0		1
第 4 轮					0
第 5 轮					

(a)

图 6.22 有向无环图的拓扑排序过程

配套课程公众号[考机研]
永久联系微信:4550060



(b)

图 6.22　有向无环图的拓扑排序过程（续）

命题追踪 ▶▶（算法题）拓扑排序的实现（2024）

拓扑排序算法（基于邻接表存储）的实现如下：

```
bool TopologicalSort(Graph G){
    InitStack(S);                   //初始化栈，存储入度为 0 的顶点
    int i;
    for(i=0;i<G.vexnum;i++)
        if(indegree[i]==0)
            Push(S,i);              //将所有入度为 0 的顶点入栈
    int count=0;                    //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){             //栈不空，则存在入度为 0 的顶点
        Pop(S,i);                   //栈顶元素出栈
        print[count++]=i;           //输出顶点 i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
        //将所有 i 指向的顶点的入度减 1，并且将入度减为 0 的顶点压入栈 S
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);          //入度为 0，则入栈
        }
    }
    if(count<G.vexnum)
        return  false;              //排序失败，有向图中有回路
    else
        return  true;               //拓扑排序成功
}
```

命题追踪 ▶▶ 不同存储方式下的拓扑排序的效率（2016）

因为输出每个顶点的同时还要删除以它为起点的边，所以采用邻接表存储时拓扑排序的时间复杂度为 $O(|V|+|E|)$，采用邻接矩阵存储时拓扑排序的时间复杂度为 $O(|V|^2)$。

命题追踪 ▶▶ DFS 实现拓扑排序的思想（2020）

此外，利用上一节的深度优先遍历也可以实现拓扑排序，下面简单描述其思路，具体代码见本节后的习题。对于有向无环图 G 中的任意结点 u, v，它们之间的关系必然是下列三种之一：

1）若 u 是 v 的祖先，则在调用 DFS 访问 u 之前，必然已对 v 进行了 DFS 访问，即 v 的 DFS 结束时间先于 u 的 DFS 结束时间。从而可考虑在 DFS 函数中设置一个时间标记，在 DFS 调用结束时，对各顶点计时。因此，祖先的结束时间必然大于子孙的结束时间。

2）若 u 是 v 的子孙，则 v 为 u 的祖先，按上述思路，v 的结束时间大于 u 的结束时间。

3）若 u 和 v 没有路径关系，则 u 和 v 在拓扑序列的关系任意。

于是，按结束时间从大到小排列，就可以得到一个拓扑排序序列。

配套课程公众号[考机研]
永久联系微信:4550060

对一个AOV网，若采用下列步骤进行排序，则称之为逆拓扑排序：

① 从AOV网中选择一个没有后继（出度为0）的顶点并输出。

② 从网中删除该顶点和所有以它为终点的有向边。

③ 重复①和②直到当前的AOV网为空。

用拓扑排序算法处理AOV网时，应注意以下问题：

1）入度为零的顶点，即没有前驱活动的或前驱活动都已经完成的顶点，工程可以从这个顶点所代表的活动开始或继续。

命题追踪 ▶ 拓扑排序序列的存在性和唯一性分析（2011、2024）

2）拓扑排序的结果可能不唯一。不少人误认为AOV网的各顶点为线性序列是拓扑序列唯一的充要条件，而它其实只是充分非必要条件。拓扑序列是否唯一的判断条件是在每次输出顶点时，检测入度为0的顶点是否唯一，若每次都唯一，则说明拓扑序列唯一。

3）AOV网中各顶点的地位平等，每个顶点编号是人为的，因此可以按拓扑排序的结果重新编号，生成AOV网的新的邻接存储矩阵，这种邻接矩阵可以是三角矩阵；但对于一般的图来说，若其邻接矩阵是三角矩阵，则存在拓扑序列；反之则不一定成立。

6.4.5 关键路径

在带权有向图中，以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开销（如完成活动所需的时间），称之为用边表示活动的网络，简称AOE网。AOE网和AOV网都是有向无环图，不同之处在于它们的边和顶点所代表的含义是不同的，AOE网中的边有权值；而AOV网中的边无权值，仅表示顶点之间的前后关系。

AOE网具有以下两个性质：

① 只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始；

② 只有在进入某顶点的各有向边所代表的活动都已结束时，该顶点所代表的事件才能发生。

在AOE网中仅有一个入度为0的顶点，称为开始顶点（源点），它表示整个工程的开始；也仅有一个出度为0的顶点，称为结束顶点（汇点），它表示整个工程的结束。

命题追踪 ▶ 关键路径的性质（2020）

在AOE网中，有些活动是可以并行进行的。从源点到汇点的有向路径可能有多条，并且这些路径长度可能不同。完成不同路径上的活动所需的时间虽然不同，但是只有所有路径上的活动都已完成，整个工程才能算结束。因此，从源点到汇点的所有路径中，具有最大路径长度的路径称为关键路径，而把关键路径上的活动称为关键活动。

完成整个工程的最短时间就是关键路径的长度，即关键路径上各活动花费开销的总和。这是因为关键活动影响了整个工程的时间，即若关键活动不能按时完成，则整个工程的完成时间就会延长。因此，只要找到了关键活动，就找到了关键路径，也就可以得出最短完成时间。

下面给出在寻找关键活动时所用到的几个参量的定义。

1．事件v_k的最早发生时间$v_e(k)$

指从源点v_1到顶点v_k的最长路径长度。事件v_k的最早发生时间决定了所有从v_k开始的活动能够开工的最早时间。可用下面的递推公式来计算：

v_e(源点)$=0$

$v_e(k)=\mathrm{Max}\{v_e(j)+\mathrm{Weight}(v_j, v_k)\}$，$v_k$为$v_j$的任意后继，$\mathrm{Weight}(v_j, v_k)$表示$<v_j, v_k>$上的权值

配套课程公众号[考机研]
永久联系微信:4550060

注　意

计算 $v_e()$值时，按从前往后的顺序进行，可以在拓扑排序的基础上计算：

① 初始时，令 $v_e[1...n]=0$。

② 输出一个入度为 0 的顶点 v_j 时，计算它所有直接后继顶点 v_k 的最早发生时间，若 $v_e[j]+\text{Weight}(v_j, v_k)>v_e[k]$，则 $v_e[k]=v_e[j]+\text{Weight}(v_j, v_k)$。以此类推，直至输出全部顶点。

2．事件 v_k 的最迟发生时间 $v_l(k)$

指在不推迟整个工程完成的前提下，即保证它的后继事件 v_j 在其最迟发生时间 $v_l(j)$能够发生时，该事件最迟必须发生的时间。可用下面的递推公式来计算：

$v_l(\text{汇点})=v_e(\text{汇点})$

$v_l(k)=\text{Min}\{v_l(j)-\text{Weight}(v_k, v_j)\}$，$v_k$ 为 v_j 的任意前驱

注　意

计算 $v_l(k)$值时，按从后往前的顺序进行，可以在逆拓扑排序的基础上计算。增设一个栈以记录拓扑序列，拓扑排序结束后从栈顶至栈底便为逆拓扑有序序列。过程如下：

① 初始时，令 $v_l[1...n]=v_e[n]$。

② 栈顶顶点 v_j 出栈，计算其所有直接前驱顶点 v_k 的最迟发生时间，若 $v_l[j]-\text{Weight}(v_k, v_j)<v_l[k]$，则 $v_l[k]=v_l[j]-\text{Weight}(v_k, v_j)$。以此类推，直至输出全部栈中顶点。

3．活动 a_i 的最早开始时间 $e(i)$

指该活动弧的起点所表示的事件的最早发生时间。若边<v_k, v_j>表示活动 a_i，则有 $e(i)=v_e(k)$。

4．活动 a_i 的最迟开始时间 $l(i)$

指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。若边<v_k, v_j>表示活动 a_i，则有 $l(i)=v_l(j)-\text{Weight}(v_k, v_j)$。

5．一个活动 a_i 的最迟开始时间 $l(i)$和其最早开始时间 $e(i)$的差额 $d(i)=l(i)-e(i)$

指该活动完成的时间余量，即在不增加完成整个工程所需总时间的情况下，活动 a_i 可以拖延的时间。若一个活动的时间余量为零，则说明该活动必须要如期完成，否则就会拖延整个工程的进度，所以称 $l(i)-e(i)=0$ 即 $l(i)=e(i)$的活动 a_i 是关键活动。

命题追踪 ▶ 求关键路径的实例（2019、2022）

求关键路径的算法步骤如下：

① 从源点出发，令 $v_e(\text{源点})=0$，按拓扑有序求其余顶点的最早发生时间 $v_e()$。

② 从汇点出发，令 $v_l(\text{汇点})=v_e(\text{汇点})$，按逆拓扑有序求其余顶点的最迟发生时间 $v_l()$。

③ 根据各顶点的 $v_e()$值求所有弧的最早开始时间 $e()$。

④ 根据各顶点的 $v_l()$值求所有弧的最迟开始时间 $l()$。

⑤ 求 AOE 网中所有活动的差额 $d()$，找出所有 $d()=0$ 的活动构成关键路径。

图 6.23 所示为求解关键路径的过程，简单说明如下：

① 求 $v_e()$：初始 $v_e(1)=0$，在拓扑排序输出顶点过程中，求得 $v_e(2)=3$，$v_e(3)=2$，$v_e(4)=\max\{v_e(2)+2, v_e(3)+4\}=\max\{5, 6\}=6$，$v_e(5)=6$，$v_e(6)=\max\{v_e(5)+1, v_e(4)+2, v_e(3)+3\}=\max\{7, 8, 5\}=8$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

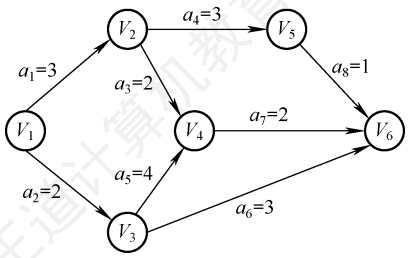
若这是一道选择题，根据上述求 $v_e()$的过程就已经能知道关键路径。

② 求 $v_l()$：初始 $v_l(6)=8$，在逆拓扑排序出栈过程中，求得 $v_l(5)=7$，$v_l(4)=6$，$v_l(3)=\min\{v_l(4)-4, v_l(6)-3\}=\min\{2,5\}=2$，$v_l(2)=\min\{v_l(5)-3, v_l(4)-2\}=\min\{4,4\}=4$，$v_l(1)$必然为 0 而无须再求。

③ 弧的最早开始时间 $e()$等于该弧的起点的顶点的 $v_e()$，结果如下表。

④ 弧的最迟开始时间 $l(i)$等于该弧的终点的顶点的 $v_l()$减去该弧持续的时间，结果如下表。

⑤ 根据 $l(i)-e(i)=0$ 的关键活动，得到的关键路径为(v_1, v_3, v_4, v_6)。



	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	3	2	6	6	8
$v_l(i)$	0	4	2	6	7	8

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(i)$	0	0	3	3	2	2	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	3	0	1

图 6.23　求解关键路径的过程

命题追踪 ▶▶ 缩短工期的相关分析（2013）

对于关键路径，需要注意以下几点：

1）关键路径上的所有活动都是关键活动，它是决定整个工程的关键因素，因此可以通过加快关键活动来缩短整个工程的工期。但也不能任意缩短关键活动，因为一旦缩短到一定的程度，该关键活动就可能会变成非关键活动。

2）网中的关键路径并不唯一，且对于有几条关键路径的网，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的。

各种图算法在采用邻接矩阵或邻接表存储时的时间复杂度如表 6.5 所示。

表 6.5　采用不同存储结构时各种图算法的时间复杂度

	Dijkstra	Floyd	Prim	Kruskal	DFS	BFS	拓扑排序	关键路径
邻接矩阵	$O(n^2)$	$O(n^3)$	$O(n^2)$	-	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
邻接表	-	-	-	$O(e\log_2 e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$

6.4.6　本节试题精选

一、单项选择题

01. 任何一个无向连通图的最小生成树（　）。

A. 有一棵或多棵　　B. 只有一棵

C. 一定有多棵　　D. 可能不存在

02. 用 Prim 算法和 Kruskal 算法构造图的最小生成树，所得到的最小生成树（　）。

A. 相同　　B. 不相同

C. 可能相同，可能不同　　D. 无法比较

03. 下列关于图的生成树和最小生成树的叙述中，正确的是（　）。

配套课程公众号[考机研]
永久联系微信:4550060

A. 只要无向连通图中没有权值相同的边，则其最小生成树唯一

B. 只要无向图中有权值相同的边，则其最小生成树一定不唯一

C. 从 n 个顶点的连通图中选取 $n-1$ 条权值最小的边，即可构成最小生成树

D. 设连通图 G 含有 n 个顶点，则含有 n 个顶点、$n-1$ 条边的子图一定是 G 的生成树

04. 设有 n 个顶点的无向连通图的最小生成树不唯一，则下列说法中正确的是（ ）。

A. 图的边数一定大于 $n-1$　　B. 图的权值最小的边一定有多条

C. 图的最小生成树的代价不一定相等　　D. 图的各条边的权值不相等

05. 用 Prim 算法求一个带权连通图的最小生成树，在算法执行的某个时刻，已选取的顶点集合 $U=\{1, 2, 3\}$，已选取的边集合 TE = $\{(1, 2), (2, 3)\}$，要选取下一条权值最小的边，应当从（ ）组中选取。

A. $\{(1, 4), (3, 4), (3, 5), (2, 5)\}$　　B. $\{(3, 4), (3, 5), (4, 5), (1, 4)\}$

C. $\{(1, 2), (2, 3), (3, 5)\}$　　D. $\{(4, 5), (1, 3), (3, 5)\}$

06. 用 Kruskal 算法求一个带权连通图的最小生成树，在算法执行的某个时刻，已选取的边集合 TE = $\{(1, 2), (2, 3), (3, 5)\}$，要选取下一条权值最小的边，不可能选取的边是（ ）。

A. (3, 6)　　B. (2, 4)　　C. (1, 3)　　D. (1, 4)

07. 下列关于图的最短路径的相关叙述中，正确的是（ ）。

A. 最短路径一定是简单路径

B. Dijkstra 算法不适合求有回路的带权图的最短路径

C. Dijkstra 算法不适合求任意两个顶点的最短路径

D. Floyd 算法求两个顶点的最短路径时，$path_{k-1}$ 一定是 $path_k$ 的子集

08. 下列关于图的最短路径的相关叙述中，正确的是（ ）。

I. Dijkstra 算法求单源最短路径不允许边的权为负

II. Dijkstra 算法求每对顶点间的最短路径的时间复杂度是 $O(n^2)$

III. Floyd 算法求每对顶点间的最短路径允许边的权为负，但不允许含有负权的回路

A. I、II 和 III　　B. 仅 I　　C. I 和 III　　D. II 和 III

09. 已知带权连通无向图 $G=(V, E)$，其中 $V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$，$E=\{(v_1, v_2)10, (v_1, v_3)2, (v_3, v_4)2, (v_3, v_6)11, (v_2, v_5)1, (v_4, v_5)4, (v_4, v_6)6, (v_5, v_7)7, (v_6, v_7)3\}$（注：顶点偶对括号外的数据表示边上的权值），从源点 v_1 到顶点 v_7 的最短路径上经过的顶点序列是（ ）。

A. v_1, v_2, v_5, v_7　　B. v_1, v_3, v_4, v_6, v_7

C. v_1, v_3, v_4, v_5, v_7　　D. $v_1, v_2, v_5, v_4, v_6, v_7$

10. 用 Dijkstra 算法求一个带权有向图的从顶点 0 出发的最短路径，在算法执行的某个时刻，已求得的最短路径的顶点集合 $S=\{0, 2, 3, 4\}$，下一个选取的目标顶点是顶点 1，则可能修改的最短路径是（ ）。

A. 从顶点 0 到顶点 3 的最短路径　　B. 从顶点 0 到顶点 2 的最短路径

C. 从顶点 2 到顶点 4 的最短路径　　D. 从顶点 0 到顶点 1 的最短路径

11. 下面的（ ）方法可以判断出一个有向图是否有环（回路）。

I. 深度优先遍历　II. 拓扑排序　III. 求最短路径　IV. 求关键路径

A. I、II、IV　　B. I、III、IV　　C. I、II、III　　D. 全部可以

12. 在有向图 G 的拓扑序列中，若顶点 v_i 在顶点 v_j 之前，则不可能出现的情形是（ ）。

A. G 中有弧$<v_i, v_j>$　　B. G 中有一条从 v_i 到 v_j 的路径

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

C. G中没有弧$<v_i, v_j>$　　D. G中有一条从v_j到v_i的路径

13. 下列关于拓扑排序的说法中，错误的是（ ）。

I. 若某有向图存在环路，则该有向图一定不存在拓扑排序

II. 在拓扑排序算法中为暂存入度为零的顶点，可以使用栈，也可以使用队列

III. 若有向图的拓扑有序序列唯一，则图中每个顶点的入度和出度最多为1

IV. 若有向图的拓扑有序序列唯一，则图中入度为0和出度为0的顶点都仅有1个

A. I、III、IV　　B. III、IV　　C. II、IV　　D. III

14. 下列关于拓扑排序的说法中，正确的是（ ）。

I. 顶点数大于1的强连通图不能进行拓扑排序

II. 在一个有向图的拓扑序列中，若顶点a在顶点b之前，则图中必有一条弧$<a, b>$

III. 若有向无环图的拓扑序列唯一，则可以唯一确定该图

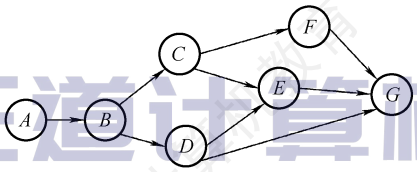
A. I和II　　B. I、II和III　　C. 仅I　　D. I和III

15. 若一个有向图的顶点不能排成一个拓扑序列，则判定该有向图（ ）。

A. 含有多个出度为0的顶点　　B. 是个强连通图

C. 含有多个入度为0的顶点　　D. 含有顶点数大于1的强连通分量

16. 下图所示有向图的所有拓扑序列共有（ ）个。



A. 4　　B. 6　　C. 5　　D. 7

17. 已知有向图$G=(V, E)$，其中$V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$，$E=\{<v_1, v_2>, <v_1, v_3>, <v_1, v_4>, <v_2, v_5>, <v_3, v_5>, <v_3, v_6>, <v_5, v_7>, <v_6, v_7>, <v_4, v_6>\}$，$G$的拓扑序列是（ ）。

A. $\{v_1, v_3, v_4, v_6, v_2, v_5, v_7\}$　　B. $\{v_1, v_3, v_2, v_6, v_4, v_5, v_7\}$

C. $\{v_1, v_3, v_4, v_5, v_2, v_6, v_7\}$　　D. $\{v_1, v_2, v_5, v_3, v_4, v_6, v_7\}$

18. 下列哪种图的邻接矩阵是对称矩阵？（ ）

A. 有向网　　B. 无向图　　C. AOV网　　D. AOE网

19. 若一个有向图具有有序的拓扑排序序列，则它的邻接矩阵必定为（ ）。

A. 对称　　B. 稀疏　　C. 三角　　D. 一般

20. 用DFS算法遍历一个无环有向图，并在DFS算法退栈返回时输出相应的顶点，则输出的顶点序列是（ ）。

A. 逆拓扑有序　　B. 拓扑有序　　C. 无序的　　D. 无法确定

21. 下列关于图的说法中，正确的是（ ）。

I. 有向图中顶点V的度等于其邻接矩阵中第V行中1的个数

II. 无向图的邻接矩阵一定是对称矩阵，有向图的邻接矩阵一定是非对称矩阵

III. 在带权图G的最小生成树G_1中，某条边的权值可能会超过未选边的权值

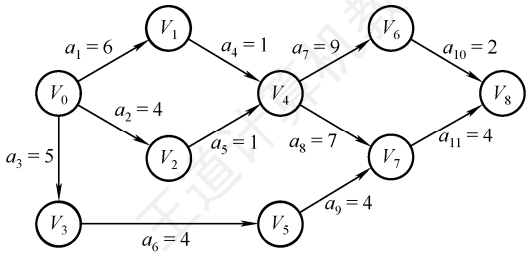
IV. 若有向无环图的拓扑序列唯一，则可以唯一确定该图

A. I、II和III　　B. III和IV　　C. III　　D. IV

22. 下图所示的AOE网中，关键路径长度为（ ）。

A. 16　　B. 17　　C. 18　　D. 19

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



23. 若某带权图为 $G=(V,E)$，其中 $V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$，$E=\{<v_1, v_2>5, <v_1, v_3>6, <v_2, v_5>3, <v_3, v_5>6, <v_3, v_4>3, <v_4, v_5>3, <v_4, v_7>1, <v_4, v_8>4, <v_5, v_6>4, <v_5, v_7>2, <v_6, v_{10}>4, <v_7, v_9>5, <v_8, v_9>2, <v_9, v_{10}>2\}$（注：边括号外的数据表示边上的权值），则 G 的关键路径的长度为（　）。

A. 19　　B. 20　　C. 21　　D. 22

24. 下面关于求关键路径的说法中，不正确的是（　）。

A. 求关键路径是以拓扑排序为基础的

B. 一个事件的最早发生时间与以该事件为始的弧的活动的最早开始时间相同

C. 一个事件的最迟发生时间是以该事件为尾的弧的活动的最迟开始时间与该活动的持续时间的差

D. 任何一个活动的持续时间的改变可能会影响关键路径的改变

25. 下列关于 AOE 网的关键路径的说法中，正确的是（　）。

I. 改变网上某一关键路径上的任意一个关键活动后，必将产生不同的关键路径

II. 关键路径上活动的时间延长多少，整个工期也就随之延长多少

III. 缩短关键路径上任意一个关键活动的持续时间可缩短关键路径长度

IV. 缩短所有关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度

V. 缩短多条关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度

A. II 和 V　　B. I、II 和 IV　　C. II 和 IV　　D. I 和 IV

26. 在求 AOE 网的关键路径时，若该有向图用邻接矩阵表示且第 i 列值全为∞，则（　）。

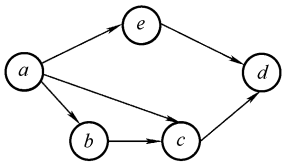
A. 若关键路径存在，第 i 个顶点一定是起点

B. 若关键路径存在，第 i 个顶点一定是终点

C. 关键路径不存在

D. 该有向图对应的无向图存在多个连通分量

27. 【2010 统考真题】对下图进行拓扑排序，可得不同拓扑序列的个数是（　）。



A. 4　　B. 3　　C. 2　　D. 1

28. 【2012 统考真题】下列关于最小生成树的叙述中，正确的是（　）。

I. 最小生成树的代价唯一

II. 所有权值最小的边一定会出现在所有的最小生成树中

III. 使用 Prim 算法从不同顶点开始得到的最小生成树一定相同

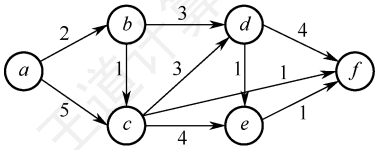
IV. 使用 Prim 算法和 Kruskal 算法得到的最小生成树总不相同

A. 仅 I　　B. 仅 II　　C. 仅 I、III　　D. 仅 II、IV

29. 【2012 统考真题】对下图所示的有向带权图，若采用 Dijkstra 算法求从源点 a 到其他各

配套课程公众号[考机研]
永久联系微信:4550060

顶点的最短路径，则得到的第一条最短路径的目标顶点是 b，第二条最短路径的目标顶点是 c，后续得到的其余各最短路径的目标顶点依次是（ ）。



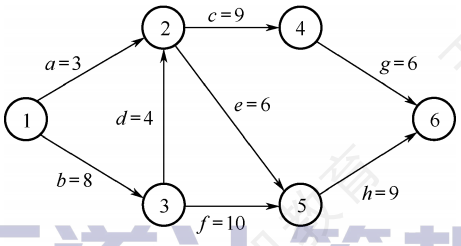
A. d, e, f　　B. e, d, f　　C. f, d, e　　D. f, e, d

30. 【2012 统考真题】若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的结论是（ ）。

A. 存在，且唯一　　B. 存在，且不唯一

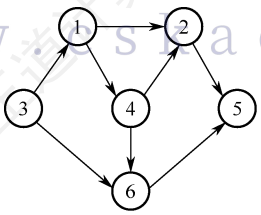
C. 存在，可能不唯一　　D. 无法确定是否存在

31. 【2013 统考真题】下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度可缩短整个工程的工期。在下列选项中，加快其进度就可缩短工程工期的是（ ）。



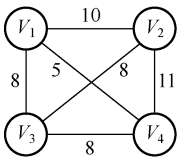
A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

32. 【2014 统考真题】对下图所示的有向图进行拓扑排序，得到的拓扑序列可能是（ ）。



A. 3, 1, 2, 4, 5, 6　　B. 3, 1, 2, 4, 6, 5　　C. 3, 1, 4, 2, 5, 6　　D. 3, 1, 4, 2, 6, 5

33. 【2015 统考真题】求下面的带权图的最小（代价）生成树时，可能是 Kruskal 算法第 2 次选中但不是 Prim 算法（从 V_4 开始）第 2 次选中的边是（ ）。



A. (V_1, V_3)　　B. (V_1, V_4)　　C. (V_2, V_3)　　D. (V_3, V_4)

34. 【2011 统考真题】下列关于图的叙述中，正确的是（ ）。

I. 回路是简单路径

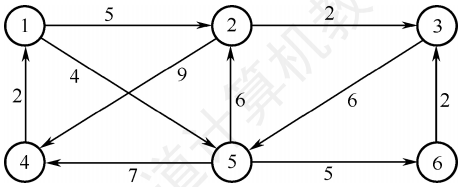
II. 存储稀疏图，用邻接矩阵比邻接表更省空间

III. 若有向图中存在拓扑序列，则该图不存在回路

A. 仅 II　　B. 仅 I、II　　C. 仅 III　　D. 仅 I、III

35. 【2016 统考真题】使用 Dijkstra 算法求下图中从顶点 1 到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（ ）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

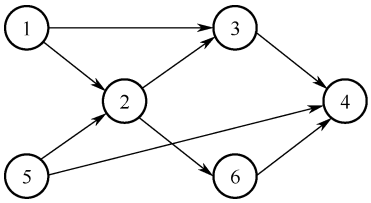


A．5, 2, 3, 4, 6　　B．5, 2, 3, 6, 4　　C．5, 2, 4, 3, 6　　D．5, 2, 6, 3, 4

36. 【2016 统考真题】若对 n 个顶点、e 条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是（　）。

A．$O(n)$　　B．$O(n+e)$　　C．$O(n^2)$　　D．$O(ne)$

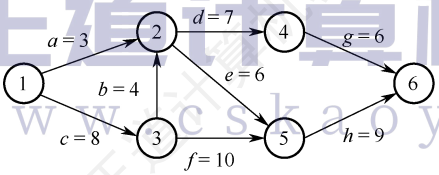
37. 【2018 统考真题】下列选项中，不是如下有向图的拓扑序列的是（　）。



A．1, 5, 2, 3, 6, 4　　B．5, 1, 2, 6, 3, 4

C．5, 1, 2, 3, 6, 4　　D．5, 2, 1, 6, 3, 4

38. 【2019 统考真题】下图所示的 AOE 网表示一项包含 8 个活动的工程。活动 d 的最早开始时间和最迟开始时间分别是（　）。

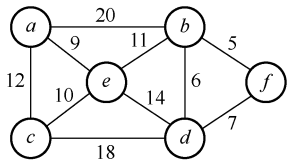


A．3 和 7　　B．12 和 12　　C．12 和 14　　D．15 和 15

39. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$，需要的顶点个数至少是（　）。

A．5　　B．6　　C．8　　D．9

40. 【2020 统考真题】已知无向图 G 如下所示，使用 Kruskal 算法求图 G 的最小生成树，加到最小生成树中的边依次是（　）。



A．$(b,f),(b,d),(a,e),(c,e),(b,e)$　　B．$(b,f),(b,d),(b,e),(a,e),(c,e)$

C．$(a,e),(b,e),(c,e),(b,d),(b,f)$　　D．$(a,e),(c,e),(b,e),(b,f),(b,d)$

41. 【2020 统考真题】修改递归方式实现的图的深度优先搜索（DFS）算法，将输出（访问）顶点信息的语句移到退出递归前（执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图 G，若输出结果中包含 G 中的全部顶点，则输出的顶点序列是 G 的（　）。

A．拓扑有序序列　　B．逆拓扑有序序列

配套课程公众号[考机研]
永久联系微信:4550060

C. 广度优先搜索序列　　D. 深度优先搜索序列

42.【2020统考真题】若使用AOE网估算工程进度，则下列叙述中正确的是（ ）。

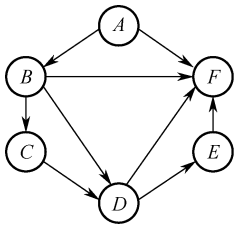
A. 关键路径是从源点到汇点边数最多的一条路径

B. 关键路径是从源点到汇点路径长度最长的路径

C. 增加任意一个关键活动的时间不会延长工程的工期

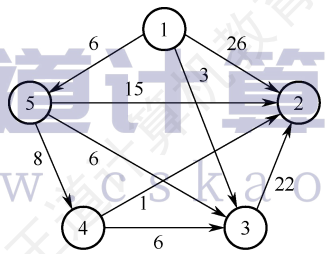
D. 缩短任意一个关键活动的时间将会缩短工程的工期

43.【2021统考真题】给定如下有向图，该图的拓扑有序序列的个数是（ ）。



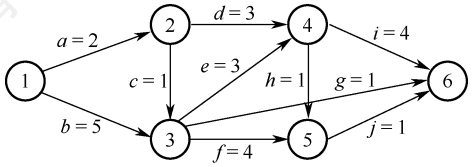
A. 1　　B. 2　　C. 3　　D. 4

44.【2021统考真题】使用Dijkstra算法求下图中从顶点1到其余各顶点的最短路径，将当前找到的从顶点1到顶点2, 3, 4, 5的最短路径长度保存在数组dist中，求出第二条最短路径后，dist中的内容更新为（ ）。



A. 26, 3, 14, 6　　B. 25, 3, 14, 6　　C. 21, 3, 14, 6　　D. 15, 3, 14, 6

45.【2022统考真题】下图是一个有10个活动的AOE网，时间余量最大的活动是（ ）。



A. c　　B. g　　C. h　　D. j

46.【2023统考真题】已知无向连通图G中各边的权值均为1。在下列算法中，一定能够求出图G中从某顶点到其余各顶点最短路径的是（ ）。

I. Prim算法　II. Kruskal算法　III. 图的广度优先搜索算法

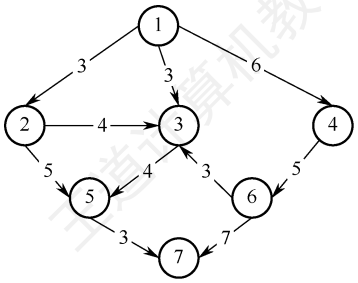
A. 仅I　　B. 仅III　　C. 仅I、II　　D. I、II、III

二、综合应用题

01. 下面是一种称为“破圈法”的求解最小生成树的方法：所谓“破圈法”，是指“任取一圈，去掉圈上权最大的边”，反复执行这一步骤，直到没有圈为止。试判断这种方法是否正确。若正确，说明理由；若不正确，举出反例（注：圈就是回路）。

02. 已知有向图如下图所示。

配套课程公众号[考机研]
永久联系微信:4550060



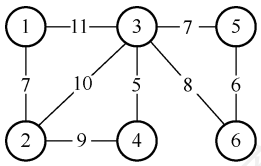
1）写出该图的邻接矩阵表示并据此给出从顶点 1 出发的深度优先遍历序列。

2）求该有向图的强连通分量的数目。

3）给出该图的任意两个拓扑序列。

4）若将该图视为无向图，分别用 Prim 算法和 Kruskal 算法求最小生成树。

03. 对下图所示的无向图，按照 Dijkstra 算法，写出从顶点 1 到其他各个顶点的最短路径和最短路径长度（顺序不能颠倒）。



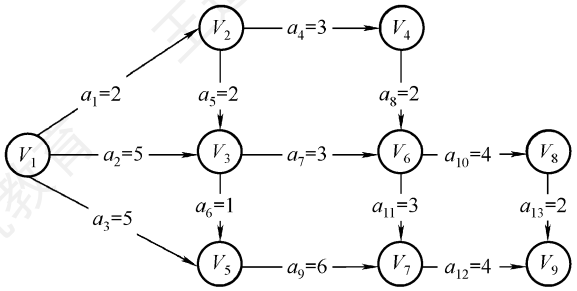
04. 下图所示为一个用 AOE 网表示的工程。

1）画出此图的邻接表表示。

2）完成此工程至少需要多少时间？

3）指出关键路径。

4）哪些活动加速可以缩短完成工程所需的时间？



05. 下表给出了某工程各工序之间的优先关系和各工序所需的时间（其中“—”表示无先驱工序），请完成以下各题：

1）画出相应的 AOE 网。

2）列出各事件的最早发生时间和最迟发生时间。

3）求出关键路径并指明完成该工程所需的最短时间。

工序代号	A	B	C	D	E	F	G	H
所需时间	3	2	2	3	4	3	2	1
先驱工序	—	—	A	A	B	A	C、E	D

06. 一连通无向图，边非负权值，问用 Dijkstra 最短路径算法能否给出一棵生成树，该树是否一定是最小生成树？说明理由。

07. 试编写利用 DFS 实现有向无环图拓扑排序的算法。

王道计算机教育 www.wangdaokaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

08. 【2009 统考真题】带权图（权值非负，表示边连接的两顶点间的距离）的最短路径问题是找出从初始顶点到目标顶点之间的一条最短路径。假设从初始顶点到目标顶点之间存在路径，现有一种解决该问题的方法：

① 设最短路径初始时仅包含初始顶点，令当前顶点 u 为初始顶点。

② 选择离 u 最近且尚未在最短路径中的一个顶点 v，加入最短路径，修改当前顶点 $u=v$。

③ 重复步骤②，直到 u 是目标顶点时为止。

请问上述方法能否求得最短路径？若该方法可行，请证明；否则，请举例说明。

09. 【2011 统考真题】已知有 6 个顶点（顶点编号为 0～5）的有向带权图 G，其邻接矩阵 A 为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。

4	6	∞	∞	∞	5	∞	∞	∞	4	3	∞	∞	3	3

要求：

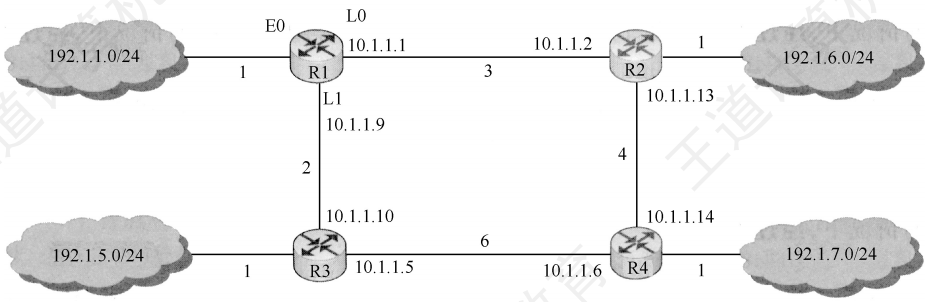
1）写出图 G 的邻接矩阵 A。

2）画出有向带权图 G。

3）求图 G 的关键路径，并计算该关键路径的长度。

10. 【2014 统考真题】某网络中的路由器运行 OSPF 路由协议，下表是路由器 R1 维护的主要链路状态信息（LSI），R1 构造的网络拓扑图（见下图）是根据题下表及 R1 的接口名构造出来的网络拓扑。

		R1 的 LSI	R2 的 LSI	R3 的 LSI	R4 的 LSI	备　注
Router ID		10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	标识路由器的 IP 地址
Link1	ID	10.1.1.2	10.1.1.1	10.1.1.6	10.1.1.5	所连路由器的 Router ID
	IP	10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	Link1 的本地 IP 地址
	Metric	3	3	6	6	Link1 的费用
Link2	ID	10.1.1.5	10.1.1.6	10.1.1.1	10.1.1.2	所连路由器的 Router ID
	IP	10.1.1.9	10.1.1.13	10.1.1.10	10.1.1.14	Link2 的本地 IP 地址
	Metric	2	4	2	4	Link2 的费用
Net1	Prefix	192.1.1.0/24	192.1.6.0/24	192.1.5.0/24	192.1.7.0/24	直连网络 Net1 的网络前缀
	Metric	1	1	1	1	到达直连网络 Net1 的费用



请回答下列问题。

1）本题中的网络可抽象为数据结构中的哪种逻辑结构？

2）针对表中的内容，设计合理的链式存储结构，以保存表中的链路状态信息（LSI）。要求给出链式存储结构的数据类型定义，并画出对应表的链式存储结构示意图（示意图中可仅以 ID 标识结点）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

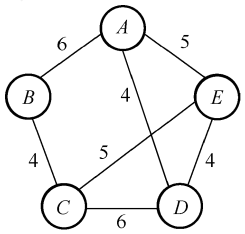
3）按照 Dijkstra 算法的策略，依次给出 R1 到达子网 192.1.x.x 的最短路径及费用。

11. 【2017 统考真题】使用 Prim 算法求带权连通图的最小（代价）生成树（MST）。请回答下列问题：

1）对下列图 G，从顶点 A 开始求 G 的 MST，依次给出按算法选出的边。

2）图 G 的 MST 是唯一的吗？

3）对任意的带权连通图，满足什么条件时，其 MST 是唯一的？



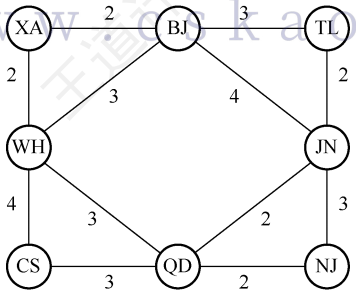
12. 【2018 统考真题】拟建设一个光通信骨干网络连通 BJ、CS、XA、QD、JN、NJ、TL 和 WH 等 8 个城市，下图中无向边上的权值表示两个城市之间备选光缆的铺设费用。

请回答下列问题：

1）仅从铺设费用角度出发，给出所有可能的最经济的光缆铺设方案（用带权图表示），并计算相应方案的总费用。

2）该图可采用图的哪种存储结构？给出求解问题 1）所用的算法名称。

3）假设每个城市采用一个路由器按 1）中得到的最经济方案组网，主机 H1 直接连接 TL 的路由器，主机 H2 直接连接 BJ 的路由器。若 H1 向 H2 发送一个 TTL = 5 的 IP 分组，则 H2 是否可以收到该 IP 分组？



13. 【2024 统考真题】2023 年 10 月 26 日，神舟十七号载人飞船发射取得圆满成功，再次彰显了中国航天事业的辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为了保证工程的有序开展，需要明确各子工程的前导子工程，以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储，类型定义如下。

```
typedef struct                          //图的类型定义
{
    int numVertices, numEdges;          //图的顶点数和有向边数
    char VerticesList[MAXV];            //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];               //邻接矩阵
} MGraph;
```

请设计算法：int uniquely(MGraqh G)，判定 G 是否存在唯一的拓扑序列，若是，则返回 1，否则返回 0。要求如下。

1）给出算法的基本设计思想。

配套课程公众号[考机研]
永久联系微信:4550060

2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

6.4.7 答案与解析

一、单项选择题

01. A

当无向连通图存在权值相同的多条边时，最小生成树可能是不唯一的；另外，这是一个无向连通图，因此最小生成树必定存在，从而选择A。

02. C

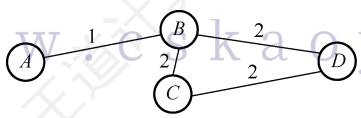
因为无向连通图的最小生成树不一定唯一，所以用不同算法生成的最小生成树可能不同，但当无向连通图的最小生成树唯一时，不同算法生成的最小生成树必定是相同的。

03. A

最小生成树算法是基于贪心策略的，每次总是选取权值最小且满足条件的边，若各边权值不同，则每次选择的新顶点也是唯一的，因此最小生成树唯一，A正确。对于B，若无向图本身就是一棵树，则最小生成树就是它本身，这时就是唯一的。对于C，选取的$n-1$条边可能构成回路。对于D，含有n个顶点、$n-1$条边的子图可能构成回路，也可能不连通。

04. A

若图的边数小于$n-1$，则图不存在最小生成树；若无向连通图的边数等于$n-1$，则最小生成树唯一，即图本身，所以图的边数一定大于$n-1$，A正确。若最小生成树不唯一，则一定存在权值相等的边，但未必是权值最小的边，如下图所示，B错误。最小生成树可能不唯一，但代价一定相同，C错误。当图的各边的权值互不相等时，图的最小生成树是唯一的，D错误。



05. A

$U=\{1,2,3\}$，$V-U=\{4,5,\cdots\}$，候选边只能是这两个顶点集之间的边，只有选项A符合题意。

06. C

若选取边(1, 3)则会构成回路。

07. A

选项A正确，见严蔚敏老师的《数据结构》教材。Dijkstra算法适合求解有回路的带权图的最短路径，也可以求任意两个顶点的最短路径，不适合求带负权值的最短路径问题。在用Floyd算法求两个顶点的最短路径时，当最短路径发生更改时，$path_{k-1}$就不是$path_k$的子集。

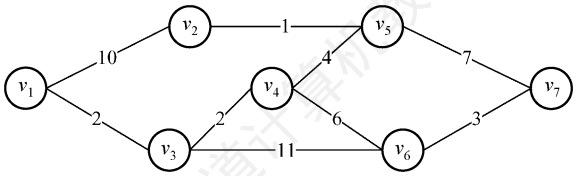
08. C

在负权图中，Dijkstra算法既不能保证每次选出的顶点都是真正的最近顶点，又不能保证已确定的最短路径不再被改变，因此Dijkstra算法不允许边的权为负，选项I正确。求每对顶点间的最短路径需要调用Dijkstra算法n次，时间复杂度为$O(n^3)$，选项II错误。Floyd算法求每对顶点间的最短路径允许有负边的存在，但不允许包含总权值为负的回路，选项III正确。

09. B

题目所描述的图G如下图所示。A, B, C, D对应的路径长度分别为18, 13, 15, 24。应用Dijkstra算法不难求出最短路径为$v_1\rightarrow v_3\rightarrow v_4\rightarrow v_6\rightarrow v_7$。

配套课程公众号[考机研]
永久联系微信:4550060



10．D

在 Dijkstra 算法的执行过程中，只可能修改从源点 0 到集合 $V-S$ 中某个顶点的最短路径。

11．A

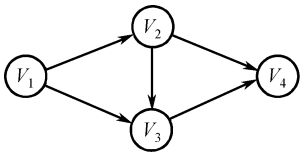
使用深度优先遍历，若从有向图上的某个顶点 u 出发，在 DFS(u)结束之前出现一条从顶点 v 到 u 的边， v 在生成树上是 u 的子孙，图中必定存在包含 u 和 v 的环，因此深度优先遍历可以检测一个有向图是否有环。拓扑排序时，当某顶点不为任何边的头时才能加入序列，存在环时环中的顶点一直是某条边的头，不能加入拓扑序列。也就是说，还存在无法找到下一个可以加入拓扑序列的顶点，则说明此图存在回路。求最短路径是允许图有环的。至于关键路径能否判断一个图有环，则存在一些争议。关键路径本身虽然不允许有环，但求关键路径的算法本身无法判断是否有环，判断是否有环是求关键路径的第一步——拓扑排序。

12．D

若图 G 中存在一条从 v_j 到 v_i 的路径，说明 V_j 是 V_i 的前驱，则要把 V_j 消去以后才能消去 V_i，从而拓扑序列中必然先输出 v_j，再输出 v_i，这显然与题意矛盾。

13．D

对于 I，若有向图中存在环，运行拓扑排序算法后，肯定会剩下有环的子图，在此环中无法再找到入度为 0 的顶点，拓扑排序也就无法再运行。对于 II，若两个顶点之间不存在祖先或子孙关系，则它们在拓扑序列中的前后关系是任意的，因此使用栈和队列都可以，因为入栈或队列的都是入度为 0 的顶点。III 项是难点，若拓扑序列唯一，则很自然联想到一个线性的有向图，下图的拓扑序列也唯一，但却不满足该条件。对于 IV，若入度为 0 的顶点不唯一，则这些顶点均可作为拓扑序列的起点；若出度为 0 的顶点不唯一，则这些顶点均可作为拓扑序列的终点。



14．C

强连通图是指有向图中任意顶点对之间都存在两条相反的路径，这意味着强连通图中一定存在环，因此不能进行拓扑排序，选项 I 正确。假设顶点 a 和 b 的入度均为 0，且分别有两条孤从 a 和 b 指向同一顶点 c，则产生的拓扑序列可以是 abc，但是此时并无一条弧<a, b>，选项 II 错误。如图 1 和 2 所示的有向图对应的拓扑序列都是 $abcd$，且都是唯一的，选项 III 错误。

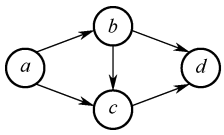


图 1

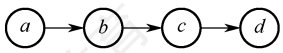


图 2

15．D

一个有向图中的顶点不能排成一个拓扑序列，表明其中存在一个顶点数目大于 1 的回路（环），该回路构成一个强连通分量，从而答案选择选项 D。

配套课程公众号[考机研]
永久联系微信:4550060

16．C

本图的拓扑排序序列有 *ABCFDEG*, *ABCDFEG*, *ABCDEFG*, *ABDCFEG* 和 *ABDCEFG*。读者应能把这一类经典习题的拓扑序列全部写出来。

17．A

拓扑序列的过程：找到入度为 0 的顶点，删除该顶点及其所有出边，并将顶点加入拓扑序列，重复直至所有顶点都加入拓扑序列。选择入度为 0 的顶点 v_1，删除与 v_1 有关的边；此时顶点 v_3 的入度为 0，选择 v_3，删除与 v_3 有关的边；以此类推，得出 G 的拓扑序列。

18．B

无向图的邻接矩阵存储中，每条边存储两次，且 $A[i][j]=A[j][i]$。

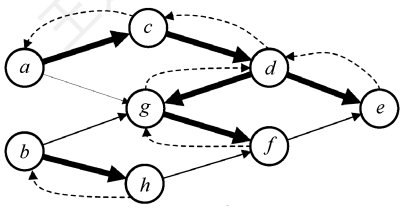
19．C

此题一直以来争议较大，因为有些书中漏掉了“有序”二字。可以证明，对有向图中的顶点适当地编号，使其邻接矩阵为三角矩阵且主对角元素全为零的充分必要条件是，该有向图可以进行拓扑排序。若这个题目把“有序”二字去掉，显然应选择选项 D。但此题题干中已经指出是“有序的拓扑序列”，因此应选选项 C。需要注意的是，若一个有向图的邻接矩阵为三角矩阵（对角线以上或以下的元素为 0），则图中必不存在环，因此其拓扑序列必然存在。

20．A

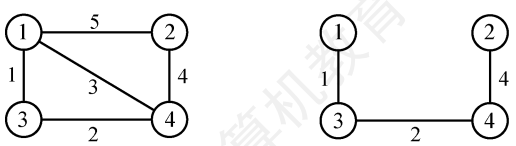
设图中有顶点 v_i，它有后继顶点 v_j，即存在边$<v_i, v_j>$。根据 DFS 的规则，v_i 入栈后，必先遍历完其后继顶点后 v_i 才会出栈，也就是说 v_i 会在 v_j 之后出栈，在如题所指的过程中，v_i 在 v_j 后打印。v_i 和 v_j 具有任意性，因此由上面的规律看出，输出顶点的序列是逆拓扑有序序列。

对有向无环图利用深度优先搜索进行拓扑排序的例子如下：如下图所示，退出 DFS 栈的顺序为 *efgdcahb*，此图的一个拓扑序列为 *bhacdgfe*。该方法的每一步均是先输出当前无后继的结点，即对每个结点 v，先递归地求出 v 的每个后继的拓扑序列。



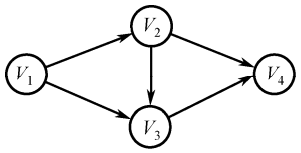
21．C

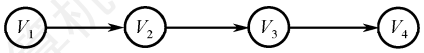
有向图邻接矩阵的第 V 行中 1 的个数是顶点 V 的出度，而有向图中顶点的度为入度与出度之和，选项 I 错。无向图的邻接矩阵一定是对称矩阵，但当有向图中任意两个顶点之间有边相连，且是两条方向相反的有向边时，有向图的邻接矩阵也是一个对称矩阵，选项 II 错。最小生成树中的 $n-1$ 条边不能保证是图中权值最小的 $n-1$ 条边，因为权值最小的 $n-1$ 条边并不一定能使图连通。在下图中，左图的最小生成树如下图所示，权值为 3 的边不在其最小生成树中。



有向无环图的拓扑序列唯一并不能唯一确定该图。在下图所示的两个有向无环图中，拓扑序列都为 V_1, V_2, V_3, V_4，IV 错。注意，很多辅导书对该命题的判断是错误的。

配套课程公众号[考机研]
永久联系微信:4550060



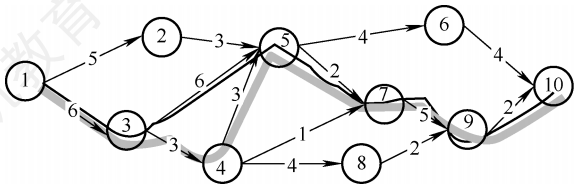


22．C

观察题图，从 V_0 到 V_8 的最长路径为 $V_0 \rightarrow V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$，长度为 6 + 1 + 9 + 2 = 18。

23．C

题目描述的图如下，得到关键路径的长度为 21，图中画出的两条路径都是关键路径。



24．C

一个事件的最迟发生时间 = min{以该事件为尾的弧的活动的最迟开始时间}，或 min{以该事件为尾的弧所指事件的最迟发生时间与该弧的活动的持续时间之差}。改变 AOE 网中任何一个活动的持续时间，需要重新计算关键活动，可能导致关键路径的改变。

25．C

若改变的是所有关键路径上的公共活动，则不一定会产生不同的关键路径（延长必然不会导致，只有缩短才有可能导致）。根据关键路径的定义，可知选项 II 正确。关键路径是源点到终点的最长路径，只有所有关键路径的长度都缩短时，整个图的关键路径才能有效缩短，但也不能任意缩短，一旦缩短到一定程度，该关键活动就可能变成非关键活动。

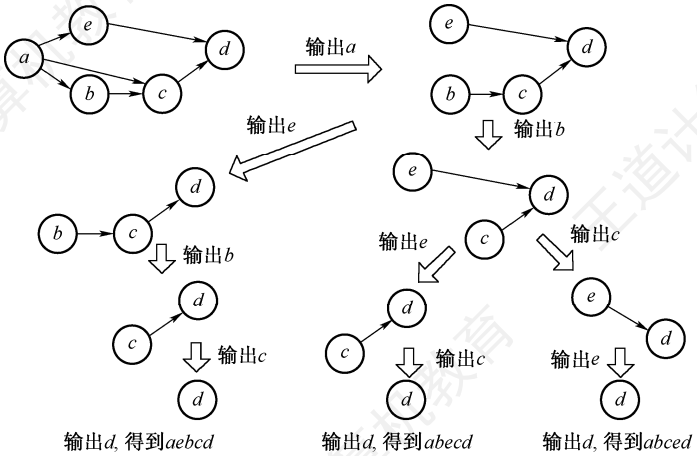
www.cskaoyan.com

26．A

邻接矩阵第 i 列值全为∞，说明顶点 i 没有入边，为整个工程的开始，若关键路径存在，则该顶点一定是起点。不能确定关键路径是否存在，也不能确定其对应的无向图的连通分量个数。

27．B

拓扑排序的过程如下图所示。



可以得到 3 种不同的拓扑序列，即 *abced*, *abecd* 和 *aebcd*。

28．A

最小生成树的树形可能不唯一（因为可能存在权值相同的边），但代价一定是唯一的，I 正

配套课程公众号[考机研]
永久联系微信:4550060

确。若权值最小的边有多条并且构成环状，则总有权值最小的边将不出现在某棵最小生成树中，II 错误。设 N 个结点构成环，N 条边的权值相等，从不同的顶点开始执行 Prim 算法，只要选取任意不同的 $N-1$ 条边，就能得到不同的最小生成树，III 错误。当最小生成树唯一时（各边的权值不同），Prim 算法和 Kruskal 算法得到的最小生成树相同，IV 错误。

29．C

从 a 到各顶点的最短路径的求解过程下：

顶点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
b	(a, b) 2				
c	(a, c) 5	(a, b, c) 3			
d	∞	(a, b, d) 5	(a, b, d) 5	(a, b, d) 5	
e	∞	∞	(a, b, c, e) 7	(a, b, c, e) 7	(a, b, d, e) 6
f	∞	∞	(a, b, c, f) 4		
集合 S	$\{a, b\}$	$\{a, b, c\}$	$\{a, b, c, f\}$	$\{a, b, c, f, d\}$	$\{a, b, c, f, d, e\}$

后续目标顶点依次为 f, d, e。

本题也可用排除法：对于 A，若下一个顶点为 d，路径 a, b, d 的长度为 5，而 a, b, c, f 的长度仅为 4，显然错误。同理可排除 B。将 f 加入集合 S 后，采用上述方法也可排除 D。

30．C

对角线以下元素均为零，表明只有顶点 i 到 j（$i<j$）可能有边，而顶点 j 到 i 一定没有边，即有向图是一个无环图，因此一定存在拓扑序列。对于拓扑序列是否唯一，试举一例：设有向图的邻接矩阵 $\begin{bmatrix}0&1&1\\0&0&0\\0&0&0\end{bmatrix}$，存在两个拓扑序列，因此该图存在可能不唯一的拓扑序列。

结论：对于任一有向图，若它的邻接矩阵中对角线以下（或以上）的元素均为零，则存在拓扑序列（可能不唯一）。反之，若图存在拓扑序列，却不一定能满足邻接矩阵中主对角线以下的元素均为零，但是可以通过适当地调整结点编号，使其邻接矩阵满足前述性质。

31．C

找出 AOE 网的全部关键路径为 *bdcg*、*bdeh* 和 *bfh*。根据性质，只有当所有关键路径的活动时间同时减少时，才能缩短工期。即正确选项中的路径必须能涵盖所有的关键路径。选项 A 和 B 不能涵盖 *bfh* 这条路径，选项 D 不能涵盖 *bdcg* 这条路径，只有选项 C 能涵盖所有关键路径，因此只有加快 f 和 d 的进度才能缩短工期（建议在图中检验）。

32．D

按照拓扑排序的算法，每次都选择入度为 0 的结点从图中删除，此图中一开始只有结点 3 的入度为 0；删除结点 3 后，只有结点 1 的入度为 0；删除结点 1 后，只有结点 4 的入度为 0；删除结点 4 后，结点 2 和结点 6 的入度都为 0，此时选择删除不同的结点，会得出不同的拓扑序列，分别处理完毕后可知可能的拓扑序列为 3, 1, 4, 2, 6, 5 和 3, 1, 4, 6, 2, 5。

33．C

从 V_4 开始，Kruskal 算法选中的第一条边一定是权值最小的(V_1, V_4)，B 错误。V_1 和 V_4 已经可达，因此含有 V_1 和 V_4 的权值为 8 的第二条边一定符合 Prim 算法，排除 A 和 D。

34．C

第一个顶点和最后一个顶点相同的路径称为回路；序列中顶点不重复出现的路径称为简单路径；回路显然不是简单路径，选项 I 错误。稀疏图是边比较少的情况，邻接矩阵存储的空间复杂度为 $O(n^2)$，必将浪费大量的空间，而邻接表存储的空间复杂度为 $O(n+e)$，所以应选用邻接

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

表，选项 II 错误。存在回路的有向图不存在拓扑序列，若拓扑排序输出结束后所余下的顶点都有前驱，则说明只得到了部分顶点的拓扑有序序列，图中存在回路，选项 III 正确。

35．B

根据 Dijkstra 算法，从顶点 1 到其余各顶点的最短路径如下表所示。

顶　点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
2	5 $v_1 \rightarrow v_2$	5 $v_1 \rightarrow v_2$			
3	∞	∞	7 $v_1 \rightarrow v_2 \rightarrow v_3$		
4	∞	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$
5	4 $v_1 \rightarrow v_5$				
6	∞	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	
集合 S	{1, 5}	{1, 5, 2}	{1, 5, 2, 3}	{1, 5, 2, 3, 6}	{1, 5, 2, 3, 6, 4}

快速解法。依次观察从顶点 1 到其他顶点的最短路径长度：顶点 1 到顶点 2 的最短路径长度为 5；顶点 1 到顶点 3 的最短路径长度为 5 + 2 = 7；顶点 1 到顶点 4 的最短路径长度为 11；顶点 1 到顶点 5 的最短路径长度为 4；顶点 1 到顶点 6 的最短路径长度为 4 + 5 = 9；最终 `dist[]`={0, 5, 7, 11, 4, 9}，根据 `dist` 数组值从小到大选择顶点顺序为 1, 5, 2, 3, 6, 4。

36．B

采用邻接表作为 AOV 网的存储结构进行拓扑排序，需要对 n 个顶点做入栈、出栈、输出各一次，当处理 e 条边时，需要检测这 n 个顶点的边链表结点，共需要的时间为 $O(n+e)$。若采用邻接矩阵作为 AOV 网的存储结构进行拓扑排序，在处理 e 条边时需对每个顶点检测相应矩阵中的某一行，寻找与它相关联的边，以便对这些顶点的入度减 1，需要的时间代价为 $O(n^2)$。

【补充】有两种常用的拓扑排序算法：基于 BFS 的算法和基于 DFS 的算法。本题未指明采用哪种算法，因此只需验证一种算法即可（说明两种算法在对应条件下的时间复杂度相同）。

基于 BFS 的算法的思想：首先找到所有入度为 0 的结点，将它们加入一个队列，并将它们作为拓扑序列的起始部分；然后依次从队列中取出结点，并删除它们与后继结点的所有边。若某个后继结点的入度变为 0，则将它也加入队列，并将它加入拓扑序列，重复这个过程。

基于 DFS 的算法的思想：在 DFS 调用过程中设定一个时间标记，当 DFS 调用结束时，对各结点计时，进而按结束时间从大到小排序，可以得到一个拓扑序列。

37．D

拓扑排序每次选取入度为 0 的结点输出，经观察不难发现拓扑序列前两位一定是 1, 5 或 5, 1（因为只有 1 和 5 的入度均为 0，且其他结点都不满足仅有 1 或仅有 5 作为前驱）。

38．C

活动 d 的最早开始时间等于该活动弧的起点所表示的事件的最早发生时间，活动 d 的最早开始时间等于事件 2 的最早发生时间 $\max\{a, b+c\} = \max\{3, 12\} = 12$。活动 d 的最迟开始时间等于该活动弧的终点所表示的事件的最迟发生时间与该活动所需时间之差，先算出图中关键路径长度为 27（对于不复杂的选择题，找出所有路径计算长度），那么事件 4 的最迟发生时间为 $\min\{27-g\} = \min\{27-6\} = 21$，活动 d 的最迟开始时间为 $21-d = 21-7 = 14$。

常规方法：按照关键路径算法算得到下表。

配套课程公众号[考机研]
永久联系微信:4550060

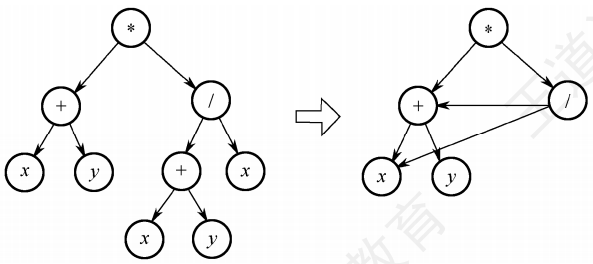
	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	12	8	19	18	27
$v_l(i)$	0	12	8	21	18	27

	a	b	c	d	e	f	g	h
$e(i)$	0	8	0	12	12	8	19	18
$l(i)$	9	8	0	14	12	8	21	18
$l(i)-e(i)$	9	0	0	2	0	0	2	0

从表中可知，活动 d 的最早开始时间和最迟开始时间分别为 12 和 14，所以选择选项 C。

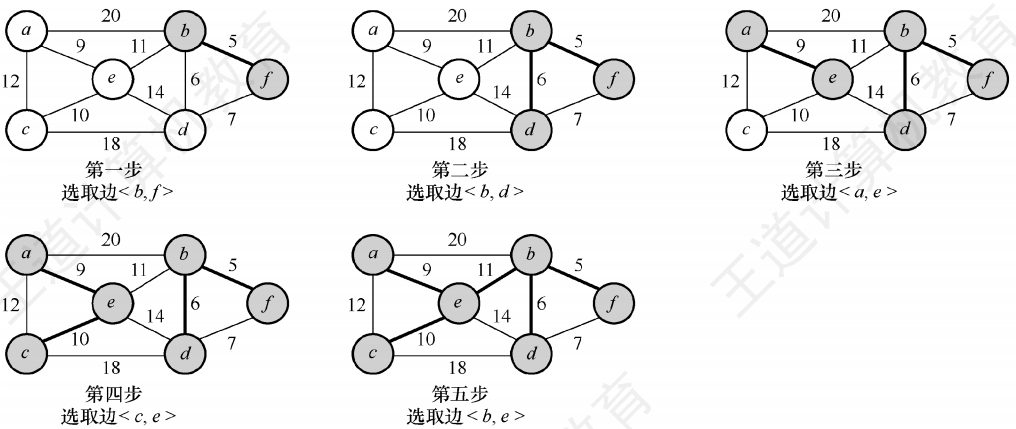
39. A

先将该表达式转换成有向二叉树，该二叉树中有些顶点是重复的，为了节省存储空间，去除重复的顶点，将有向二叉树去重转换成有向无环图，如下图所示。



40. A

Kruskal 算法：按权值递增顺序依次选取 $n-1$ 条边，并保证这 $n-1$ 条边不构成回路。初始构造一个仅含 n 个顶点的森林；第一步，选取权值最小的边(b, f)加入最小生成树；第二步，剩余边中权值最小的边为(b, d)，加入最小生成树，第二步操作后权值最小的边(d, f)不能选，因为会与之前已选取的边形成回路；接下来依次选取权值 9、10、11 对应的边加入最小生成树，此时 6 个顶点形成了一棵树，最小生成树构造完成。按照上述过程，加到最小生成树的边依次为(b, f), (b, d), (a, e), (c, e), (b, e)。其生成过程如下所示。



41. B

根据题干所提供的信息可知：

①图 G 为有向无环图，因此一定存在拓扑序列和逆拓扑序列。

②DFS 的性质是顶点 v_i 的所有后继顶点 v_j 出栈后，v_i 才会出栈。

③本题要求执行输出语句后立刻退出递归，即执行完输出语句后立即出栈，因此后入栈的

配套课程公众号[考机研]
永久联系微信:4550060

顶点先输出，结合②不难得出只有输出顶点 v_i 的所有后继顶点 v_j 后，v_i 才会输出。

综合上述分析，输出的顶点序列是逆拓扑有序序列。

42．B

关键路径是指权值之和最大而非边数最多的路径，选项 A 错误。选项 B 是关键路径的概念。无论是存在一条还是存在多条关键路径，增加任意一个关键活动的时间都会延长工程的工期，因为关键路径始终是权值之和最大的那条路径，选项 C 错误。仅有一条关键路径时，减少关键活动的时间会缩短工程的工期；存在多条关键路径时，缩短一条关键活动的时间不一定会缩短工程的工期，缩短了路径长度的那条关键路径不一定还是关键路径，选项 D 错误。

43．A

求拓扑序列的过程：从图中选择无入边的结点，输出该结点并删除该结点的所有出边，重复上述过程，直至全部结点都已输出，这样求得的拓扑序列为 *ABCDEF*。每次输出一个结点并删除该结点的所有出边后，都发现有且仅有一个结点无入边，因此该拓扑序列唯一。

44．C

在执行 Dijkstra 算法时，首先初始化 dist[]，若顶点 1 到顶点 i（i = 2, 3, 4, 5）有边，就初始化为边的权值；若无边，就初始化为∞；初始化顶点集 S 只含顶点 1。Dijkstra 算法每次选择一个到顶点 1 距离最近的顶点 j 加入顶点集 S，并判断由顶点 1 绕行顶点 j 后到任意一个顶点 k 是否距离更短，若距离更短（dist[j] + arcs[j][k] < dist[k]），则将 dist[x]更新为 dist[j] + arcs[j][k]；重复该过程，直至所有顶点都加入顶点集 S。数组 dist 的变化过程如下图所示，可知将第二个顶点 5 加入顶点集 S 后，数组 dist 更新为 21, 3, 14, 6。

45．B

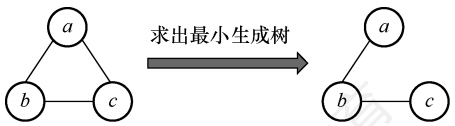
在 AOE 网中，活动的时间余量 = 结束顶点的最迟开始时间 − 开始顶点的最早开始时间 − 该活动的持续时间。根据关键路径算法得到下表：

结点编号	1	2	3	4	5	6
最早开始时间 $v_e(i)$	0	2	5	8	9	12
最迟开始时间 $v_l(i)$	0	4	5	8	11	12

c 的时间余量 = $v_l(3) - v_e(2) - 1 = 5 - 2 - 1 = 2$，$g$ 的时间余量 = $v_l(6) - v_e(3) - 1 = 12 - 5 - 1 = 6$，$h$ 的时间余量 = $v_l(5) - v_e(4) - 1 = 11 - 8 - 1 = 2$，$j$ 的时间余量 = $v_l(6) - v_e(5) - 1 = 12 - 9 - 1 = 2$。

46．B

Prim 算法和 Kruskal 算法用于求解最小生成树，最小生成树中某顶点到其余各顶点的路径不一定具有最短路径的性质。例如，在下图所求得的最小生成树中，a 到 c 的路径长度为 2，但原图中 a 到 c 的最短路径长度为 1。



图的广度优先搜索算法总按距离由近到远来遍历图中的每个顶点，因此可用来求解非带权图（或各边权值均相同）的单源最短路径问题。

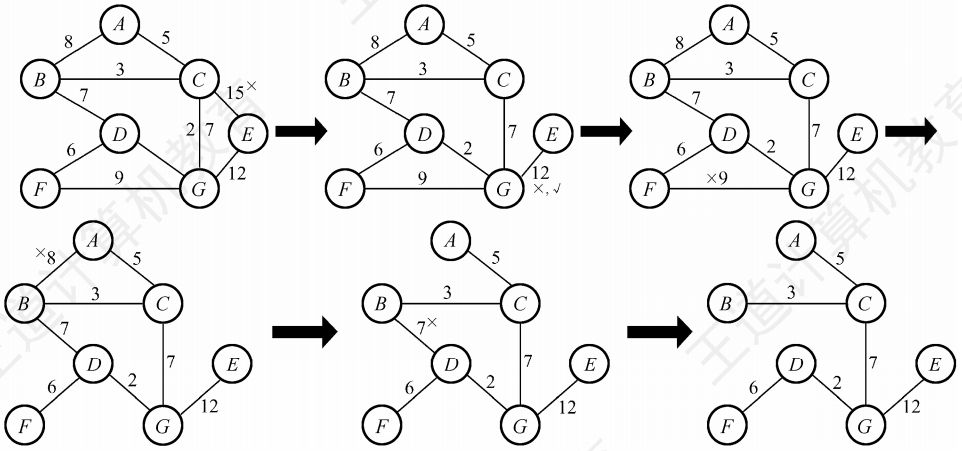
二、综合应用题

01．【解答】

这种方法是正确的。

配套课程公众号[考机研]
永久联系微信:4550060

经过“破圈法”之后，最终没有回路，因此一定可以构造出一棵生成树。下面证明这棵生成树是最小生成树。记“破圈法”生成的树为 T，假设 T 不是最小生成树，则必然存在最小生成树 T_0，使得它与 T 的公共边尽可能多，则将 T_0 与 T 取并集，得到一个图，此图中必然存在回路，“破圈法”的定义就是从回路中去除权最大的边，因此此时生成的 T 的权必然是最小的，这与原假设矛盾，从而 T 是最小生成树。下图说明了“破圈法”的过程：



02.【解答】

1）该图的邻接矩阵为

$$A=\begin{array}{c} \\ 1\\2\\3\\4\\5\\6\\7 \end{array}\begin{array}{c} \begin{array}{ccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{array} \\ \begin{bmatrix} 0 & 3 & 3 & 6 & \infty & \infty & \infty \\ \infty & 0 & 4 & \infty & 5 & \infty & \infty \\ \infty & \infty & 0 & \infty & 4 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & 5 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & 3 \\ \infty & \infty & 3 & \infty & \infty & 0 & 7 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix} \end{array}$$

得到的深度优先遍历序列为 1, 2, 3, 5, 7, 4, 6。

2）解题思路：当某个顶点只有出弧而没有入弧时，其他顶点无法到达这个顶点，不可能与其他顶点和边构成强连通分量（这个单独的顶点构成一个强连通分量）。

① 顶点 1 无入弧构成第一个强连通分量。删除顶点 1 及所有以之为尾的弧。

② 顶点 2 无入弧构成一个强连通分量。删除顶点 2 及所有以之为尾的弧。

③ ……

以此类推，最后得到每个顶点都是一个强连通分量，所以强连通分量数目为 7。

3）该图的两个拓扑序列如下：

① 1, 2, 4, 6, 3, 5, 7

② 1, 4, 2, 6, 3, 5, 7

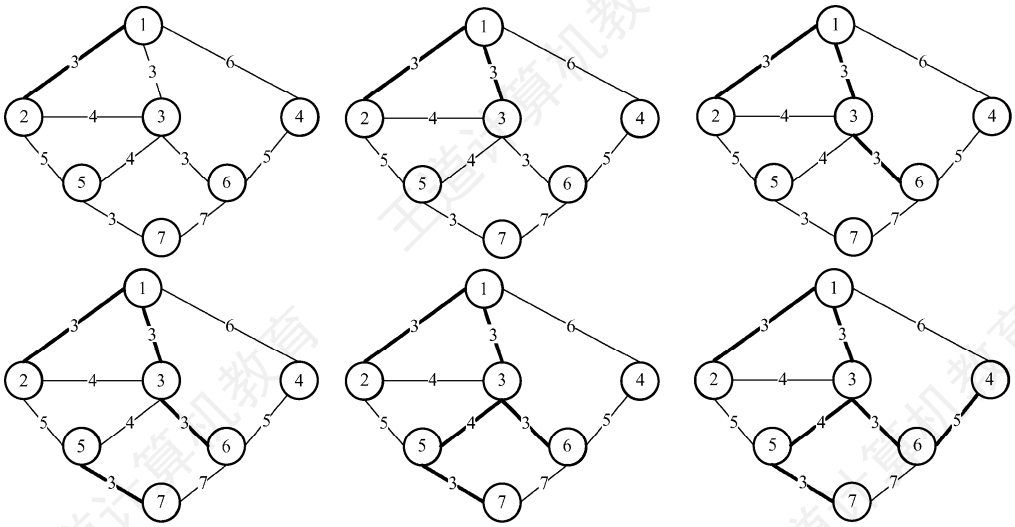
4）若视该图为无向图：

用 Prim 算法生成最小生成树的过程如下：

1—2，1—3，3—6，3—5，5—7，6—4（图略）。

用 Kruskal 算法生成最小生成树的过程如下图所示。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



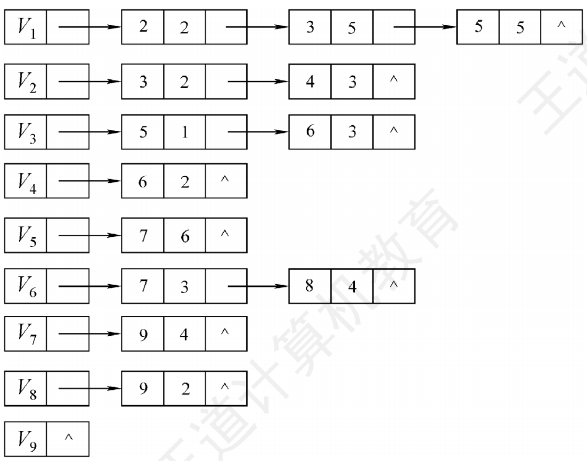
03.【解答】

根据 Dijkstra 算法，求从顶点 1 到其余各顶点的最短路径如下表所示。

顶点	从顶点 1 到各终点的 dist 值				
	第 1 轮	第 2 轮	第 3 轮	第 4 轮	
2	**7** **v_1, v_2**				
3	11 v_1, v_3	**11** **v_1, v_3**			
4	∞	16 v_1, v_2, v_4	**16** **v_1, v_2, v_4**		
5	∞	∞	18 v_1, v_3, v_5	**18** **v_1, v_3, v_5**	
6	∞	∞	19 v_1, v_3, v_6	19 v_1, v_3, v_6	**19** **v_1, v_3, v_6**
集合 S	**{1, 2}**	**{1, 2, 3}**	**{1, 2, 3, 4}**	**{1, 2, 3, 4, 5}**	**{1, 2, 3, 4, 5, 6}**

04.【解答】

1）该图的邻接表表示如下图所示。



王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

求关键路径的算法如下：

① 输入 e 条弧<j, k>，建立 AOE 网的存储结构。

② 从源点 v_1 出发，令 $v_e(1)=0$，求 $v_e(j)$，$2 \leqslant j \leqslant n$。

③ 从汇点 v_n 出发，令 $v_l(n)=v_e(n)$，求 $v_l(i)$，$1 \leqslant i \leqslant n-1$。

④ 根据各顶点的 v_e 和 v_1 值，求每条弧 s（活动）的最早开始时间 $e(s)$和最晚开始时间 $l(s)$，其中 $e(s)=l(s)$为关键活动。

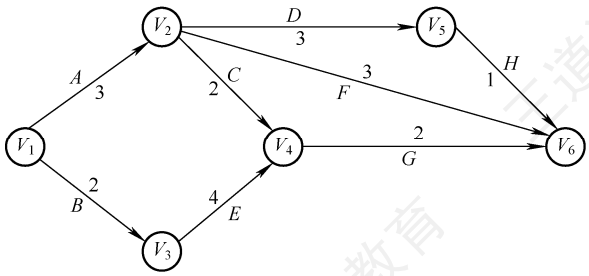
2）根据以上算法可以得到至少需要时间 16。

3）关键路径为(V_1, V_3, V_5, V_7, V_9)。

4）活动 a_2, a_6, a_9, a_{12} 加速，可以缩短工程所需的时间。

05.【解答】

1）根据题表可以画出 AOE 网如下图所示。



求解各事件和活动的最早发生时间与最迟发生时间公式分别如下：

① v_e(源点) = 0，$v_e(k)$ = Max{$v_e(j)$ + Weight(v_j, v_k)}，Weight(v_j, v_k)表示从 v_j 指向 v_k 的弧的权值。

② v_l(汇点) = v_e(汇点)，$v_l(j)$ = Min{$v_l(k)$ − Weight(v_j, v_k)}，Weight(v_j, v_k)表示从 v_j 指向 v_k 的弧的权值。

③ 若边<v_k, v_j>表示活动 a_i，则有 $e(i)=v_e(k)$。

④ 若边<v_k, v_j>表示活动 a_i，则有 $l(i)=v_l(j)-\text{Weight}(v_k, v_j)$。

⑤ $d(i)=l(i)-e(i)$。

关键路径即由 $d(i)=0$ 的 i 构成。

2）根据上述公式，各事件的最早发生时间 v_e 和最迟发生时间 v_l 如下表所示。

	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	3	2	6	6	8
$v_l(i)$	0	4	2	6	7	8

3）根据上述公式，各活动最早发生时间 e、最迟发生时间 l 和时间余量 $d(i)=l(i)-e(i)$如下表所示。

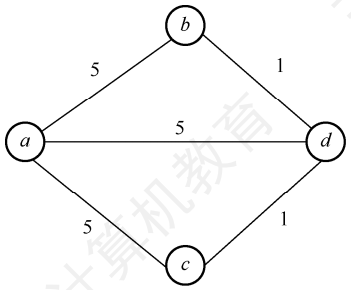
	A	B	C	D	E	F	G	H
$e(i)$	0	0	3	3	2	3	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	2	0	1

所以关键路径为 B、E、G，完成该工程最少需要 8（单位依题意而定）。

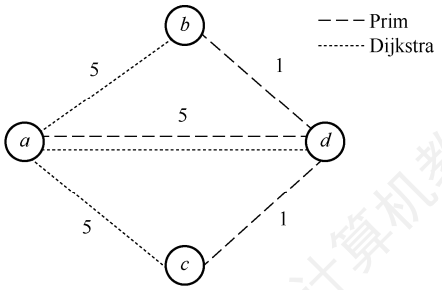
配套课程公众号[考机研]
永久联系微信:4550060

06.【解答】

Dijkstra 算法每一步都会贪婪地选择与源点 v_0 最近的下一条边，直到 v_0 连接到图中所有顶点。Prim 算法（已知是最小生成树算法）与 Dijkstra 算法高度相似，但是在每个阶段，它贪婪地选择与该阶段已加入 MST 中任意一个顶点最近的下一条边。显然，Dijkstra 算法可以产生一棵生成树，但该树不一定是最小生成树，只需举出一个反例即可，以下图 G 为例（将 a 作为源点）。



(a) 图 G



(b) 两种算法产生的生成树

Dijkstra 算法得到的路径集合为$\{(a, b), (a, c), (a, d)\}$，该生成树的总权值为 5 + 5 + 5 = 15。

Prim 算法得到的边集合为$\{(a, d), (b, d), (c, d)\}$，该最小生成树的总权值为 5 +1 + 1 = 7。

显然，Dijkstra 算法得到的生成树不一定是最小生成树。

07.【解答】

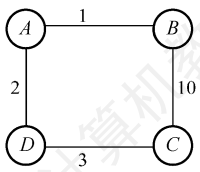
本节前面给出了 DFS 实现拓扑排序的思想，下面是利用 DFS 求各顶点结束时间的代码（在 DFS 的基础上加入了 time 变量）。将结束时间从大到小排序，即可得到拓扑序列。

```
bool visited[MAX_VERTEX_NUM];              //访问标记数组
void DFSTraverse(Graph G){
    for(v=0;v<G.vexnum;++v)
        visited[v]=FALSE;                  //初始化访问标记数组
    time=0;
    for(v=0;v<G.vexnum;++v)                //本代码从 v=0 开始遍历
        if(!visited[v]) DFS(G,v);
}
void DFS(Graph G,int v){
    visited[v]=TRUE;
    visit(v);
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
        if(!visited[w]){                   //w 为 v 的尚未访问的邻接点
            DFS(G,w);
        }
    time=time+1;finishTime[v]=time;
}
```

08.【解答】

该方法不一定能（或不能）求得最短路径。

例如，对于下图所示的带权图，若按照题中的原则，从 A 到 C 的最短路径是 $A\rightarrow B\rightarrow C$，事实上其最短路径是 $A\rightarrow D\rightarrow C$。

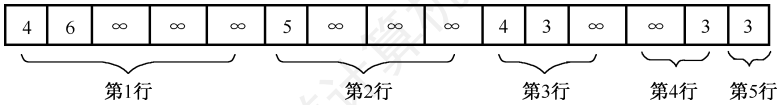


09.【解答】

1）在上三角矩阵 A[6][6]中，第 1 行至第 5 行主对角线上方的元素个数分别为 5, 4, 3, 2, 1，

配套课程公众号[考机研]
永久联系微信:4550060

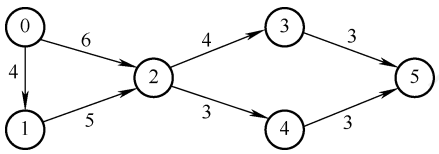
由此可以画出压缩存储数组中的元素所属行的情况，如下图所示。



可以用“平移”的思想，将前5个、后4个、后3个、后2个、后1个元素，分别移动到矩阵对角线（“0”）右边的行上。图 G 的邻接矩阵 $\boldsymbol{A}$ 为

$$\boldsymbol{A}=\begin{bmatrix}0 & 4 & 6 & \infty & \infty & \infty\\ \infty & 0 & 5 & \infty & \infty & \infty\\ \infty & \infty & 0 & 4 & 3 & \infty\\ \infty & \infty & \infty & 0 & \infty & 3\\ \infty & \infty & \infty & \infty & 0 & 3\\ \infty & \infty & \infty & \infty & \infty & 0\end{bmatrix}$$

2）有向带权图 G 如下图所示。



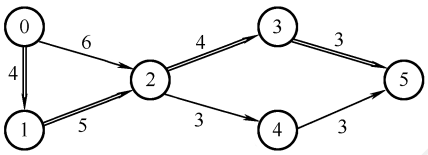
3）先计算各个事件的最早发生时间，得到 $v_e()$和 $v_l()$数组如下表所示。

i	0	1	2	3	4	5
$v_e(i)$	0	4	9	13	12	16
$v_l(i)$	0	4	9	13	13	16

接下来计算所有活动的最早和最迟发生时间 $e()$和 $l()$，如下表所示。

	$a_{0\to1}$	$a_{0\to2}$	$a_{1\to2}$	$a_{2\to3}$	$a_{2\to4}$	$a_{3\to5}$	$a_{4\to5}$
$e()$	0	0	4	9	9	13	12
$l()$	0	3	4	9	10	13	13
$l()-e()$	0	3	0	0	1	0	1

满足 $l()-e()=0$ 的路径就是关键路径，所以关键路径为 $a_{0\to1}$, $a_{1\to2}$, $a_{2\to3}$, $a_{3\to5}$，如下图所示（双线箭头表示），长度为 $4+5+4+3=16$。



按求关键路径的公式计算较为复杂，建议考生面临此类题时直接穷举各条路径即可。

10.【解答】

本题初看起来感觉难度较大，但仔细分析就可发现考查的其实是邻接表的数据结构。

1）图题中给出的是一个简单的网络拓扑图，可以抽象为无向图。

2）图的常用存储结构有邻接矩阵法和邻接表法，其中邻接表法属于链式存储结构，因此本题的基本思路就是写出邻接表的数据类型定义，并根据题意调整相应的边表结点和顶点表结点的成员变量。具体分析如下：邻接表由表头顶点和弧顶点组成。根据题目给出的图和表，可将顶点分为三类：路由器、网络和链路。路由器是连接网络和链路的载体，因此可将它作为表头顶点。网络和链路则是连接路由器的边，因此可将它们作为弧顶

配套课程公众号[考机研]
永久联系微信:4550060

点。为了简化代码，可将网络和链路的结构合并为一个，用一个标志位来区分它们，这样就可用邻接表来实现图的存储。链式存储结构如下图所示。

弧结点的两种基本形态

Flag=1	Next
ID	
IP	
Metric	

Flag=2	Next
Prefix	
Mask	
Metric	

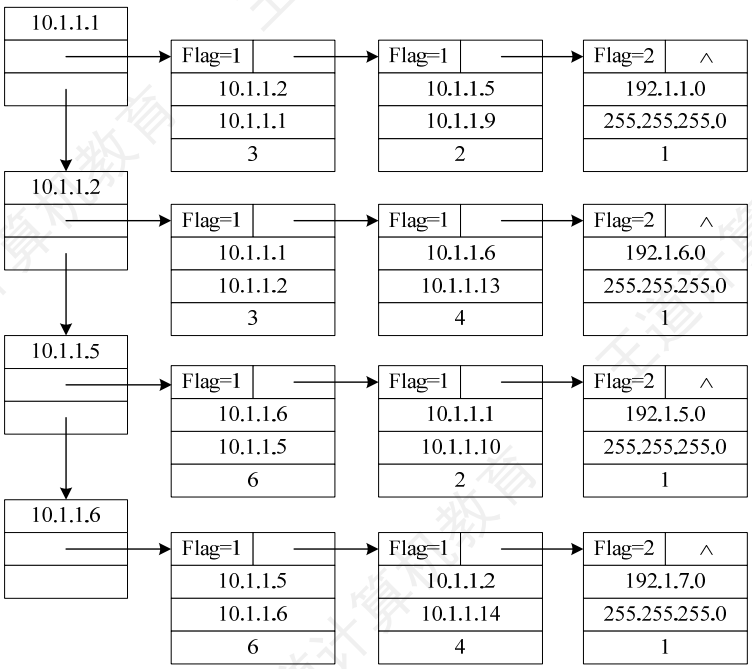
表头结点结构示意

RouterID
LN_link
Next

其数据类型定义如下：

```
typedef struct{
unsigned int ID, IP;
}LinkNode;                  //Link 的结构
typedef struct{
unsigned int Prefix, Mask;
}NetNode;                   //Net 的结构
typedef struct Node{
int Flag;                   //Flag=1 为 Link;Flag=2 为 Net
union{
    LinkNode Lnode;
    NetNode Nnode
}LinkORNet;
Unsigned int Metric;
struct Node *next;
}ArcNode;                   //弧结点
typedef struct hNode{
unsigned int RouterID;
ArcNode *LN_link;
Struct hNode *next;
}HNODE;                     //表头结点
```

对应表的链式存储结构示意图如下所示。



3）计算结果如下表所示。

配套课程公众号[考机研]
永久联系微信:4550060

	目的网络	路　径	代价（费用）
步骤1	192.1.1.0/24	直接到达	1
步骤2	192.1.5.0/24	R1→R3→192.1.5.0/24	3
步骤3	192.1.6.0/24	R1→R2→192.1.6.0/24	4
步骤4	192.1.7.0/24	R1→R2→R4→192.1.7.0/24	8

11.【解答】

1）Prim 算法属于贪心策略。算法从一个任意的顶点开始，一直长大到覆盖图中的所有顶点为止。算法的每一步在连接树集合 S 的顶点和其他顶点的边中，选择一条使得树的总权重增加最小的边加入集合 S。当算法终止时，S 就是最小生成树。

① S 中顶点为 A，候选边为$(A, D), (A, B), (A, E)$，选择(A, D)加入 S。

② S 中顶点为 A, D，候选边为$(A, B), (A, E), (D, E), (C, D)$，选择$(D, E)$，加入 S。

③ S 中顶点为 A, D, E，候选边为$(A, B), (C, D), (C, E)$，选择(C, E)加入 S。

④ S 中顶点为 A, D, E, C，候选边为$(A, B), (B, C)$，选择(B, C)加入 S。

⑤ S 就是最小生成树。

依次选出的边为

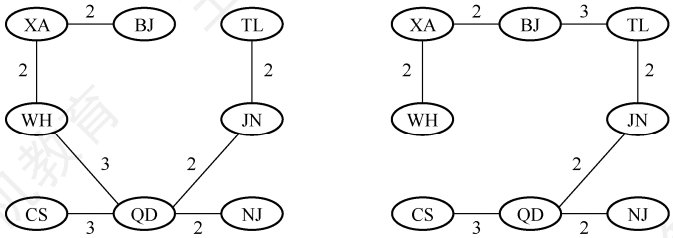
$$(A, D), (D, E), (C, E), (B, C)$$

2）图 G 的 MST 是唯一的。第一小题的最小生成树包括了图中权值最小的 4 条边，其他边除(A, E)外都比这 4 条边大，但若用(A, E)替换同权值的(C, E)，A, D, E 三个顶点构成了回路，因此不能替换，所以此图的 MST 唯一。

3）当带权连通图的任意一个环中所包含的边的权值均不相同时，其 MST 是唯一的。此题不要求回答充分必要条件，所以回答一个限制边权值的充分条件即可。

12.【解答】

1）为了求解最经济的方案，可把问题抽象为求无向带权图的最小生成树。可以采用手动 Prim 算法或 Kruskal 算法作图。注意本题的最小生成树有两种构造，如下图所示。



方案的总费用为 16。

2）存储题中的图可采用邻接矩阵（或邻接表）。构造最小生成树采用 Prim 算法（或 Kruskal 算法）。

3）TTL = 5，即 IP 分组的生存时间（最大传递距离）为 5，方案 1 中 TL 和 BJ 的距离过远，TTL = 5 不足以让 IP 分组从 H1 传送到 H2，因此 H2 不能收到 IP 分组。而方案 2 中 TL 和 BJ 邻近，H2 可以收到 IP 分组。

13.【解答】

1）算法的基本设计思想：建立图 G 各顶点的入度表 `degree[]`。选择入度为 0 的顶点 v，将 v 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 `G.numVertices` 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或者存在多个拓扑序列，返回 0。

配套课程公众号[考机研]
永久联系微信:4550060

2）用 C 语言描述的算法如下：

```
int uniquely(MGraph G){//判定有向图是否存在唯一的拓扑序列
    int *degree,i,j,count=0,in0=-1,prev_in0;
    degree=(int*)malloc(G.numVertices*sizeof(int));
    for(j=0;j<G.numVertices;j++){//计算各顶点的入度
        degree[j]=0;
        for(i=0;i<G.numVertices;i++)
            degree[j]+=G.Edge[i][j];
        if(degree[j]==0){
            if(in0==-1) in0=j;          //入度为 0 的顶点
            else in0=-2;                //有多个入度为 0 的顶点
        }
    }
    while(in0>=0){
        count++;
        prev_in0=in0;
        in0=-1;
        for(j=0;j<G.numVertices;j++)
            if(G.Edge[prev_in0][j]>0){
                if(--degree[j]==0){     //邻接点入度值减 1
                    if(in0==-1) in0=j; //入度为 0 的顶点
                    else in0=-2;        //有多个入度为 0 的顶点
                }
            }
    }
    free(degree);
    if(count==G.numVertices) return 1;
    else return 0;
}
```

王道计算机教育 www.cskaoyan.com

归纳总结

1．关于图的基本操作

本章中的很多程序对采用邻接表或邻接矩阵的存储结构都适用，主要原因是在图的基本操作函数中保持了相同的参数和返回值，而封闭了内部实现细节。

例如，取 x 邻接顶点 y 的下一个邻接顶点的函数 NextNeighbor(G,x,y)。

1）用邻接矩阵作为存储结构

```
int NextNeighbor(MGraph& G,int x,int y){
    if(x!=-1 && y!=-1){
        for(int col=y+1;col<G.vexnum;col++)
            if(G.Edge[x][col]>0 && G.Edge[x][col]<maxWeight)
return col;             //maxWeight 代表∞
    }
    return -1;
}
```

2）用邻接表作为存储结构

```
int NextNeighbor(ALGraph& G,int x,int y){
    if(x!=-1){                                  //顶点 x 存在
        ArcNode *p=G.vertices[x].first;         //对应边链表第一个边结点
        while(p!=NULL && p->data!=y)            //寻找邻接顶点 y
            p=p->next;
        if(p!=NULL && p->next!=NULL)
```

配套课程公众号[考机研]
永久联系微信:4550060

```
                return p->next->data;          //返回下一个邻接顶点
            }
            return -1;
        }
```

2. 关于图的遍历、连通性、生成树、关键路径的几个要点

1）在执行图的遍历时，因为图中可能存在回路，且图的任意一个顶点都可能与其他顶点相连，所以在访问完某个顶点后可能会沿某些边又回到了曾经访问过的顶点。因此，需要设置一个辅助数组visited[]标记顶点是否已被访问过，避免重复访问。

2）深度优先搜索时利用回溯法对图遍历，一般利用递归方法实现，每当向前递归查找某一邻接结点之前，必须判断该结点是否访问过。另外，递归算法均可借助栈来实现非递归算法，深度优先搜索也不例外，具体程序见6.3.4节的综合应用题03。

3）广度优先搜索是一种分层的遍历过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。因此，它不是一个递归的过程。

4）一个给定的图的邻接矩阵表示是唯一的，但对于邻接表来说，若边的输入先后次序不同，则生成的邻接表表示也不同。

5）图的最小生成树首先必须是带权连通图，其次要在 n 个顶点的图中选择 $n-1$ 条边将其连通，使得其权值总和达到最小，且不出现回路。

6）加速某一关键活动不一定能缩短整个工程的工期，因为AOE网中可能存在多条关键路径。可能存在称为桥的一种特殊关键活动，它位于所有的关键路径上，只有它加速才会缩短整个工期。

王道计算机教育 www.cskaoyan.com

思维拓展

【网易有道笔试题】求一个无向连通图的割点。割点的定义是，若除去此结点和与其相关的边，无向图不再连通，描述算法。

提　示

要判断一个点是否为割点，最简单直接的方法是，先把这个点和所有与它相关的边从图中去掉，再用深搜或广搜来判断剩下的图的连通性，这种方法适合判断给定结点是否为割点；还有一种比较复杂的方法可以快速找出所有割点，有兴趣的读者可自行搜索相关资料。

配套课程公众号[考机研]
永久联系微信:4550060